

Framework/Facilitator/STFSTWizard.PRW

4073 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "STFSTWizard.ch"
2 #INCLUDE "TCBROWSE.CH"
3 #include "PROTHEUS.CH"
4 #INCLUDE "apwizard.ch"
5 #INCLUDE "POSCSS.CH"
6
7
8 #DEFINE X3_USADO_EMUSO      "EEEEEEEEEEEEEE "
9 #DEFINE X3_USADO_NAOUSADO  "EEEEEEEEEEEEEEEE"
10
11
12 Static oStepWiz           := Nil //Objto tipo Wizard
13 Static oNewPag            := Nil //Objeto que adiciona nova pagina no wizard
14 Static cL1_DOC            := "" //Numero do Documento NF-e
15 Static cL1_SERIE          := "" //Numero da Serie do documento NF-e
16
17 //-----
18 /*{Protheus.doc} STFStWizard
19 Wizard de Estação
20 @return NIL
21 @obs
22 @sample
23 */
24 //-----
25 Function STFStWizard()
26 Local oDlg                := NIL //Objeto Wizard
27 Local aItens              := NIL //Itens do Wizard
28 Local cLg_Codigo          := SLG->(CriarVar("LG_CODIGO")) //codigo da Estação
29 Local cLog                := "" //Variável de Log
30 Local aSize               := MsAdvSize(.F.) //Dimensões da Janela
31 Local aCoord              := {aSize[7],0 , aSize[6],aSize[5] } //Coordenadas da Janela
32 Local cImpFisc            := CriarVar("LG_IMPFIsc") //Nome da Impressãra Fiscal
33 Local cPorta              := CriarVar("LG_PORTIF") //Porta da Impressora Fiscal
34 Local lConecta            := .F. //ECF Conectado?
35 Local cEstAnt             := "" //Estação Anterior
36 Local lCaixaECF           := nModulo == 12 .AND. IsCaixaLoja(xNumCaixa()) .AND. lFiscal //Caixa é Fiscal?
37 Local aRet               := {}
38
39   STFStrategyECF()
40
41   If lCaixaECF
42     LjCloseDevices()
43   EndIf
44
45
46   oDlg := ApWizard():New( STR0001 , ,STR0002 , STR0003 ,{ || .T. } , { || IIF(lCaixaECF,LjCloseDevices(), STWCcloseDevice()), oDlg := FreeObj(oDlg)} , .T. , , { || .t.
47   } , .F. , aCoord ) //"Apresentação"###"Wizard da Estação"###"Este Wizard fornece passo-a-passo para os itens da estação a serem configurados"
48   LoadPanels( @oDlg, @cLg_CODIGO, @aItens, 1, ;
49               @cLog, @cImpFisc, @cPorta, @lConecta)
49
50   oDlg:Activate ( .t. , { || IIF(lCaixaECF,LjCloseDevices(), STWCcloseDevice()), cEstAnt := cEstacao }, { || .t. }, { || .t.} )
51

```

```

52
53     cEstacao := cEstAnt
54
55     If lConecta
56         aRet := STFFireEvent(      ProcName(0)                ,;      // Nome do processo
57                                 "STCloseCom"                  ,;      // Nome do evento
58                                 {cPorta } )
59
60
61     EndIf
62
63     If lCaixaECF
64         LjOpenDevices()
65     EndIf
66
67 Return
68 //-----
69 /*{Protheus.doc} LoadPanels
70 Carrega os Painéis do Wizard
71 @param cCargo  Nó
72 @param oTree   Arvore
73 @param aDados  Array dos nós
74 @param cIdAmb  Código do Ambiente
75 @param oPanel  Painei
76 @param oDlg    Janela
77 @param cLog     Log
78 @param cImpFisc Impressora Fiscal
79 @param cPorta   Porta de Comunicação
80 @param lConecta Im'ressora Conectada
81 @return  NIL
82 @obs
83 @sample
84 */
85 //-----
86 Static Function LoadPanels(oDlg, cLG_CODIGO, aItens, nPanelAtu, ;
87                           cLog, cImpFisc,      cPorta, lConecta)
88
89 Local nI      := 0      //Variável contadora
90 Local nInicial := 0      //Panel Inicial
91 Local nFinal   := 0      //Panel Final
92
93 DEFAULT oDlg := NIL
94 DEFAULT cLG_CODIGO := SLG->(CriaVar("LG_CODIGO"))
95 DEFAULT aItens := {}
96 DEFAULT cLog := ""
97 DEFAULT cImpFisc := ""
98 DEFAULT cPorta := ""
99 DEFAULT lConecta := .F.
100
101
102 //2
103     nPanelAtu++

```

```

104 oDlg:NewPanel ( "Estação", STR0004 , { || .F. } , { || !Empty(cLg_CODIGO) } , { || .t. } ,.T., { || CriaCompestat(oDlg, @cLG_CODIGO)} ) //"Informe o código da Estação"
105 //3
106 nPanelAtu++
107 oDlg:NewPanel ( STR0005, STR0006, { || .T. } , { || .t. } , { || .t. } ,.T., { || cEstacao := cLG_CODIGO, SelectItens(oDlg, @cLG_CODIGO, @aItens, @cLog)} ) //"Item a
    configurar"###"Selecione o item a ser configurado/validado"
108
109 nInicial := nPanelAtu
110 aItens := LoadItens(nPanelAtu, @oDlg, @cLG_CODIGO, @cLog, ;
111     @cImpFisc, @cPorta , @lConecta)
112
113 If Len(aItens) > 0
114     nFinal := aTail(aItens)[5]
115 EndIf
116
117 For nI := 1 to Len(aItens)
118     oDlg:NewPanel( aItens[nI, 02], , { || .F. } , { || .t. } , { || .t. } ,.T., &("{ || STFStWzLoadPanel(aItens, " + Str(nI) + ", oDlg, cLg_CODIGO, @cImpFisc, @cPorta,
        @lConecta, @cLog)}"))
119     LoadDetItens( aItens, nI, oDlg, @cLG_CODIGO, @cLog,@cImpFisc, @cPorta, @lConecta)
120 Next
121
122 oDlg:NewPanel ( STR0007, STR0008, { || .F. } , { || .T. } , { || .t. } ,.T., { || Exibelog(oDlg, oDlg:nPanel, cLog) } ) //"Log"###"Log de atividades realizadas"
123
124 Return
125
126 //-----
127 /*/{Protheus.doc} CriaCompestat
128 Carrega o código da Estação
129 @param oDlg Janela
130 @param cLG_CODIGO Codigo da Estação
131 @param cLog Log
132 @return NIL
133 @obs
134 @sample
135 */
136 //-----
137 Static Function CriaCompestat(oDlg, cLG_CODIGO, cLog)
138 Local oPanel := NIL //Panel Atual
139 Local nRow := 0 //Linhas do Panel
140 Local nCol := 0 //Colunas do panel
141 Local aObjects := {} //Array de objetos
142 Local aPosObj := {} //Posicao dos objetos
143 Local nTam := 0 //tamanho do controle
144 Local nTamSX3 := SLG->(TamSX3("LG_CODIGO")[1]) //Tamanho do Campo Estação
145 Local aArea := GetArea() //WorkArea Ativa
146 Local aAreaSLG := SLG->(GetArea()) //WorkArea SLG
147
148 DEFAULT oDlg := NIL
149 DEFAULT cLG_CODIGO := SLG->(CriaVar("LG_CODIGO"))
150 DEFAULT cLog := ""
151
152 oPanel := oDlg:GetPanel(oDlg:nPanel)
153 nRow := Int((oPanel:nHeight) / 2) //Linha do Painel

```

```

154     nCol      := Int((oPanel:nWidth ) / 2)   //Coluna do Painel
155
156
157     nTam := CalcFieldSize("C",nTamSX3,0,"@!",STR0009) //"Código da Estação"
158     aAdd(aObjects, { nTam, 6, .F., .F.}) //Impar Say
159     aAdd(aObjects, { nTam, 10, .F., .F.}) //Impar Sa
160
161     aPosObj := MsObjSize( { 0 , 15 , nRow ,nCol, 2, 3,3,3}, aObjects, .T., .F.)
162
163     @aPosObj[01,01],aPosObj[01, 2] SAY STR0009 SIZE nTam,10 PIXEL OF oPanel      //"Código da Estação"
164     @aPosObj[02,01],aPosObj[2, 2] MsGet cLG_CODIGO Size  nTam,10 PIXEL OF oPanel F3 "SLG"  PIXEL Picture "@!" VALID ExistCpo("SLG", cLG_CODIGO, 1)
165
166     RestArea(aAreaSLG)
167     RestArea(aArea)
168 Return
169
170 //-----
171 /*{Protheus.doc} LoadItens
172 Carrega as funcionalidades do Wizard
173 @param nPanelAtu  Panel Atual
174 @param oDlg       Janela
175 @param cLG_CODIGO Codigo da Estação
176 @param cLog       Log
177 @param cImpFisc  Impressora Fiscal
178 @param cPorta    Porta de Comunicação
179 @param lConecta  Im'ressora Conectada
180 @return aItens Componentes a configurar
181 @obs
182 @sample
183 */
184 //-----
185 Static Function LoadItens(nPanelAtu, oDlg, cLG_CODIGO, cLog, cImpFisc, cPorta, lConecta)
186 Local aItens := {}           //Itens do Wizard
187
188 DEFAULT nPanelAtu  := 1      //Panel Atual
189 DEFAULT oDlg       := ""     //Janela
190 DEFAULT cLG_CODIGO := ""     //Codigo da Estação
191 DEFAULT cLog       := ""     //Log da Rotina
192 DEFAULT cImpFisc   := ""     //Impressora Fiscal
193 DEFAULT cPorta     := ""     //Porta de Comunicação
194 DEFAULT lConecta   := .F.    //Conectado
195
196 nQtdPanels := 5 //quantidade de Paineis
197 nPanelAtu++
198 aAdd(aItens, { .T.,; //Habilitado?
199              STR0010,; //Descrição //"Impressora Fiscal"
200              &("{ || STFLdWzIF(oDlg, cLG_CODIGO, " + Str(nPanelAtu) + ", @cLog, aItens[1], @cImpFisc, @cPorta, @lConecta)} ") ,; //Bloco
201              nPanelAtu, ; //Painel Atual
202              nPanelAtu + nQtdPanels, ; //Painel Seguinte
203              "STFLdWzIF(oDlg, cLG_CODIGO, " + Str(nPanelAtu) + ", @cLog, aItens[1], @cImpFisc, @cPorta, @lConecta) ")}) //Bloco Carregar
204
205 nPanelAtu += nQtdPanels

```

```

206     nQtdPanels := 2 //Cadastro e teste da transação
207     aAdd(aItens, {.F., ;
208         STR0011,; //"TEF"
209         &("{ || STFLdWzTEF(oDlG, cLG_CODIGO, " + Str(nPanelAtu) + ", @cLog, aItens[2], lConecta ) } " ),;
210         nPanelAtu, ;
211         nPanelAtu + nQtdPanels, ;
212         "STFLdWzTEF(oDlG, cLG_CODIGO, " + Str(nPanelAtu) + ", @cLog, aItens[2], lConecta)"}))
213
214     Return aItens
215
216 //-----
217 /*{Protheus.doc} LoadDetItens
218 Carrega os paineis do facilitador
219 @param aItens  Funcionalidades do Wizard
220 @param nItem
221 @param oDlG    nI
222 @param cLG_CODIGO  Codigo da Estação
223 @param cLog      Log
224 @param cImpFisc  Impressora Fiscal
225 @param cPorta    Porta de Comunicação
226 @param lConecta  Im'ressora Conectada
227 @return  NIL
228 @obs
229 @sample
230 */
231 //-----
232 Function LoadDetItens(aItens, nItem, oDlG, cLG_CODIGO, ;
233                     cLog, cImpFisc, cPorta, lConecta)
234
235 Local nI          := 0 //Variavel contadora
236 Local nTotItens := 0 //total de itens
237
238 DEFAULT aItens      := {} //Paineis da Funciopnalidade
239 DEFAULT nItem       := 0 //Item da funcionalidade
240 DEFAULT oDlG        := NIL //Janela
241 DEFAULT cLG_CODIGO  := "" //Estação
242 DEFAULT cLog        := "" //Variavel de Log
243 DEFAULT cImpFisc    := "" //Impressora Fiscal
244 DEFAULT cPorta      := "" //Porta de Comunicação
245 DEFAULT lConecta    := .F. //ECF Conectado?
246
247     nTotItens := aItens[nItem, 05] - aItens[nItem, 04]
248     nTotItens--
249
250     Do Case
251     Case nItem == 1
252         For nI := 1 to nTotItens
253             oDlG:NewPanel( aItens[nItem, 02], , { || .F. } , { || .T. } , { || .t. } ,.T., &("{ || "+ aItens[nItem, 06] + "}") )
254             Next
255     Case nItem == 2
256         For nI := 1 to nTotItens
257             oDlG:NewPanel( aItens[nItem, 02], , { || .F. } , { || .T. } , { || .t. } ,.T., &("{ || "+ aItens[nItem, 06] + "}") )

```

```

258         Next
259     Endcase
260
261 Return
262
263 //-----
264 /*{Protheus.doc} SelectItens
265 Selecionar as funcionalidades a serem executadas
266 @param oDlg      nI
267 @param cLG_CODIGO Codigo da Estação
268 @param aItens    Itens do Wizard
269 @param cLog      Log
270 @return  NIL
271 @obs
272 @sample
273 */
274 //-----
275 Static Function SelectItens(oDlg, cLG_CODIGO, aItens, cLog)
276
277 Local nRow      := 0    //Linha
278 Local nCol      := 0    //Coluna
279 Local aObjects  := {}   //Objetos
280 Local nI        := 1    //Contador
281 Local cVar      := ""    //Variavel
282 Local cGet      := ""    //Controle
283 Local cVariable := ""    //Variavel
284
285 DEFAULT aItens      := {}           //Itens da Funcionalidade
286 DEFAULT oDlg        := NIL          //Janela
287 DEFAULT cLG_CODIGO := SLG->(CriaVar("LG_CODIGO"))//Estação
288
289     cLog:= STR0012 + cLG_CODIGO + "]" //" - Configurando dados da estação ["
290
291     oDlg:SetMessage( STR0013+ cLG_CODIGO + "]", oDlg:nPanel , .F. ) //"Selecione o item a ser configurado/validado da Estação ["
292
293     oPanel := oDlg:GetPanel(oDlg:nPanel)
294     nRow   := Int((oPanel:nHeight ) / 2) //Linha do Painel
295     nCol   := Int((oPanel:nWidth ) / 2)  //Coluna do Painel
296     @0,0 to nRow, nCol
297
298     For nI := 1 to Len(aItens)
299
300         nTam := CalcFieldSize("C",Len(aItens[nI,02])+5,0,"",aItens[nI,02])
301         aAdd(aObjects, { nTam, 10, .F., .F.}) //Impar Say
302     Next
303
304     aPosObj := MsObjSize( { 0 , 15 , nRow ,nCol, 2, 3,3,3}, aObjects, .T., .F.)
305
306
307     For nI := 1 to Len(aItens)
308
309         cVariable := "aItens["+Str(nI)+"]{01}"

```

```

310         cGet := "{ |U| Iif(PCOUNT() > 0, "+cVariable+" := U, "+cVariable+" )} "
311
312         TCheckBox():New(aPosObj[nI, 01], aPosObj[nI, 02],aItens[nI, 02],&cGet,oPanel, aObjects[nI, 01],aObjects[nI, 02],,,,,,T.,,,)
313
314     Next
315
316 Return
317
318 //-----
319 /*{Protheus.doc} STFStWzLoadPanel
320 Carrega o Panel
321 @param aItens  Itens da Funcionalidade
322 @param nItens  Item da funcionalidade
323 @param oDlg    Janela
324 @param cLG_CODIGO Codigo da Estação
325 @param cImpFisc Impressora Fiscal
326 @param cPorta  Porta de Comunicação
327 @param lConecta Impressora Conectada
328 @param cLog    Log
329 @return  .T.
330 @obs
331 @sample
332 */
333 //-----
334 Function STFStWzLoadPanel(aItens,  nItens,      oDlg ,      cLG_CODIGO, ;
335                          cImpFisc, cPorta,      lConecta,  cLog)
336
337 DEFAULT aItens      := {}    //Itens da funcionalidade
338 DEFAULT nItens      := 1     //Item da Funcionalidade
339 DEFAULT oDlg        := NIL   //Janela
340 DEFAULT cLG_CODIGO  := ""    //Estação
341 DEFAULT cImpFisc     := ""    //Impressora Fiscal
342 DEFAULT cPorta      := ""    //Porta de Comunicação
343 DEFAULT lConecta     := .F.   //Impressora Conectada
344 DEFAULT cLog        := ""    //Log de Execução
345
346     If !aItens[nItens, 01]
347
348         oDlg:SetPanel(aItens[nItens, 05])
349
350     Else
351         cLog += CRLF + space(5 * 1) + STR0014 + aItens[nItens, 02] + "]" //" - Configurando o item [ "
352         Eval( aItens[nItens, 03])
353     EndIf
354 Return .T.
355
356 //-----
357 /*{Protheus.doc} ExibeLog
358 Exibe o Log de Execução do Wizard
359 @param oDlg    Janela
360 @param nPanel  //Painel
361 @param cLog    Log

```

```

362 @return cLog Log de Execução da Rotina
363 @obs
364 @sample
365 /*/
366 //-----
367 Static Function ExibeLog(oDlg, nPanel, cLog)
368
369 Local aObjects := {} //Objetos da Janela
370 Local aPosObj := {} //Posicionamento de Objetos da Janela
371 Local nRow := 0 //Linhas do Panel
372 Local nCol := 0 //Colunas do panel
373 Local oPanel := NIL //Painel
374 Local nRow1 := 0 //Linha
375
376 DEFAULT oDlg := NIL //Janela
377 DEFAULT nPanel := 1 //Panel
378 DEFAULT cLog := "" //Log
379
380 oPanel := oDlg:GetPanel(nPanel)
381
382 nRow := Int( oPanel:nHeight/2) //Linha do Painel
383 nCol := Int( oPanel:nWidth/ 2)
384
385 oPanel := TScrollBar():New( oPanel, 0,0,nRow,nCol)
386 nCol := Int( nCol *.97 )
387 nRow1 := Int( nRow *.93)
388
389 aObjects := { {CalcFieldSize("C",110,0),nRow1, .F., .F.} }
390 aPosObj2 := MsObjSize( { 0 , 15 , nRow1 ,nCol, 2, 3,3,3}, aObjects, .T., .F.)
391
392 TMultiGet():New(aPosObj2[1, 1],aPosObj2[1, 2],{|| Iif(PCOUNT() > 0, cLog := U,cLog)} ,oPanel,;
393 aObjects[1, 01],aObjects[1, 02],/*oFont*/,.T.,;
394 /*nClrFore*/,/*nClrBack*/,/*oCursor*/,.T.,;
395 /*cMg*/,.T.,/*lCenter*/,;
396 /*lRight*/,.T.,/*bChange*/,;
397 ,.F., .T.)
398 Return cLog
399
400
401 //-----
402 /*{Protheus.doc} STFLdWzIF
403 Paineis do Wizard da Impressora Fiscal
404 @param oDlg Janela
405 @param cLG_CODIGO Codigo da Estacao
406 @param nPanelIni Panel Inicial do Wizard
407 @param cLog Log
408 @param aItem Item da funcionalidade
409 @param cImpFisc Impressora Fiscal
410 @param cPorta Porta de Comunicação
411 @param lConecta Impressora Conectada
412 @obs
413 @sample

```

```

414 /*/
415 //-----
416 Function STFLdWzIF(oDlg,    cLG_CODIGO, nPanelIni,  cLog, ;
417                    aItem,  cImpFisc,   cPorta,    lConecta)
418
419 Local nItem      := 0    //Item do Painei
420 Local nPanel     := 0    //Painel atual
421 Local aImpFisc   := {}   //Impressora Fiscal
422 Local aPortas    := {}   //Porta de Comunicação
423 Local lRet       := .T.  //Retorno
424 Local cPDV       := CriarVar("LG_PDV")    //PDV
425 Local cTamSer    := CriarVar("LG_TAMSER") //Tamanho do número de série C00
426 Local cSerie     := CriarVar("LG_SERPDV") //Numero de Fabricação
427 Local aAliqICM   := {}   //Alíquotas de ICMS
428 Local aAliqISS   := {}   //Alíquotas de ISS
429 Local aAliqECF   := {}   //Alíquotas do ECF
430 Local aAliqPtr   := {}   //Alíquotas do Protheus
431 Local aRet       := {}   //Retorno da rotina
432 Local aTotPtr    := {}   //Totalizadores do Protheus
433 Local aTotNFisc  := {}   //Totalizadores não-fiscais
434 Local cImpFiscST:= ""    //Impressora da Estação
435 Local cPortaST  := ""    //Porta da Estação
436 Local aFormaECF := {}   //Formas de pagamentos do ECF
437 Local aFormaPtr := {}   //Formas de pagamento do Protheus
438 Local lTotvsApi := .F.   //ECF cadastrado no totvsApi?
439 Local cMessage  := ""    //Mensagem
440 Local cMessage2 := ""    //Segunda Mensagem
441 Local nEspaco   := 5     //Espaço
442 Local nNivel    := 2     //Nível
443
444 DEFAULT oDlg      := NIL //Janela
445 DEFAULT cLG_CODIGO:= ""   //Estação
446 DEFAULT nPanelIni := 0    //Painel Inicial
447 DEFAULT cLog      := ""   //Log da Rotina
448 DEFAULT aItem     := {}   //Paineis da Rotina
449 DEFAULT cImpFisc  := ""   //Impressora Fiscal
450 DEFAULT cPorta    := ""   //Porta de Comunicação
451 DEFAULT lConecta  := .F.  //Impressora Conectada
452
453 nPanel := oDlg:nPanel
454
455 nItem := (nPanel - nPanelIni) + 1
456
457 cMessage2 := STR0015+ cLG_CODIGO + "]"  //" da Estação ["
458 Do Case
459     Case nItem == 1
460
461
462         cMessage := STR0016 + aItem[2] //"Configuração dos Dados da "
463
464         cLog += CRLF + space(nEspaco * nNivel) + " - " + cMessage
465         nNivel++

```

```

466
467     lRet := STFWzDtIF(cLG_CODIGO, @cImpFisc, @cPorta, @cPDV, ;
468                     @cTamSer, @aImpFisc, @aPortas, @lConecta, ;
469                     @cSerie, @cLog, (nEspaco * nNivel))
470
471     //Carrega a Tela
472     If lRet
473         STFWzLdIF(cLG_CODIGO, @cImpFisc, @cPorta, @cPDV, ;
474                 @cTamSer, aImpFisc, aPortas, oDlg, ;
475                 nPanel, @lConecta, @cSerie,@cLog, ;
476                 (nEspaco * nNivel))
477
478         //So avança se tiver gravado os dados
479
480         If !lRet
481             oDlg:SetPanel(aItem[5])
482         EndIf
483
484     Else
485
486         oDlg:SetPanel(aItem[5])
487     EndIf
488     Case nItem == 2
489         cMessage := cMessage + STR0017 + aItem[2] //"Programando Alíquotas na "
490
491         cLog += CRLF + space(nEspaco * nNivel) + " - " + cMessage
492         nNivel++
493
494         cImpFiscST := STFFGetStat("IMPFISC")
495         cPortaST := STFFGetStat("PORTIF")
496
497         If ( cImpFisc <> cImpFiscSt .OR. cPorta <> cPortaSt)
498
499             If lConecta
500                 //Fecha o ECF
501                 aRet := STFFireEvent(      ProcName(0)
502                                     "STCcloseCom"
503                                     {cPorta } )
504                                     ,;      // Nome do processo
505                                     ,;      // Nome do evento
506
507                 lRet := Len(aRet) > 0 .AND. aRet[1] == 0
508                 lConecta := !lRet
509
510                 cLog += CRLF + space(nEspaco * nNivel) + IIF(lRet, "", "[ERR0]" ) + STR0018+AllTrim(cImpFisc)+STR0019 +cPorta+"]" //" - Fechando Impressora Fiscal
511                 ["###"] Porta["
512             EndIf
513
514             If lRet
515                 cImpFisc := cImpFiscST

```

```

517         cPorta := cPortaST
518     EndIf
519
520 EndIf
521
522 //Busca as Alíquotas
523 lRet := lRet .AND. STFWzLdAIF(cLG_CODIGO, cImpFisc, cPorta, @lConecta, ;
524                             @aAliqICM, @aAliqISS, @aAliqECF, @cLog,;
525                             (nEspaco * nNivel) )
526
527
528 If lRet .AND. AliasIndic("MFD")
529     lRet := STFWzLdAP(cLG_CODIGO, @aAliqPtr, @aAliqECF, oDLG, ;
530                 nPanel ,@cLog, (nEspaco * nNivel))
531
532
533     If lRet
534         STFWzPrAP(cLG_CODIGO, @aAliqPtr, @aAliqECF, oDLG, ;
535                 nPanel, @cLog, (nEspaco * nNivel))
536     Else
537         oDlg:SetPanel(aItem[5])
538     EndIf
539 Else
540     oDlg:SetPanel(aItem[5])
541 EndIf
542
543 Case nItem == 3 //totalizadores não-fiscais
544     cMessage := STR0020 + aItem[2] //"Programando Totalizadores Não-Fiscais na "
545
546     cLog += CRLF + space(nEspaco * nNivel) + " - " + cMessage
547     nNivel++
548     //Busca os Totalizadores
549     lRet := STFWzLdTIF(cLG_CODIGO, cImpFisc, cPorta, @lConecta, ;
550                     @aTotNFisc, @lTotvsApi, @cLog, (nEspaco * nNivel))
551
552
553     If lRet
554         STFWzLdTP(cLG_CODIGO, @aTotPtr, @aTotNFisc, lTotvsApi,;
555                 @cLog, (nEspaco * nNivel))
556
557
558         STFWzPrTP(cLG_CODIGO, @aTotPtr, @aTotNFisc, lTotvsApi, ;
559                 oDLG, nPanel, @cLog, (nEspaco * nNivel))
560
561     Else
562         oDlg:SetPanel(aItem[5])
563     EndIf
564
565 Case nItem == 4
566     cMessage := STR0021 + aItem[2] //"Programar Formas de Pagamento na "
567     cLog += CRLF + space(nEspaco * nNivel) + " - " + cMessage
568     nNivel++

```

```

569
570      //Busca as formas de pagamento
571      lRet := STFWzLdFIF(cLG_CODIGO, cImpFisc, cPorta, @lConecta, ;
572                      @aFormaECF, @lTotvsApi, @cLog, (nEspaco * nNivel) )
573
574
575      If lRet //Busca as formas de pagamento do Sistema
576          STFWzLdFP(cLG_CODIGO, cImpFisc, @aFormaPtr, @aFormaECF, ;
577                  lTotvsApi, @cLog, (nEspaco * nNivel))
578
579
580          STFWzPrFP(cLG_CODIGO, @aFormaPtr, @aFormaECF, lTotvsApi, ;
581                  oDLG, nPanel, @cLog, (nEspaco * nNivel))
582
583      Else
584          oDlg:SetPanel(aItem[5])
585      EndIf
586
587
588      Case nItem == 5 //Programar horário de ver~so
589          cMessage := STR0022 + aItem[2] //"Programar Horário de Verão na "
590          cLog += CRLF + space(nEspaco * nNivel) + " - " + cMessage
591          nNivel++
592
593
594          lRet := STFWzPrHP(cLG_CODIGO, cImpFisc, @lConecta, oDLG, ;
595                          nPanel, @cLog, (nEspaco * nNivel))
596
597          If !lRet
598              oDlg:SetPanel(aItem[5])
599          EndIf
600      EndCase
601
602      If lRet
603          oDlg:SetMessage( cMessage + cMessage2, oDlg:nPanel, .t. )
604      EndIf
605      Return
606
607 //-----
608 /*{Protheus.doc} STFLdWzTEF
609 Paineis do Wizard do TEF
610 @param oDlg          Janela
611 @param cLG_CODIGO    Código da Estacao
612 @param nPanelIni     Panel Inicial do Wizard
613 @param cLog          Log
614 @param aItem         Item da funcionalidade
615 @param lConecta      Impressora Conectada
616 @obs
617 @sample
618 */
619 //-----
620 Function STFLdWzTEF(oDlg, cLG_CODIGO, nPanelIni, cLog, ;

```

```

621             aItem, lConecta)
622
623 Local nItem      := 0    //Item
624 Local nPanel     := 0    //Painel
625 Local oPanel     := 0    //Objeto Paine
626 Local oCfgTef    := NIL  //Objeto TEF 2.0
627 Local lRet       := .T.  //Retorno
628 Local oPanel2    := NIL  //Painel 2
629 Local nRow       := 0    //Linha
630 Local nCol       := 0    //Coluna
631 Local nRow1      := 0    //Linha 2 seção
632 Local nEspaco    := 5    //Espaços
633 Local nNivel     := 2    //Nivel
634 Local cMessage2  := ""    //Segunda Mensagem
635 Local cMessage   := ""    //Mensagem
636 Local cLogTmp    := ""    //Log Temporário
637
638 DEFAULT oDlg      := NIL  //Janela
639 DEFAULT cLG_CODIGO := ""    //Codigo da Estação
640 DEFAULT nPanelIni := 0    //Painel Inicial
641 DEFAULT cLog      := ""    //log
642 DEFAULT lConecta  := .F.  //Impressora Conectada?
643
644     nPanel := oDlg:nPanel
645
646     nItem := (nPanel - nPanelIni) + 1
647     oPanel := oDlg:GetPanel(nPanel)
648
649
650     cMessage2 := STR0015+ cLG_CODIGO + "]"    //" da Estação ["
651
652
653     Do Case
654     Case nItem == 1
655
656         cMessage := STR0023 //"Configurações TEF"
657         oDlg:SetMessage( cMessage + cMessage2 , oDlg:nPanel , .F. )
658         cLog += CRLF + space(nNivel * nEspaco) + " - " + cMessage + cMessage2
659
660         nRow      := Int((oPanel:nHeight ) / 2) //Linha do Paine
661         nCol      := Int((oPanel:nWidth ) / 2)  //Coluna do Paine
662         nRow1     := nRow-30
663
664         oPanel2 := TScrollBar():New( oPanel, 0,0,nRow,nCol)
665
666         oCfgTef := STFStTEF20( oPanel2, cLG_CODIGO)
667
668         If ValType(oCfgTef) == "0"
669             nNivel++
670             @nRow1+3, 0 TO nRow, nCol OF oPanel PIXEL
671             TButton():New( nRow1+8, 05, STR0024,oPanel,{|| STFLdGrTEF(oCfgTef, @cLog, aItem, oDlg;; //"Gravar"
672                                     (nNivel * nEspaco)) },55,15,,,F.,.T.,.F.,.F.,.F.,{ || .T. },,.F. )

```

```

673     Else
674         oDlg:SetPanel(aItem[5])
675     EndIf
676
677     Case nItem == 2
678         cMessage := STR0025 //"Teste de comunicação TEF"
679         //cLogTmp := cMessage + cMessage2
680         cLogTmp += CRLF + space(nNivel * nEspaco) + " - " + cMessage + cMessage2
681         //Realiza a Transação TEF - Administrativa
682         oDlg:SetMessage( cMessage + cMessage2, oDlg:nPanel , .F. )
683         nNivel++
684         STFLdTTEF( @cLogTmp, aItem, oDlg, cLG_CODIGO,;
685                 lConecta, (nNivel * nEspaco))
686         STFLdDTEF(oDlg, oDlg:nPanel, cLogTmp)
687
688         cLog += cLogTmp
689
690     EndCase
691
692 Return
693
694 //-----
695 /*{Protheus.doc} STFLdGrTEF
696 Função de Gravação dos Dados do TEF 2.0
697 @param oCfgTef      Configuração TEF
698 @param cLog         Log
699 @param aItem        Item do Wizard TEF
700 @param oDlg         Janela
701 @param nEspacos     Espaços do log
702 @return lRet Gravado com sucesso
703 @obs
704 @sample
705 */
706 //-----
707 Function STFLdGrTEF(oCfgTef, cLog, aItem, oDlg,;
708                 nEspacos)
709 Local lRet := .t. //Retorno da gravação
710
711 DEFAULT oCfgTef := NIL //Configurações TEF 2.0
712 DEFAULT cLog    := ""  //Log da Rotina
713 DEFAULT aItem   := {}  //Itens do Wizard
714 DEFAULT oDlg    := NIL //Janela
715 DEFAULT nEspacos:= 5   //Espacos
716
717 If ValType(oCfgTef) == "0" .And. ValType(aItem) == "A" .AND. ValType(oDlg) == "0"
718
719     If !oCfgTef:TefVl(oCfgTef, @lRet)
720         cLog += CRLF + space(nEspacos) + STR0026 + ; //" [ERR0] - Gravação do Registro TEF "
721         CRLF + space(nEspacos+5) + STR0027 + oCfgTef:cMensagem //" [ERR0] - "
722         If !MsgYesNo(STR0028) //"Problemas na gravação.Editar o registro novamente ?"
723             oCfgTef := FreeObj(oCfgTef)
724             oDlg:SetPanel(aItem[5])

```

```

725         EndIf
726     Else
727         oCfgTef:Salvar()
728     EndIf
729 EndIf
730
731 Return lRet
732
733 //-----
734 /*{Protheus.doc} STFLdTTEF
735 Realiza a função de transação de Teste do TEF 2.0
736 @param cLog      Log
737 @param aItem      Item do Wizard TEF
738 @param oDlg       Janela
739 @param cLG_CODIGO Código da Estacao
740 @param lConecta   Impressora Conectada?
741 @param nEspacos   Espaços do log
742 @return lRet Gravado com sucesso
743 @obs
744 @sample
745 */
746 //-----
747 Function STFLdTTEF( cLog, aItem, oDlg, cLG_CODIGO, ;
748                   lConecta, nEspacos)
749
750 Local oTEF20 := STBGetTEF()
751 Local lPrinter := .F. //Objeto printer ativo
752 Local lRestore := .F. //Restaura os servicos?
753 Local oDados := NIL //Objeto da transacao TEF
754 Local oRetTran := NIL //Retorno da transação TEF
755
756
757 DEFAULT cLog      := "" //log da rotina
758 DEFAULT aItem      := {} //Itens da funcionalidade
759 DEFAULT oDlg       := NIL //Janela
760 DEFAULT cLG_CODIGO := "" //Codigo da Estação
761 DEFAULT lConecta   := .F. //Impressora conectada
762 DEFAULT nEspacos   := 5 //Espaços
763
764
765 If oTEF20== "0"
766     STWCloseDevice(@oTEF20) //Fecha o objeto TEF para iniciá-lo novamente
767     lRestore := .T.
768 Else
769     If !Empty(cLG_CODIGO)
770
771         oTEF20 := LJC_TEF():New(STFGetStat("CODIGO") )
772         If !oTEF20:lAtivo
773             cLog += CRLF + space(nEspacos) + STR0029 //" [ERR0] - Problemas ao iniciar TEF "
774         Else
775             //Senão inicia a transação administrativa
776             lPrinter := !Empty(STFGetStat("PORTIF")) .AND. !Empty(STFGetStat("IMPFISC"))

```

```

777      //Envia comando para abrir a impressora
778      If lPrinter .AND. !lConecta
779          lPrinter := STFWzCIF(STFGetStat("IMPFISC"), STFGetStat("PORTIF"), @cLog, nEspacos)
780      EndIf
781
782      oDados := LJCDadosTransacaoADM():New(0,0, Date(), Time(), .T.)
783      oRetTran := oTEF20:Cartao():FuncoesAdm(oDados)
784
785
786      If oRetTran:oRetorno:lTransOk
787
788          If oRetTran:oRetorno:oViaCaixa:Count() > 0 .OR. oRetTran:oRetorno:oViaCliente:Count() > 0
789
790              oTEF20:Cupom():Inserir("G", oRetTran:oRetorno:oViaCaixa,oRetTran:oRetorno:oViaCliente, "A",;
791                  "", "", , oRetTran:nValor, 1,;
792                  0)
793
794              cLog += CRLF + space(nEspacos) + STR0030 //" - Transação Administrativa realizada."
795              If oTEF20:Cupom():Imprimir()
796                  cLog += CRLF + space(nEspacos) + STR0031 //" - Impressão Transação Administrativa.."
797                  oTEF20:Desfazer()
798              Else
799                  cLog += CRLF + space(nEspacos) + STR0032 //" [ERR0] - Impressão Transação Administrativa."
800                  oTEF20:Desfazer()
801              EndIf
802
803              Else
804                  cLog += CRLF + space(nEspacos) + STR0030 //" - Transação Administrativa realizada."
805                  oTEF20:Desfazer()
806              EndIf
807
808              Else
809                  cLog += CRLF + space(nEspacos) + STR0033 //" [ERR0] - Transação Administrativa."
810                  oTEF20:Desfazer()
811              EndIf
812
813              oDados := FreeObj(oDados)
814              STWCloseDevice(@oTEF20)
815          EndIf
816      EndIf
817  EndIf
818
819
820  If lRestore
821      STWOpenDevi(@oTEF20)
822  EndIf
823
824  Return
825
826
827  //-----
828  /*/{Protheus.doc} STFWzDtIF

```

```

829 Carrega os dados cadastrados na estação
830 @param cLG_CODIGO    Código da Estacao
831 @param cImpFisc       Impressora Fiscal
832 @param cPorta         Porta de Comunicação
833 @param cPDV           Numero do PDV
834 @param cTamSer        Tamanho do C00
835 @param aImpFisc       Array com os modelos de Impressoras Fiscais
836 @param aPortas        Array com as portas
837 @param lConecta       Impressora Conectada?
838 @param cSerie         Numero de fabricação
839 @param cLog           Log
840 @param nEspacos       Espaços do log
841 @return lRet Execução da rotina com sucesso
842 @obs
843 @sample
844 /*/
845 //-----
846
847 Function STFwzDtIF( cLG_CODIGO, cImpFisc, cPorta, cPDV, ;
848                   cTamSer, aImpFisc, aPortas, lConecta, ;
849                   cSerie , cLog,      nEspacos)
850
851
852 Local aDados      := {} //Retorno do Strategy
853 Local aRet        := {} //Retorno da Execução da Rotina
854 Local aAreaSLG    := SLG->(GetArea()) //WorkArea SLG
855 Local lRet        := .t. //Retorno da Rotina
856
857 DEFAULT cLG_CODIGO := SLG->(CriaVar("LG_CODIGO")) //Codigo da Estacao
858 DEFAULT cImpFisc   := "" //Impressora Fiscal
859 DEFAULT cPorta     := "" //Porta de Comunicação
860 DEFAULT cPDV       := "" //Numero do PDV
861 DEFAULT cTamSer    := "" //Numero do C00
862 DEFAULT aImpFisc   := {} //Array de Impressoras Fiscais
863 DEFAULT aPortas    := {} //Portas de Comunicação
864 DEFAULT cSerie     := "" //Numero de Fabricacao
865 DEFAULT cLog       := "" //Log da Rotina
866 DEFAULT nEspacos   := 15 //Espacos do log da rotina
867
868 aPortas := {"COM1", "COM2", "COM3", ;
869            "COM4" , "COM5", "COM6" }
870
871 cLog += CRLF + space(nEspacos) + STR0034 //" - Localizando configurações da Estação."
872
873 SLG->(DbSetOrder(1))
874 If SLG->(DbSeek(xFilial() + cLG_CODIGO))
875     cImpFisc := SLG->LG_IMPFIsc
876     cPorta   := SLG->LG_PORTIF
877     cPDV     := SLG->LG_PDV
878     cTamSer  := SLG->LG_TAMSER
879     cSerie   := SLG->LG_SERPDV
880 EndIf

```

```

881
882   If !Empty(cLG_CODIGO) .AND. SLG->(FieldPos("LG_ISPOS") ) > 0 .AND. SLG->LG_ISPOS == "1"
883
884       //Carrega os Dados da Estação
885       aRet := STFFireEvent( ProcName(0) ,;          // Nome do processo
886                           "STCheckDLL",; // Nome do evento
887                           { .T. } )
888
889       cLog += CRLF + space(nEspacos) + STR0035  //" - Buscando modelos de Impressoras Fiscais."
890
891       aDados := {space(5000), .t.}
892       STFFireEvent( ProcName(0) ,;          // Nome do processo
893                   "STListPrint",; // Nome do evento
894                   aDados )
895
896       aImpFisc := LJStrToArr(AllTrim(aDados[1]))
897       aSort(aImpFisc, ,, { |x,y| x< y})
898
899   Else
900       //   MsgAlert(STR0036 + xNumCaixa() + STR0037) //"0 usuário "###" não possui privilégios de caixa ."
901       MsgAlert("0 Wizard somente é aplicável a estação com POS")
902       lRet := .F.
903       cLog += CRLF + space(nEspacos) + "0 Wizard somente é aplicável a estação com POS" //" [ERR0] - 0 usuário "###" não possui privilégios de caixa ."
904   EndIf
905   RestArea(aAreaSLG)
906
907   Return lRet
908
909
910 //-----
911 /*/{Protheus.doc} STFWzLdIF
912 Função para atualização dos dados da Estação
913 @param cLG_CODIGO    Codigo da Estacao
914 @param cImpFisc      Impressora Fiscal
915 @param cPorta        Porta de Comunicação
916 @param cPDV          Numero do PDV
917 @param cTamSer       Tamanho do C00
918 @param aImpFisc      Array com os modelos de Impressoras Fiscais
919 @param aPorta        Array com as portas
920 @param oDlg          Janela
921 @param nPanel        Painel da rotina
922 @param lConectado    Impressora Conectada?
923 @param cSerie        Numero de fabricação
924 @param cLog          Log
925 @param nEspacos      Espaços do log
926 @return lRet Execução da rotina com sucesso
927 @obs
928 @sample
929 */
930 //-----
931 Function STFWzLdIF(cLG_CODIGO, cImpFisc, cPorta, cPDV, ;
932                  cTamSer, aImpFisc, aPorta, oDlg, ;

```

```

933         nPanel, lConectado, cSerie,cLog,,
934         nEspacos)
935
936 Local lRet      := .T. // Retorno da Função
937 Local nRow      := 0   //Linha
938 Local nCol      := 0   //Coluna
939 Local nRow1     := 0   //Linha 1
940 Local oPanel    := NIL //Panel
941 Local oBtn1     := NIL //Botao 1
942 Local oBtn2     := NIL //Botao 2
943 Local oBtn3     := NIL //Botao 3
944 Local oBtn4     := NIL //Botao 4
945 Local oBtn5     := NIL //Botao 5
946 Local oBtn6     := NIL //Botao 6
947 Local aItens    := {}  //Itens
948 Local nC        := 0   //Contador
949 Local oSayConn  := NIL //Objeto say que indicará se o componente está conectado
950 Local oSayCon2  := NIL //Objeto Say
951 Local oMemECF   := NIL //Resultado dos Dados do ECF
952 Local aObjects  := {}  //Objetos da Janela
953 Local cDadosImp := ""   //Dados da Impressora Fiscal
954 Local cEcfcOo   := ""   //Tamanho do C00
955 Local cECFPDV   := ""   //PDV
956 Local cECFCNPJ  := ""   //CNPJ do ECF
957 Local cECFData  := ""   //Data do ECF
958 Local cECFNFab  := ""   //Numero de Fabricacao do ECF
959 Local cECFModelo:= ""   //Modelo do ECF
960 Local cECFPorta := ""   //Porta do ECF
961 Local cDadosECF := ""   //Dados do ECF
962 Local aCampos   := {"cImpFisc", "cPorta", "cPDV", "cTamSer", "cSerie"}
963 Local lEdited   := .F. //Editado
964 Local cCNPJ     := SM0->M0_CGC //CNPJ da Empresa
965
966 DEFAULT lConectado := .F.
967 DEFAULT cLG_CODIGO := SLG->(CriaVar("LG_CODIGO")) //Codigo da Estacao
968 DEFAULT cImpFisc    := "" //Impressora Fiscal
969 DEFAULT cPorta      := "" //Porta de Comunicação
970 DEFAULT cPDV        := "" //Numero do PDV
971 DEFAULT cTamSer     := "" //Numero do C00
972 DEFAULT aImpFisc    := {} //Array de Impressoras Fiscais
973 DEFAULT aPorta      := {} //Portas de Comunicação
974 DEFAULT oDlg        := NIL //Janela
975 DEFAULT nPanel      := 1 //Painel
976 DEFAULT cSerie      := "" //Numero de Fabricacao
977 DEFAULT cLog        := "" //Log da Rotina
978 DEFAULT nEspacos    := 15 //Espacos do log da rotina
979
980 aItens := { { "S", STR0039, TamSx3("LG_IMPFIISC")[1]};; //"Modelo da Impressora Fiscal"
981             { "L", "cImpFisc", NIL, TamSx3("LG_IMPFIISC")[1]+50, aImpFisc, STR0039, { |U| Iif(PCOUNT() > 0, ( cImpFisc := U, lEdited := .T.), cImpFisc)}, .F. } ,;
982             //"Modelo da Impressora Fiscal"
983             { "S", STR0040, TamSx3("LG_PORTIF")[1]};; //"Porta de Comunicação"
984             { "L", "cPorta" , NIL, TamSx3("LG_PORTIF")[1]+5, aPorta, STR0040, { |U| Iif(PCOUNT() > 0, (cPorta := U, lEdited := .T.), cPorta)} , .F. } ,; //"Porta de

```

```

Comunicação"
984      { "S", STR0041 , TamSx3("LG_PDV")[1]},; //"Num PDV"
985      { "G", "cPDV" , NIL, TamSx3("LG_PDV")[1], , "Num PDV", { |U| Iif(PCOUNT() > 0, (cPDV := U, lEdited := .T.), cPDV)} , .F.} ,;
986      { "S", "Tam C00"},;
987      { "G", "cTamSer" , NIL, TamSx3("LG_TAMSER")[1], , STR0042, { |U| Iif(PCOUNT() > 0, (cTamSer := U, lEdited := .T.), cTamSer)} , .F.},; //"Tam Coo"
988      { "S", STR0043, TamSX3("LG_SERPDV")[1]},; //"Numero de Fabricação"
989      { "G", STR0044 , NIL, TamSx3("LG_SERPDV")[1], , STR0045, { |U| Iif(PCOUNT() > 0, (cSerie := U, lEdited := .T.), cSerie)}, .T. } }      //"Serie"###"Número
de Fabricação"

990
991 oPanel := oDlg:GetPanel(nPanel)
992
993 nRow      := Int((oPanel:nHeight ) / 2) //Linha do Painei
994 nCol      := Int((oPanel:nWidth ) / 2)  //Coluna do Painei
995 nRow1 := nRow-40
996
997 oPanel2 := TScrollBox():New( oPanel, 0,0,nRow1,nCol)
998
999
1000 @0,0 to nRow1, nCol
1001
1002 For nC := 1 to Len(aItens)
1003     IF aItens[nC, 1] == "S" //.OR. aItens[nC, 1] == "L"
1004         nTam := CalcFieldSize("C",Len(aItens[nC,02]),0,"",aItens[nC,02])
1005     Else
1006         nTam := CalcFieldSize("C",aItens[nC,04],0,"",aItens[nC,02])
1007     EndIf
1008     aAdd(aObjects, { nTam, IIF(aItens[nC, 1] == "S", 6, 10), .F., .F.})
1009 Next
1010
1011 //Label do Objeto conectado
1012 nTam := CalcFieldSize("C",Len(STR0046)+10,0,"",STR0046) //"Objeto Conectado"###"Objeto Conectado"
1013 aAdd(aObjects, { nTam, 10, .F., .F.}) //Linha em Branco
1014 aAdd(aObjects, { nTam, 10, .F., .F.}) //Say da Conexão do ECF
1015 aAdd(aObjects, { nTam, 10, .F., .F.}) //Linha em Branco
1016 aAdd(aObjects, { nTam, 10, .F., .F.}) //Say da Conexão do ECF
1017 nTam := CalcFieldSize("C",110,0,"@",STR0047) //"Dados do ECF"
1018 aAdd(aObjects,{nTam ,5*10 , .F., .F.}) //Impar Say
1019
1020
1021 aPosObj := MsObjSize( { 0 , 15 , nRow1 ,nCol, 0, 3,3,3}, aObjects, .T., .F.)
1022
1023
1024 For nC := 1 to Len(aItens)
1025
1026     cVariable := aItens[nC][02]
1027     cGet := "{ |U| Iif(PCOUNT() > 0, "+cVariable+" := U, "+cVariable+"}) "
1028
1029
1030     Do Case
1031     Case aItens[nC, 01] == "S" //Objeto Say
1032         tSay():New( aPosObj[nC, 1], aPosObj[nC, 2] , &("{ || ' " + aItens[nC, 02] + " ' } "),oPanel2,,,,,.T.,, , )
1033

```

```

1034
1035     Case aItens[nC, 01] == "G" //Objeto GET
1036
1037         aItens[nC, 03] := TGet():New( aPosObj[nC, 1],aPosObj[nC, 2],aItens[nC, 07],oPanel2,aObjects[nC, 01],,,
        .F.,.F.,, aItens[nC, 8], .F.,,cVariable,,,,.T.,) /*&(cBlkVld)*,,,,.T.,.T.,.T.,,
1038     Case aItens[nC, 01] == "L" //Objeto Lista
1039         aItens[nC, 03] := TComboBox():New( aPosObj[nC, 1],aPosObj[nC, 2], aItens[nC, 07], aItens[nC, 05], aObjects[nC, 01], aObjects[nC, 02], oPanel2, ,,
        ,,,.T.,,,.F.,,,.T.,,,cVariable)
1040
1041
1042     EndCase
1043
1044 Next
1045
1046     nC := nC + 1
1047     oSayConn:= tSay():New( aPosObj[nC, 1], aPosObj[nC, 2] , { || IF(lConectado, STR0048, STR0049) },oPanel2,,,,,.T.,, , ) //"Impressora Conectada"###"Impressora
Desconectada"
1048     nC := nC + 2
1049     oSayCon2:= tSay():New( aPosObj[nC, 1], aPosObj[nC, 2] , { || STR0047 },oPanel2,,,,,.T.,, , ) //"Dados do ECF"
1050     nC := nC + 1
1051     oMemECF := TMultiGet():New(aPosObj[nC, 1],aPosObj[nC, 2],{ || U| Iif(PCOUNT() > 0, cDadosECF := U, cDadosECF)},oPanel2,,
1052         aObjects[nC, 01],aObjects[nC, 02],/*oFont*/,.F.,;
1053         /*nClrFore*/,/*nClrBack*/,/*oCursor*/,.T.,;
1054         /*cMg*/,.T.,/*lCenter*/,;
1055         /*lRight*/,.T.,/*bChange*/,;
1056         ,.F.,.T.)
1057
1058
1059 @nRow1+3, 0 TO nRow, nCol OF oPanel PIXEL
1060
1061 oBtn2 := TButton():New( nRow1+8, 05, STR0050,oPanel,{|| STFWzIIF(@cImpFisc, @cPorta, @cLog, aPorta, nEspacos, @lConectado) , cECFModelo := cImpFisc , cECFPorta :=
cPorta , oBtn1:Refresh( ) , oSayConn:Refresh( ) },55,15,,,F.,.T.,.F.,.F.,{ || .T. },,.F. ) //"Identificar Modelo"
1062
1063 oBtn1 := TButton():New( nRow1+8, 070, STR0051,oPanel,{|| cDadosECF := "" , lConectado := STFWzCIF(cImpFisc, cPorta, @cLog, nEspacos) , oBtn3:Refresh(),
oSayConn:Refresh( ) , oSayCon2:Refresh( ) , oMemECF:Refresh( ) },55,15,,,F.,.T.,.F.,.F.,{ || !Empty(cImpFisc) .AND. !Empty(cPorta) },,.F. ) //"Conectar"
1064
1065 oBtn3 := TButton():New( nRow1+8, 145, STR0052,oPanel,{|| IIF(lConectado, STFWzDIF( cImpFisc, cPorta, cPDV , cTamSer, ; //"Capturar Dados"
1066                                     cCNPJ, cSerie, @cEcfcCo, @cECFPDV, ;
1067                                     @cECFCNPJ, @cECFData, @cECFNFab, @cDadosECF,;
1068                                     @cLog, nEspacos), MsgInfo(STR0053) ), oSayCon2:Refresh(),
oMemECF:Refresh( ) },55,15,,,F.,.T.,.F.,.F.,{ || lConectado },,.F. ) //"ECF não Conectado"
1069
1070 oBtn4 := TButton():New( nRow1+8, 220, STR0054,oPanel,{|| STFWzUIF(@cImpFisc, @cPorta, @cPDV , @cTamSer,; //"Atualizar Dados"
1071                                     @cSerie, cEcfcCo, cECFPDV, cECFModelo,;
1072                                     cECFPorta, cECFNFab, aCampos, aItens,;
1073                                     @cLog, nEspacos ), lEdiTed:= .T. , oBtn5:Refresh( ) },55,15,,,F.,.T.,.F.,.F.,{ ||
lConectado .AND. (!Empty(cEcfcCo) .OR. !Empty(cECFPDV) ) },,.F. )
1074
1075 oBtn5 := TButton():New( nRow1+8, 295, STR0055,oPanel,{|| STFWzRIF(cLG_CODIGO, cImpFisc, cPorta, cPDV ,; //"Gravar Dados"
1076                                     cTamSer, cSerie, @cLog, nEspacos), lEdiTed := .F., oBtn4:Refresh( )
},55,15,,,F.,.T.,.F.,.F.,{ || lEdiTed },,.F. )
1077

```

```

1078
1079 Return lRet
1080
1081
1082 //-----
1083 /*{Protheus.doc} STFwzIIF
1084 Função para identificação do ECF
1085 @param cLG_CODIGO    Código da Estacao
1086 @param cImpFisc      Impressora Fiscal
1087 @param cPorta        Porta de Comunicação
1088 @param cLog          Log
1089 @param aPortas       Array com as portas
1090 @param lConecta      Impressora Conectada?
1091 @param nEspacos      Espaços do log
1092 @return lRet Execução da rotina com sucesso
1093 @obs
1094 @sample
1095 */
1096 //-----
1097 Static Function STFwzIIF(cImpFisc, cPorta, cLog, aPortas, ;
1098                          nEspacos, lConecta)
1099 Local aRet      := {} //Array de retorno
1100 Local aDados    := {} //Dados de retorno do ECF
1101 Local nI        := 0  //Contador
1102 Local nP        := 0  //Contador
1103 Local cGetCliDir:= GetClientDir() //Diretorio do Smart client
1104 Local nFabr     := 0  //Fabricante
1105 Local aImpFisc  := {} //Array de modelos de Impressora Fiscal
1106 Local lRet      := .T. //Retorno da rotina
1107 Local nPortas   := 1  //quantidade de Portas
1108 Local lAchou    := .F. //ECF localizado
1109 Local nPos      := 0  //Posicao
1110 Local cTmp      := ""  //Variavel temporaria
1111 Local aFabricDLL := {
1112     ;
1113     {"DARUMA", .T., .F., GetClientDir() + "DARUMA32.DLL", "DARUMA FS2000 - V. 01.00" }, ;
1114     {"SWEDA", .F., .F., GetClientDir() + "CONVECF.DLL", "SWEDA IF ST120 - V. 01.00.01" }, ;
1115     {"BEMATECH", .F., .F., GetClientDir() + "BEMAFI32.DLL", "BEMATECH MP-3000 TH FI V01.01.01" }, ;
1116     {"EPSON", .F., .F., GetClientDir() + "INTERFACEEPSON.DLL", "EPSON TM-H6000 FBII V01.07.00"}, ;
1117     {"LOGGER", .T., .F., "", "URANO ZPM 200"}, ;
1118     {"ITAUTEC ZPM", .T., .F., GetClientDir() + "RFDZPMAPI.DLL", "ITAUTEC ZPM-300 256M PRT41 V01.03.00"}, ;
1119     {"IBM", .F., .F., GetClientDir() + "IBMF32.DLL", "IBM 4610-KN4 V01.00.02"}, ;
1120     {"ITAUTEC", .T., .F., GetClientDir() + "RFDBFIAPI.DLL", "ITAUTEC QW PRINTER 6000 MT2 V01.00.05"}, ;
1121     } //Principal DLL do Fabricante
1122 Local nPosLogger := aScan( aFabricDLL, { |f| f[1] == "LOGGER" }) //Posicao do Fabricante Logger (3 dlls)
1123
1124 DEFAULT cImpFisc := ""
1125 DEFAULT cPorta := ""
1126 DEFAULT cLog := ""
1127 DEFAULT nEspacos := 5
1128 DEFAULT aPortas := {}
1129 DEFAULT lConecta := .F.

```

```

1130 nFabr := Len(aFabricDLL)
1131
1132 //Carrega o Diretorio system
1133 aDados := { space(100)}
1134 aRet := STFFireEvent(      ProcName(0)                                ,;      // Nome do processo
1135                        "STSystemDirectory"                            ,;      // Nome do evento
1136                        aDados )
1137
1138
1139 If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0 .AND. !Empty(aDados[1])
1140     cTmp := aDados[1]
1141     aFabricDLL[nPosLogger, 4] := {cTmp+"\\DLLG2.DLL", cTmp+"\\LEITURA.DLL", cTmp+"\\AT017.DLL"}
1142 EndIf
1143
1144 aDados := {}
1145
1146 // MSGRUN
1147 For nI := 1 to nFabr
1148     If ValType(aFabricDLL[nI,04]) == "C"
1149         aFabricDLL[nI, 03] := File(aFabricDLL[nI,04])
1150     Else
1151         aFabricDLL[nI, 03] := .T.
1152         aEval(aFabricDLL[nI,04], { | l| aFabricDLL[nI, 03] := aFabricDLL[nI, 03] .AND. File(l)} )
1153     EndIf
1154 Next
1155
1156 cLog += CRLF + space(nEspacos) + STR0035 //" - Buscando modelos de Impressoras Fiscais."
1157
1158 aDados := {space(5000), .t., .T.}
1159 STFFireEvent( ProcName(0) ,;      // Nome do processo
1160              "STListPrintCod",; // Nome do evento
1161              aDados )
1162
1163 aImpFisc := LJStrToArr(AllTrim(aDados[1]))
1164 nFabr := Len(aImpFisc)
1165
1166
1167 For nI := 1 to nFabr
1168
1169     aImpFisc[nI] := StrTokArr(aImpFisc[nI], "|")
1170     aImpFisc[nI,1] := StrTran(aImpFisc[nI,1], '""')
1171     aImpFisc[nI,2] := StrTran(aImpFisc[nI,2], '""')
1172     //Quebra a primeira coluna por Fabricante + Marca + versão deDll
1173     cTmp := aImpFisc[nI,1]
1174     aImpFisc[nI,1] := {"","","","",cTmp}
1175     nP := At(space(1), AllTrim(cTmp) ) -1
1176     aImpFisc[nI,1, 1] := Upper(Substr(cTmp, 1, nP))
1177     cTmp := AllTrim(StrTran(Substr(cTmp, nP + 1), "."))
1178
1179     nP := Len(cTmp)
1180     Do While IsDigit(Substr(cTmp,nP, 1))
1181         nP--

```

```

1182 EndDo
1183 If nP <> Len(cTmp) //Achoo numeros
1184     aImpFisc[nI, 1, 3] := Substr(cTmp, nP+1)
1185     cTmp := AllTrim(Substr(cTmp, 1, nP))
1186     If Right(cTmp, 1) == "V"
1187         cTmp := AllTrim(Left(cTmp, Len(cTmp) -1))
1188         If Right(cTmp, 1) == "-"
1189             cTmp := AllTrim(Left(cTmp, Len(cTmp) -1))
1190         EndIf
1191     EndIf
1192 EndIf
1193 aImpFisc[nI, 1, 2] := StrTran(StrTran(cTmp,"-"), space(1))
1194
1195 Next nI
1196 aSort(aImpFisc, ,, { |x,y| x[1,1] + x[1,2] + x[1,3] < y[1,1] + y[1,2] + y[1,3] })
1197
1198 nFabr := Len(aFabricDLL )
1199 For nI := 1 to nFabr
1200     If aFabricDLL[nI, 03] //Dll Exist
1201         //Conecta no modelo
1202         //nPortas := 1
1203         nPos := 0
1204         nPortas := IIF( aFabricDLL[nI, 02 ], Len(aPortas), 1) //Conecta pela porta
1205
1206         For nP := 1 to nPortas
1207             cPorta := aPortas[nP]
1208
1209             MsgRun(STR0056 + cPorta + STR0057 + aFabricDLL[nI, 01] + " ", STR0058 , { || lConecta := STFWzCIF(aFabricDLL[nI, 05], cPorta, @cLog, nEspacos) }) //
Nome do processo //"Conectando Porta [####] Fabricante [####]Aguarde.... "
1210
1211             If lConecta
1212                 Exit
1213             EndIf
1214
1215             aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
1216                                 "STCloseCom" ,; // Nome do evento
1217                                 {cPorta} )
1218
1219         Next
1220
1221         If lConecta
1222             //Achoo a Porta
1223             //Verifica no modelo
1224             aDados := { "45", space(6) } // T0 D0: Marca
1225             aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
1226                                 "STPrinterStatus",; // Nome do evento
1227                                 aDados )
1228
1229             If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 1
1230                 If !Empty(aDados[2])
1231                     nPos := aScan(aImpFisc, { |i| AllTrim(i[2]) == AllTrim(aDados[2])})
1232

```

```

1233         lAchou := (nPos > 0)
1234     EndIf
1235
1236     If !lAchou      //
1237         //lAchou
1238         aDados := { "46", space(100) } // TO DO: Marca
1239         aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
1240             "STPrinterStatus",; // Nome do evento
1241             aDados ) //Fabricante + Modelo + Versão
1242
1243         If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 1
1244             If !Empty(aDados[2])
1245                 lAchou := STFWzIISF( aImpFisc, aDados[2], @nPos, 3) //Busca Marca modelo e FirmWare
1246             Else
1247                 lAchou := .F.
1248             EndIf
1249         EndIf
1250     EndIf
1251 EndIf
1252
1253     If lAchou
1254         cImpFisc := aImpFisc[nPos, 1, 4]
1255         Exit
1256     Else
1257         aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
1258             "STCloseCom" ,; // Nome do evento
1259             {cPorta } )
1260
1261     EndIf
1262 EndIf
1263
1264
1265 EndIf
1266 Next nI
1267
1268     If !lAchou
1269         cLog += CRLF + space(nEspaces) + STR0059 //" - [ERR0] Impressora não detectada. Verifique se a mesma encontra-se ligada e/ou conectada no computador"
1270         MsgInfo(STR0060) //"Impressora não detectada. Verifique se a mesma encontra-se ligada e/ou conectada no computador"
1271     Else
1272         cLog += CRLF + space(nEspaces) + STR0061 + cImpFisc + STR0062 + cPorta + "]" //" - Localizada Impressora "###" na porta ["
1273
1274     EndIf
1275
1276 Return lRet
1277
1278 //-----
1279 /*{Protheus.doc} STFWzIISF
1280 Função para seleção do modelo de ECF
1281 @param aImpressoras Array com os modelos de Impressoras Fiscais
1282 @param cNomeDLL Nome do modelo retornado pela DLL
1283 @param nPos Posição do Modelo no Array de Modelos de ECF
1284 @param nNivel Nivel de Busca 3 - Fabricante + Modelo + DLL , 2- Fabricante + Modelo, 1 - Fabricante

```

```

1285 @param cFabricDLL   Nome do Fabricante retornado pelo ECF
1286 @param cModeloDLL   Nome do Modelo retornado pelo ECF
1287 @param cVerDLL      Versão da DLL retornado pelo ECF
1288 @return  \Ret Localizou o ECF?
1289 @obs
1290 @sample
1291 /*/
1292 //-----
1293 Static Function STFwzIISF(aImpressoras, cNomeDLL,   nPos, nNivel, ;
1294                          cFabricDLL, cModeloDLL, cVerDLL )
1295
1296 Local lAchou          := .F. //Localizou op ECF?
1297 Local cFabric         := ""  //Fabricante
1298 Local cModelo         := ""  //Modelo
1299 Local cDLL            := ""  //DLL
1300 Local cTmp            := ""  //Temporária
1301 Local nC              := 0    //contador
1302 Local aImpTmp         := {} //Impressoras Selecionadas
1303 Local bCond           := { | l | l[1,1] == cFabricDLL .AND. l[1,2] == cModeloDLL .AND. l[1,3] == cVerDLL }
1304 Local aListaImp       := {} //Lista de impressoras do Fabricante
1305
1306 DEFAULT aImpressoras := {} //Lista de modelos de impressoras fiscais
1307 DEFAULT cNomeDLL      := "" //Nome do Modelo de Impressora Fiscal retornado pela DLL
1308 DEFAULT nPos          := 0  //Posicao do Modelo de Impressora Fiscal Selecionado
1309 DEFAULT nNivel        := 0  //Nivel de busca
1310 DEFAULT cFabricDLL    := "" //Fabricante retornado pelo ECF
1311 DEFAULT cModeloDLL    := "" //Modelo retornado pelo ECF
1312 DEFAULT cVerDLL       := "" //Versão retornado pelo ECF
1313 DEFAULT aImpTmp       := {} //Array dos modelos, conforme o fabricante
1314
1315 If !Empty(cNomeDLL)
1316
1317     If Empty(cFabricDLL)
1318         nC := At(space(1), cNomeDLL)
1319         cNomeDLL := Upper(cNomeDLL)
1320         cFabricDLL := AllTrim(Substr(cNomeDll, 1, nC-1))
1321         cTmp := Substr(cNomeDll, nC+1)
1322         If At(cFabricDLL, cTmp) > 0
1323             //Ajusta o nome da dll
1324             cTmp := StrTran(cTmp, cFabricDLL)
1325             cNomeDLL := cFabricDLL + " " + AllTrim(cTmp)
1326         EndIf
1327         If (nC := At(" - V.", cTmp) ) > 0
1328             cModeloDLL := StrTran(StrTran(SubsTr(cTmp, 1, nC-1), "-") , space(1))
1329             cTmp := SubsTr(cTmp, nC+5)
1330             If !Empty(cTmp)
1331                 cVerDLL := AllTrim(StrTran(cTmp, "."))
1332             Else
1333                 nNivel := 2
1334                 bCond := { | l | l[1,1] == cFabricDLL .AND. l[1,2] == cModeloDLL }
1335             EndIf
1336         Else

```

```

1337         cModeloDLL := AllTrim(cTmp) //somente o modelo
1338         If nNivel > 2
1339             nNivel := 2
1340             bCond := { | l | l[1,1] == cFabricDLL .AND. l[1,2] == cModeloDLL }
1341         EndIf
1342     EndIf
1343 EndIf
1344
1345
1346 While !lAchou .AND. nNivel > 0
1347
1348     lAchou := (nPos := aScan( aImpressoras, bCond ) ) > 0
1349     If lAchou .AND. nNivel <= 2
1350         //Filtra os níveis parecidos com o filtro e exibe para seleção
1351         aListaImp := {}
1352         Do While Eval(bCond, aImpressoras[nPos])
1353             aAdd(aListaImp, {aImpressoras[nPos, 1, 4], aImpressoras[nPos, 1, 1], aImpressoras[nPos, 1, 2], aImpressoras[nPos, 1, 3], nPos})
1354             nPos++
1355         EndDo
1356         If Len(aListaImp) > 0
1357
1358             nPos := STFWzIITF(aListaImp, cNomeDLL, cFabricDLL , cModeloDLL , cVerDLL)
1359
1360             If nPos == 0
1361                 lAchou := .F.
1362                 Exit
1363             EndIf
1364         EndIf
1365     EndIf
1366 EndIf
1367 nNivel--
1368 If nNivel == 2
1369     bCond := { | l | l[1,1] == cFabricDLL .AND. l[1,2] == cModeloDLL }
1370 Else
1371     bCond := { | l | l[1,1] == cFabricDLL }
1372 EndIf
1373 EndDo
1374 EndIf
1375
1376 Return lAchou
1377
1378 //-----
1379 /*{Protheus.doc} STFWzIISF
1380 Função para seleção do modelo de ECF pelo usuário
1381 @param aLstaImp Array com os modelos de Impressoras Fiscais do Fabricante
1382 @param cNomeDLL Nome do modelo retornado pela DLL
1383 @param cFabricDLL Nome do Fabricante retornado pelo ECF
1384 @param cModeloDLL Nome do Modelo retornado pelo ECF
1385 @param cVerDLL Versão da DLL retornada pelo ECF
1386 @return nPos Posicao do Modelo Selecionado
1387 @obs
1388 @sample

```

```

1389 /*/
1390 //-----
1391 Static Function STFWzIITF(aListaImp, cNomeDLL, cFabricDLL , cModeloDLL , ;
1392                          cVerDLL)
1393
1394 Local oDlgSelImp      := NIL    //Janela
1395 Local nPos            := 0      //Posicao do Modelo
1396 Local oList           := NIL    //Objeto Lista
1397 Local cList           := ""     //Modelo
1398 Local aCoord          := MsAdvSize(.T.) //Coordenadas da janela
1399
1400 DEFAULT aListaImp     := {}     //Lista de Modelos do Fabricante
1401 DEFAULT cNomeDLL      := ""     //Nome da DLL retornado pelo ECF
1402 DEFAULT cFabricDLL    := ""     //Fabricante da DLL retornado pelo ECF
1403 DEFAULT cModeloDLL    := ""     //Modelo da DLL retornado pelo ECF
1404 DEFAULT cVerDLL       := ""     //Versão da DLL retornado pelo ECF
1405
1406
1407      oDlgSelImp := TDialog():New(000,000,aCoord[6]/2,aCoord[5]/2,0emToAnsi(STR0063),,,,,,oMainWnd,.T.) //"Seleção da Administrador de Cartão de Crédito/Débito"
1408      //"Seleção do Modelo de Impressora Fiscal"
1409
1410      TSay():New(005,003,{|| STR0064 + AllTrim(Upper(cNomeDLL))},oDlgSelImp,,,F.,.F.,.F.,.T.,CLR_HBLUE,CLR_WHITE,oDlgSelImp:nClientWidth/2-7,008) //"Identifique qual
1411      administradora do sistema é a administradora identificada a seguir: " //"Identifique qual o modelo de ECF é o identificado a seguir: "
1412
1413      TGroup():New(015,003,oDlgSelImp:nClientHeight/2-30,oDlgSelImp:nClientWidth/2-7,STR0065,oDlgSelImp,,,T.,.F. ) //"Lista das Administradoras Cadastradas" //"Lista
1414      das Impressoras Fiscais Cadastradas"
1415
1416      oList := TWBrowse():New(026,005,oDlgSelImp:nClientWidth/2-15,oDlgSelImp:nClientHeight/2-58,,{STR0066,STR0067,STR0068,STR0069},,oDlgSelImp,,,,,,,,F.,.T.,
1417      ,.F.,,) //"Tipo#"Administradora" //"Nome"###"Marca"###"Modelo"###"Versão DLL"
1418
1419      oList:SetArray(aListaImp)
1420      oList:blDblClick := {|| nPos := aListaImp[oList:nAT,5] }
1421
1422      oList:bLine := { || {aListaImp[oList:nAT,1], aListaImp[oList:nAT,2], aListaImp[oList:nAT,3], aListaImp[oList:nAT,4]}}
1423
1424      TButton():New(oDlgSelImp:nClientHeight/2-27,003,0emToAnsi(STR0070),oDlgSelImp,{|| nPos := aListaImp[oList:nAT,5], oDlgSelImp:End() },040,010,,,,T.,.,{|| })
1425      //"&Ok"
1426
1427      oDlgSelImp:Activate(,,,T.)
1428
1429 Return nPos
1430
1431 //-----
1432 /*/{Protheus.doc} STFWzCIF
1433 Função para conexão da Impressora Fiscal
1434 @param cImpFisc      Modelo de Impressora Fiscal
1435 @param cPorta        Porta de Conexao
1436 @param cLog          Log de Execucao
1437 @param nEspacos      Espacos do log
1438 @return lConecta     Impressora Conectada
1439 @obs

```

```

1436 @sample
1437 /*/
1438 //-----
1439 Static Function STFWzCIF(cImpFisc, cPorta, cLog, nEspacos)
1440     Local aRet := {}
1441     Local lConecta := .T.
1442
1443
1444     DEFAULT cImpFisc := ""
1445     DEFAULT cPorta := ""
1446     DEFAULT cLog := ""
1447     DEFAULT nEspacos := 5
1448
1449     aRet := STFFireEvent( ProcName(0)           ,;      // Nome do processo
1450                         "STWOpenCom"         ,;      // Nome do evento
1451                         {AllTrim(cImpFisc)    ,;
1452                         AllTrim(cPorta),;
1453                         .F.} )
1454
1455     If !(lConecta := Len(aRet) > 0 .AND. aRet[1] = 0 )
1456
1457         STFMessage("STWOpenDevi", "STOP", STR0071) //"Falha no Comando de Abertura e Selecao da Porta"
1458         STFShowMessage("STWOpenDevi")
1459
1460     EndIf
1461
1462     cLog += CRLF + space(nEspacos) + IIF(lConecta, "", STR0072 ) + STR0073+AllTrim(cImpFisc)+STR0019 +cPorta+"]" //"[ERR0]"###" - Conectando Impressora Fiscal ["###"]
1463     Porta["
1464 Return lConecta
1465
1466 //-----
1467 /*/{Protheus.doc} STFWzDIF
1468 Função para retornar os dados da Impressora Fiscal
1469 @param cImpFisc      Modelo de Impressora Fiscal
1470 @param cPorta        Porta de Conexao
1471 @param cPDV          Codigo do PDV
1472 @param cTamSer       Tamanho do C00
1473 @param cCNPJ         CNPJ
1474 @param cSerie        Numero de Fabricacao
1475 @param cECFC00       Tamanho do Numero do C00 do ECF
1476 @param cECFPDV       Numero do PDV do ECF
1477 @param cECFCNPJ      CNPJ do ECF
1478 @param cECFData      Data/Hora do ECF
1479 @param cECFNFab      Numero de Fabricação do ECF
1480 @param cDadosECF     Dados do ECF
1481 @param cLog          Log de Execucão
1482 @param nEspacos      Espacos do log
1483 @obs
1484 @sample
1485 /*/
1486 //-----

```

```

1487 Static Function STFWzDIF(  cImpFisc,  cPorta,  cPDV ,  cTamSer;;
1488                             cCNPJ,    cSerie,  cEcfcCoo, cECFPDV, ;
1489                             cECFCNPJ,  cECFData, cECFNFab, cDadosECF;;
1490                             cLog,      nEspacos)
1491
1492 Local aRet      := {} //Retorno do Stategy
1493 Local aDados    := {} //Dados retornados pelo ECF
1494 Local lDateOk   := .t. //Data correta
1495 Local cLogT     := ""  //Log Temporário
1496 Local lSucesso  := .T. //Rotina Executada com sucesso
1497 Local nEspacoT  := 0   //Espaco temporario
1498 Local cTxtTmp   := ""  //Texto temporario
1499
1500 DEFAULT cImpFisc := "" //Modelo de Impressora fiscal
1501 DEFAULT cPorta   := "" //Porta da Impressora fiscal
1502 DEFAULT cPDV     := "" //Codigo do PDV
1503 DEFAULT cTamSer  := "" //Numero do C00
1504 DEFAULT cCNPJ    := "" //CNPJ
1505 DEFAULT cSerie   := "" //Numero de Serie
1506 DEFAULT cEcfcCoo := "" //C00 do ECF
1507 DEFAULT cECFPDV := "" //PDV do ECF
1508 DEFAULT cECFCNPJ := "" //CNPJ do ECF
1509 DEFAULT cECFData := "" //Data do ECF
1510 DEFAULT cECFNFab := "" //Numero de Fabricacao do ECF
1511 DEFAULT cDadosECF := "" //Dados do ECF
1512 DEFAULT cLog      := "" //log da rotina
1513 DEFAULT nEspacos := 5   //espacos
1514
1515 cDadosECF := ""
1516
1517 cEcfcCoo := ""
1518 cECFPDV := ""
1519 cECFCNPJ := ""
1520 cECFData := ""
1521 cECFNFab := ""
1522
1523 nEspacoT := nEspacos + 5
1524
1525 //PDV
1526 cTxtTmp := STR0074 //"Número do PDV: "
1527 //cDadosECF +=
1528 aDados := {space(20), Nil}
1529 aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
1530                      "STGetPDV"
1531                      aDados )
1532                      ,; // Nome do evento
1533
1533 If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0 .AND. !Empty(aDados[1])
1534 cECFPDV := AllTrim(aDados[1])
1535 cTxtTmp += cECFPDV
1536 If AllTrim(cPDV) <> cECFPDV
1537 cTxtTmp += space(5) + STR0075 + cPDV + "]" //Difere do informado ["
1538 cLogT += CRLF + space(nEspacoT) + " - " + cTxtTmp

```

```

1539     EndIf
1540
1541     cDadosECF += cTxtTmp
1542 Else
1543     cDadosECF += STR0076 //"Não identificado"
1544     cLogT += CRLF + space(nEspacoT) + STR0077 //" [ERR0] - Identificação do Número do PDV"
1545 EndIf
1546 cDadosECF += CRLF
1547
1548 //C00
1549 aDados := {space(20), nil}
1550 cTxtTmp := STR0078 //"Número do Cupom X Capacidade do C00: "
1551
1552 aRet := STFFireEvent( ProcName(0) ,;          // Nome do processo
1553                     "StGetReceipt"
1554                     aDados )
1555                                     ,;          // Nome do evento
1556
1556 If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0 .AND. !Empty(aDados[1])
1557     cEcfcCo := AllTrim(aDados[1])
1558
1559     cTxtTmp += cEcfcCo + " X " + Replicate("9", Len(cEcfcCo))
1560     If Len(AllTrim(cTamSer)) <> Len(cEcfcCo)
1561         cTxtTmp += space(5) + STR0075 + cTamSer + "]" //"Difere do informado ["
1562         cLogT += CRLF + space(nEspacoT) + " - " + cTxtTmp
1563     EndIf
1564
1565     cDadosECF += cTxtTmp
1566 Else
1567     cDadosECF += STR0076 //"Não identificado"
1568     cLogT += CRLF + space(nEspacoT) + STR0079 //" [ERR0] - Identificação do Númro do Cupom"
1569 EndIf
1570
1571 cDadosECF += CRLF
1572
1573 cTxtTmp := "CNPJ: "
1574 aDados := {"20", space(20)}
1575 aRet := STFFireEvent( ProcName(0) ,;          // Nome do processo
1576                     "STPrinterStatus"
1577                     aDados )
1578                                     ,;          // Nome do evento
1579
1579 If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 1 .AND. !Empty(aDados[2])
1580     cECFCNPJ := AllTrim(aDados[2])
1581     cTxtTmp += cECFCNPJ
1582
1583     If AllTrim(cCNPJ) <> StrTran(StrTran(StrTran(cECFCNPJ, "."), "/"), "-")
1584         cTxtTmp += space(5) + STR0080 + cCNPJ + "]" //"Difere do cadastrado ["
1585         cLogT += CRLF + space(nEspacoT) + " - " + cTxtTmp
1586     EndIf
1587
1588     cDadosECF += cTxtTmp
1589 Else
1590     cDadosECF += STR0076 //"Não identificado"

```

```

1591      cLogT += CRLF + space(nEspacoT) + STR0081 //[ERR0] - Identificação do CNPJ do ECF"
1592  EndIf
1593  cDadosECF += CRLF
1594
1595  cTxtTmp := STR0082 //"Número de Fabricação: "
1596  aDados := {space(30), Nil}
1597  aRet := STFFireEvent( ProcName(0) ;;          // Nome do processo
1598                      "StGetSerie"
1599                      aDados )
1600                      ;;          // Nome do evento
1601
1602  If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0 .AND. !Empty(aDados[1])
1603    cECFNFab := AllTrim(aDados[1])
1604    cTxtTmp += cECFNFab
1605
1606    If AllTrim(cSerie) <> cECFNFab
1607      cTxtTmp += space(5) + STR0080 + cSerie + "]" //"Difere do Cadastrado ["
1608      cLogT += CRLF + space(nEspacoT) + " - " + cTxtTmp
1609    EndIf
1610    cDadosECF += cTxtTmp
1611  Else
1612    cDadosECF += STR0076 //"Não identificado"
1613    cLogT += CRLF + space(nEspacoT) + STR0083 //[ERR0] - Identificação do Número de Fabricação"
1614  EndIf
1615  cDadosECF += CRLF
1616
1617  aRet := STFFireEvent( ProcName(0) ;;          // Nome do processo
1618                      "STCheckHour"
1619                      {} )
1620                      ;;          // Nome do evento
1621
1622  If Len(aRet) > 0 .AND. ValType(aRet[1]) == "L"
1623    lDateOk := aRet[1]
1624  EndIf
1625
1626  cTxtTmp := STR0084 //"Data e Hora: "
1627
1628  aDados := {"2", space(20)}
1629  aRet := STFFireEvent( ProcName(0) ;;          // Nome do processo
1630                      "STPrinterStatus"
1631                      aDados )
1632                      ;;          // Nome do evento
1633
1634  If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 1 .AND. !Empty(aDados[2])
1635    cTxtTmp += AllTrim(aDados[2])
1636
1637    aDados := {"1", space(20)}
1638    aRet := STFFireEvent( ProcName(0) ;;          // Nome do processo
1639                      "STPrinterStatus"
1640                      aDados )
1641                      ;;          // Nome do evento
1642
1643  If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 1 .AND. !Empty(aDados[2])
1644    cTxtTmp += " " + AllTrim(aDados[2])
1645    cDadosECF += cECFData
1646    If !lDateOk
1647      cTxtTmp += space(5) + STR0085 + DtoC(GetRmtDate() ) + " " + GetRmtTime() + "]" //"Divergem da data do computador ["

```

```

1643         cLogT += CRLF + space(nEspacoT) + " - " + cTxtTmp
1644     EndIf
1645 Else
1646     cTxtTmp += STR0076 //"Não identificado"
1647     cLogT += CRLF + space(nEspacoT) + STR0086 //" [ERR0] - Identificação da Hora do ECF"
1648 EndIf
1649 cDadosECF += cTxtTmp
1650 Else
1651     cDadosECF += STR0076 //"Não identificado"
1652     cLogT += CRLF + space(nEspacoT) + STR0087 //" [ERR0] - Identificação da Data do ECF"
1653 EndIf
1654
1655 cLog += CRLF + space(nEspacos) + IIF(!Sucesso, "", STR0072 ) + " - " + STR0088 + cLogT //" [ERR0]####Capturando dados do ECF."
1656
1657 Return
1658
1659 //-----
1660 /*{Protheus.doc} STFWzUIF
1661 Função para atualizar os dados do ECf no Wizard
1662 @param cImpFisc      Modelo de Impressora Fiscal
1663 @param cPorta        Porta de Conexao
1664 @param cPDV          Codigo do PDV
1665 @param cTamSer       Tamanho do C00
1666 @param cSerie        Numero de Fabricacao
1667 @param cECFC00       Tamanho do Numero do C00 do ECF
1668 @param cECFPDV       Numero do PDV do ECF
1669 @param cECFModelo    Modelo do ECF
1670 @param cECFPorta     Porta do ECF
1671 @param cECFNFab      Numero de Fabricação do ECF
1672 @param aCampos       Campos a serem atualizados
1673 @param aItens        Itens do Wizard
1674 @param cLog          Log de Execucao
1675 @param nEspacos      Espacos do log
1676 @obs
1677 @sample
1678 */
1679 //-----
1680 Static Function STFWzUIF(cImpFisc, cPorta, cPDV, cTamSer,
1681                          cSerie, cEcfCoo, cECFPDV, cECFModelo,
1682                          cECFPorta, cECFNFab, aCampos, aItens,
1683                          cLog, nEspacos) //Verificando o campo - cImpFisc
1684
1685 Local nPos      := 0 //Posicao do item
1686 Local cCampo    := "" //Campo
1687 Local cC00      := "" //C00
1688
1689 DEFAULT cImpFisc := "" //Modelo de Impressora Fiscal
1690 DEFAULT cPorta   := "" //Porta da Impressora Fiscal
1691 DEFAULT cPDV     := "" //Numero do PDV
1692 DEFAULT cTamSer  := "" //Tamanho do C00
1693 DEFAULT cSerie   := "" //Numero de Fabricacao
1694 DEFAULT cECFC00  := "" //C00 do ECF

```

```

1695 DEFAULT cECFPDV      := ""    //PDV do ECF
1696 DEFAULT cECFModelo   := ""    //Modelo do ECF
1697 DEFAULT cECFPorta     := ""    //Porta do ECF
1698 DEFAULT cECFNFab      := ""    //Numero do Fabricacao
1699 DEFAULT aCampos       := {"cImpFisc", "cPorta", "cPDV" , "cTamSer", "cSerie"} //Campos
1700 DEFAULT aItens         := {}    //Itens do Wizard
1701 DEFAULT cLog           := ""    //Log
1702 DEFAULT nEspacos       := 5     //Espacos
1703
1704 //Verifica se o modelo do ECF foi atualizado
1705 If !Empty(cECFModelo) .AND. AllTrim(cImpFisc) <> AllTrim(cECFModelo)
1706     cImpFisc := PadR(cECFModelo, TamSX3("LG_IMPFIISC")[1])
1707     IF (nPos := aScan(aItens, { |c| c[2] == aCampos[1] } ) ) > 0
1708         aItens[nPos, 03]:Refresh()
1709     EndIf
1710 EndIf
1711
1712 //Verifica se a porta do ECF foi atualizada
1713 If !Empty(cECFPorta) .AND. AllTrim(cPorta) <> AllTrim(cECFPorta)
1714     cPorta := PadR(cECFModelo, TamSX3("LG_PORTIF")[1])
1715     IF (nPos := aScan(aItens, { |c| c[2] == aCampos[2] } ) ) > 0
1716         aItens[nPos, 03]:Refresh()
1717     EndIf
1718 EndIf
1719
1720 //Verifica se o PDV foi atualizado
1721 If !Empty(cECFPDV) .AND. AllTrim(cPDV) <> cECFPDV
1722     cPDV := PadR(cECFPDV, TamSX3("LG_PDV")[1])
1723     IF (nPos := aScan(aItens, { |c| c[2] == aCampos[3] } ) ) > 0
1724         aItens[nPos, 03]:Refresh()
1725     EndIf
1726 EndIf
1727
1728 //Verifica se o numero de Sério foi atualizado
1729 cC00 := Replicate("9", Len(cECFCoo))
1730 If !Empty(cECFPDV) .AND. AllTrim(cTamSer) <> cC00
1731     cTamSer := PadR(cC00, TamSX3("LG_TAMSER")[1])
1732     IF (nPos := aScan(aItens, { |c| c[2] == aCampos[4] } ) ) > 0
1733         aItens[nPos, 03]:Refresh()
1734     EndIf
1735 EndIf
1736
1737 If !Empty(cECFNFab) .AND. AllTrim(cSerie) <> cECFNFab
1738     cSerie := PadR(cECFNFab, TamSX3("LG_SERPDV")[1])
1739     IF (nPos := aScan(aItens, { |c| c[2] == aCampos[5] } ) ) > 0
1740         aItens[nPos, 03]:Refresh()
1741     EndIf
1742 EndIf
1743
1744 cLog := CRLF + space(nEspacos) + STR0089 //" - Atualizando dados na Janeça"
1745

```

```

1747
1748 Return
1749
1750 //-----
1751 /*{Protheus.doc} STFWzRIF
1752 Função para atualizar o cadastro da estação
1753 @param cLG_CODIGO    Código da Estação
1754 @param cImpFisc      Modelo de Impressora Fiscal
1755 @param cPorta        Porta de Conexao
1756 @param cPDV          Código do PDV
1757 @param cTamSer       Tamanho do C00
1758 @param cSerie        Numero de Fabricacao
1759 @param cLog          Log de Execucao
1760 @param nEspacos      Espacos do log
1761 @obs
1762 @sample
1763 */
1764 //-----
1765 Static Function STFWzRIF(cLG_CODIGO, cImpFisc, cPorta, cPDV , ;
1766                        cTamSer, cSerie, cLog, nEspacos)
1767
1768 Local aArea := GetArea() //WorkArea Ativa
1769 Local aAreaSLG := SLG->(GetArea()) //WorkArea SLG
1770
1771 DEFAULT cLG_CODIGO := "" //Codigo da Estação
1772 DEFAULT cImpFisc := "" //Impressora Fiscal
1773 DEFAULT cPorta := "" //Porta de Comunicação
1774 DEFAULT cPDV := "" //PDV
1775 DEFAULT cTamSer := "" //C00
1776 DEFAULT cSerie := "" //Numero de Fabricação
1777 DEFAULT cLog := "" //Log da rotina
1778 DEFAULT nEspacos := 5 //Espacos
1779
1780
1781 SLG->(DbSetOrder(1))
1782 If !Empty(cLG_CODIGO) .AND. SLG->(DbSeek(xFilial() + cLG_CODIGO))
1783   RecLock("SLG", .F.)
1784
1785   If !Empty(cImpFisc) .AND. AllTrim(cImpFisc) <> AllTrim(SLG->LG_IMPFISC )
1786     SLG->LG_IMPFISC := cImpFisc
1787   EndIf
1788
1789   If !Empty(cPorta) .AND. AllTrim(cPorta) <> AllTrim(SLG->LG_PORTIF)
1790     SLG->LG_PORTIF := cPorta
1791   EndIf
1792
1793   If !Empty(cPDV) .AND. AllTrim(cPDV) <> AllTrim(SLG->LG_PDV)
1794     SLG->LG_PDV := cPDV
1795   EndIf
1796
1797   If !Empty(cTamSer) .AND. AllTrim(cTamSer) <> AllTrim(SLG->LG_TAMSER)
1798     SLG->LG_TAMSER := cTamSer

```

```

1799      EndIf
1800
1801
1802      If !Empty(cSerie) .AND. AllTrim(cSerie) <> AllTrim(SLG->LG_SERPDV)
1803          SLG->LG_SERPDV := cSerie
1804      EndIf
1805
1806
1807      SLG->(MsUnLock())
1808      cLog += CRLF + space(nEspacos)+ STR0090      //" - Cadastro de Estação atualizado."
1809  Else
1810      cLog += CRLF + space(nEspacos)+ STR0091 //" - Estação não localizada."
1811  EndIf
1812
1813  RestArea(aAreaSLG)
1814  RestArea(aArea)
1815  Return
1816
1817  //-----
1818  /*{Protheus.doc} STFWzLdAIF
1819  Função para carregar as alíquotas da Impressora Fiscal
1820  @param cLG_CODIGO    Codigo da Estação
1821  @param cImpFisc      Modelo de Impressora Fiscal
1822  @param cPorta        Porta de Conexao
1823  @param lConecta      ECF Conectado
1824  @param aAliqICM      Alíquotas de ICM
1825  @param aAliqISS      Alíquotas de ISS
1826  @param aAliqECF      Alíquotas do ECF
1827  @param cLog          Log de Execucao
1828  @param nEspacos      Espacos do log
1829  @return lRet         Execução com sucesso
1830  @obs
1831  @sample
1832  */
1833  //-----
1834  Function STFWzLdAIF(cLG_CODIGO, cImpFisc, cPorta, lConecta,;
1835                    aAliqICM, aAliqISS, aAliqECF, cLog,;
1836                    nEspacos )
1837
1838  Local lRet      := .T. //Retorno da rotina
1839  Local aRet      := {}  //Retorno do strategy
1840  Local aDados    := {}  //Dados retornados pelo ECF
1841  Local nC        := 0   //Contador
1842
1843  DEFAULT cLG_CODIGO := "" //Codigo da Estacao
1844  DEFAULT cImpFisc    := "" //Modelo de Impressora Fiscal
1845  DEFAULT cPorta      := "" //Porta de Comunicacao
1846  DEFAULT lConecta    := .T. //Impressora Conectada
1847  DEFAULT aAliqICM    := {} //Alíquotas de ICMS
1848  DEFAULT aAliqISS    := {} //Alíquotas de ISS
1849  DEFAULT aAliqECF    := {} //Alíquotas de ECF
1850  DEFAULT cLog        := "" //log da rotina

```

```

1851 DEFAULT nEspacos      := 5      //Espacos
1852
1853     aAliqICM := {}
1854     aAliqISS := {}
1855     aAliqECF := {}
1856
1857
1858     If !lConecta
1859         If Empty(cImpFisc)
1860
1861             cImpFisc := STFGetStat("IMPFISC")
1862         EndIf
1863         If Empty(cPorta)
1864             cPorta := STFGetStat("PORTIF")
1865         EndIf
1866
1867         lConecta := lRet := STFWzCIF(cImpFisc, cPorta, @cLog, nEspacos)
1868
1869     EndIf
1870
1871     If lRet
1872         //Busca as Aliquotas de ICMS
1873         aDados := {space(100)}
1874         aRet := STFFireEvent( ProcName(0) ,;          // Nome do processo
1875                             "STGetAliq"              ,;          // Nome do evento
1876                             aDados )
1877
1878         If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0
1879
1880             aAliqICM := StrTokArr( aDados[1] , "|" )
1881
1882             aDados := {space(100)}
1883             aRet := STFFireEvent( ProcName(0) ,;          // Nome do processo
1884                                 "STGetAliISS",;
1885                                 aDados )
1886             If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0
1887                 aAliqISS := StrTokArr( aDados[1] , "|" )
1888             Else
1889                 lRet := .F.
1890             EndIf
1891
1892             cLog += CRLF + space(nEspacos) + IIF(lRet, "", STR0072) + STR0092 //"[ERR0]"###" - Capturando alíquotas de ICMS do ECF"
1893         Else
1894             lRet := .F.
1895
1896         EndIf
1897
1898         cLog += CRLF + space(nEspacos) + IIF(lRet, "", STR0072) + STR0093          //"[ERR0]"###" - Capturando alíquotas
1899         de ISS do ECF"
1900
1901         //Busca as Aliquotas de ISS
1902     EndIf

```

```

1902
1903     If lRet
1904         aEval(aAliqICM, { |a| IIF(Val(a) > 0, (nC := nC + 1, aAdd(aAliqECF, {StrZero(nC, 2),"1", Val(a)} )), NIL )})
1905         aEval(aAliqISS, { |a| IIF(Val(a) > 0, (nC := nC + 1, aAdd(aAliqECF, {StrZero(nC, 2), "2", Val(a)} )), NIL )})
1906     EndIf
1907
1908 Return lRet
1909
1910 //-----
1911 /*{Protheus.doc} STFWzLdAP
1912 Função para carregar as alíquotas da Impressora Fiscal
1913 @param cLG_CODIGO    Código da Estação
1914 @param aAliqPtr      Alíquotas do Protheus
1915 @param aAliqECF      Alíquotas do ECF
1916 @param oDLG          Janela
1917 @param nPanel        Panel atual
1918 @param cLog          Log de Execucao
1919 @param nEspacos      Espacos do log
1920 @return lRet         Execução com sucesso
1921 @obs
1922 @sample
1923 */
1924 //-----
1925 Function STFWzLdAP(cLG_CODIGO, aAliqPtr, aAliqECF, oDLG,
1926                  nPanel, cLog, nEspacos)
1927
1928     Local aArea := GetArea()
1929     Local aAreaMFD := MFD->(GetArea())
1930     Local lRet := .T.
1931     Local cStatus := ""
1932     Local cSFI := ""
1933     Local cCampos := "" //Codigo dos campos
1934     Local cFilMFD := MFD->(xFilial())
1935
1936
1937     MFD->(DbSetOrder(1)) //MFD_FILIAL
1938     MFD->(DbSeek(cFilMFD))
1939
1940     Do While MFD->(Eof()) .and. MFD->MFD_FILIAL == cFilMFD
1941         cStatus := "1" //Não programado
1942         If MFD->(aScan(aAliqECF, {|e| e[2] == MFD_IMPOST .AND. e[3] == MFD_ALIQ }))) > 0
1943             cStatus := "2" //Programado
1944         EndIf
1945
1946         //Verifica se o Campo Existe na SFI
1947         //Tipo de Imposto
1948         //Alíquota
1949         cCampos := ""
1950         cSFI := STFWzLdCAP(MFD->MFD_IMPOST, MFD->MFD_ALIQ, @cCampos)
1951         aAdd(aAliqPtr, { cStatus, MFD->MFD_IMPOST, MFD->MFD_ALIQ, cSFI, cCampos } )
1952
1953         MFD->(DbSkip(1))

```

```

1954 EndDo
1955
1956 cLog += CRLF + space(nEspacos) + STR0094  //" - Buscando alíquotas cadastradas no arquivo"
1957
1958 RestArea(aAreaMFD)
1959 RestArea(aArea)
1960
1961 Return lRet
1962
1963 //-----
1964 /*{Protoheus.doc} STFWzLdCAP
1965 Função para verificar se o imposto está cadastrado no SFI
1966 @param cImpost      Codigo do Imposto 1- ICMS, 2=ISS
1967 @param nAliq        Alíquotas do Protoheus
1968 @param cCampo        Nome dos campos separados por virgulas
1969 @return cRet        1 = Cadastrado,2=Não Cadastrado, 3= Dispensado
1970 @obs
1971 @sample
1972 /**
1973 //-----
1974 Static Function STFWzLdCAP(cImpost, nAliq, cCampos)
1975     Local cAliq := "" //Codigo da Aliquota do Imposto
1976     Local cCampo := "" //Campo de Base do Imposto
1977     Local cCampo2 := "" //Campo de Código do tributo "T"
1978     Local lRet := .F. //validacao dos campos
1979     Local cRet := "2" //Não Cadastrado
1980
1981     //Local lOpen := MyOpenSM0Ex()
1982
1983     DEFAULT cImpost := "1" //ICMS
1984     DEFAULT nAliq := 0
1985     DEFAULT cCampos := ""
1986
1987
1988     cAliq := AllTrim(Str(Int(nAliq), 2))
1989     cCampos := ""
1990
1991     If nAliq > 0
1992
1993         If cImpost == "1"
1994             If nAliq == 1 .AND. LjAnalisaLeg(16)[1]
1995
1996                 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
1997                 //^ Grava a aliq. 0,01%(Garantido Integral ECF Mato Grosso) ^
1998                 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
1999                 cCampo := "FI_BAS001"
2000                 cCampos := cCampo
2001             Else
2002
2003                 cCampo := "FI_BAS" + cAliq
2004                 cCampo2 := "FI_COD" + cAliq
2005

```

```

2006         cCampos := cCampo + "," + cCampo2
2007
2008     EndIf
2009
2010     If !Empty(cCampo)
2011         lRet := SFI->(FieldPos(cCampo)) > 0
2012     EndIf
2013
2014     If !Empty(cCampo2)
2015         lRet := lRet .AND. SFI->(FieldPos(cCampo2)) > 0
2016     EndIf
2017
2018     cRet := IIF(lRet, "1", "2")
2019
2020
2021     Else //ISS - somente a base
2022         cCampo := "FI_BIS" + cAliq
2023         cCampos := cCampo
2024         If SFI->(FieldPos(cCampo)) > 0
2025             cRet := "1"
2026         EndIf
2027     EndIf
2028
2029 EndIf
2030
2031 Return cRet
2032
2033
2034
2035 //-----
2036 /*{Protheus.doc} STFWzPrAP
2037 Função para cadastro das alíquotas a serem programadas
2038 @param cLG_CODIGO    Código da Estação
2039 @param aAliqPtr      Alíquotas do Protheus
2040 @param aAliqECF      Alíquotas do ECF
2041 @param oDLG          Janela
2042 @param nPanel        Panel atual
2043 @param cLog          Log de Execucao
2044 @param nEspacos      Espacos do log
2045 @obs
2046 @sample
2047 */
2048 //-----
2049 Function STFWzPrAP(cLG_CODIGO, aAliqPtr, aAliqECF, oDlg,;
2050                 nPanel, cLog, nEspacos)
2051 //Monta o TCBrowse
2052 Local oOK      := LoadBitmap(GetResources(),'br_verde') //Objeto BMP OK
2053 Local oNO      := LoadBitmap(GetResources(),'br_vermelho') //Objeto BMP NO
2054 Local aObjects := {} //Objetos da janela
2055 Local aPosObj  := {} //Posicao do objeto
2056 Local nAliquota := 00.00 //Aliquota a ser programada
2057 Local oBrowseECF:= NIL //Browse de aliquotas ECF

```

```

2058 Local oBrowsePtr:= NIL //Browse de aliquotas o Protheus
2059 Local nRow      := 0 //Linhas do Panel
2060 Local nCol       := 0 //Colunas do panel
2061 Local nTipo      := 1 //tipo do Imposto
2062 Local oTipo      := NIL //tipo da aliquota
2063 Local nLargB     := 0 //Largura do Browse
2064
2065 DEFAULT cLG_CODIGO := SLG->(CriaVar("LG_CODIGO")) //Estação
2066 DEFAULT aAliqPtr   := {} //aliquotas do Protheus
2067 DEFAULT aAliqECF    := {} //Aliquotas do ECF
2068 DEFAULT oDlg       := NIL //Janela
2069 DEFAULT nPanel      := 1 //Panel
2070 DEFAULT cLog        := "" //Log
2071 DEFAULT nEspacos    := 5 //Espacos do Log
2072
2073 oPanel := oDlg:GetPanel(nPanel)
2074
2075 nRow := Int( oPanel:nHeight/2) //Linha do Painel
2076 nCol := Int( oPanel:nWidth/ 2)
2077
2078 oPanel := TScrollBar():New( oPanel, 0,0,nRow,nCol)
2079 nCol := Int( nCol *.97 )
2080
2081 nLargB := (nCol/2) - 20
2082
2083 nTam := CalcFieldSize("C",Len(STR0095),0,"@!",STR0095) //1 //"Alíquotas cadastradas no ECF"###"Alíquotas cadastradas no ECF"
2084 aAdd(aObjects, { nTam, 6, .F., .F.}) //Impar Say //1
2085 aAdd(aObjects, { nLargB, 150, .F., .F.}) //Impar Sa //2
2086 aAdd(aObjects, { 10, 10, .F., .F.}) //Linha em branco //3
2087 nTam := CalcFieldSize("C",Len(STR0096),0,"@!",STR0096) //4 //"Programar Alíquotas"###"Programar Alíquotas"
2088 aAdd(aObjects, { nTam, 12, .F., .F.}) //4
2089 nTam := CalcFieldSize("N",5,9,"@E 99,99%",STR0097) //"Alíquota"
2090 aAdd(aObjects, { nTam, 6, .F., .F.}) //5
2091 aAdd(aObjects, { nTam, 10, .F., .F.}) //6
2092 nTam := CalcFieldSize("C",Len("ICMS")+5,0,"@!","ICMS")
2093 aAdd(aObjects, { nTam, 6, .F., .F.}) //7 - mesma linha do ICMS
2094 aAdd(aObjects, { 55, 15, .F., .F.}) //8 botão programar
2095
2096
2097 aPosObj := MsObjSize( { 0 , 15 , nRow ,nCol, 2, 3,3,3}, aObjects, .T., .F.)
2098
2099
2100 //oPanel := oDlg:GetPanel(nPanel)
2101
2102 //Monta o TCBrowse
2103 tSay():New( aPosObj[01, 1], aPosObj[01, 2] , { || STR0095 } ,oPanel,,,,,T,,, , ) //"Alíquotas cadastradas no ECF"
2104
2105 oBrowseECF := TCBrowse():New( aPosObj[02,01] , aPosObj[02,02], nLargB, aObjects[02,02], ,,,oPanel,,,,,,,,,F,,,T,,,F,,, )
2106 oBrowseECF:SetArray(aAliqECF)
2107 oBrowseECF:AddColumn(TCColumn():New(STR0098,{||aAliqECF[oBrowseECF:nAt,01] },"@!",,,,"LEFT",CalcFieldSize("C",2,0,"@!",STR0098),.F.,.F.,,,,)) //"Código"###"Código"
2108 oBrowseECF:AddColumn(TCColumn():New(STR0099,{||IIF(aAliqECF[oBrowseECF:nAt,02]= "1", STR0100, STR0101) },"@!",,,,"LEFT",CalcFieldSize("C",4,0,"@!",STR0099),.F.,.F.,,,,))
//"Tipo"###"ICMS"###"ISS"###"Tipo"

```

```

2109 oBrowseECF:AddColumn(TCColumn():New(STR0102,{||aAliqECF[oBrowseECF:nAT,03] },"@E 99.99",,,,"LEFT",CalcFieldSize("N",5,5,"@E 99.99",STR0102),.F.,.F.,,,,)) //"Alíquota
%####Alíquota %"
2110
2111 tSay():New( aPosObj[01, 1], nLargB+aPosObj[02,02]+5 , { || STR0103 } ,oPanel,,,,,.T.,, , ) //"Alíquotas cadastradas no sistema"
2112 oBrowsePtr := TCBrowse():New( aPosObj[02,01] , nLargB+aPosObj[02,02]+5,(nCol - (nLargB+aPosObj[02,02]+5)), aObjects[02,02],,,,oPanel,,,,,,.F.,.T.,.F.,, , )
2113 oBrowsePtr:SetArray(aAliqPtr)
2114 oBrowsePtr:AddColumn(TCColumn():New(STR0104,{||IIf(aAliqPtr[oBrowsePtr:nAt,01]== "2",oOK,oNO) },"",,,,"LEFT",CalcFieldSize("C",2,0,"@",STR0104),.T.,.F.,,,,))
//"Status"####"Status"
2115 oBrowsePtr:AddColumn(TCColumn():New(STR0099,{||IIf(aAliqPtr[oBrowsePtr:nAt,02]== "1", STR0100, STR0101) },"@",,,,"LEFT",CalcFieldSize("C",4,0,"@",STR0099),.F.,.F.,,,,))
//"Tipo"####"ICMS"####"ISS"####"Tipo"
2116 oBrowsePtr:AddColumn(TCColumn():New(STR0102,{||aAliqPtr[oBrowsePtr:nAT,03] },"@E 99.99",,,,"LEFT",CalcFieldSize("N",5,5,"@E 99.99",STR0102),.F.,.F.,,,,)) //"Alíquota
%####Alíquota %"
2117 oBrowsePtr:AddColumn(TCColumn():New(STR0105,{|| IIf(aAliqPtr[oBrowsePtr:nAT,04] == "1", STR0106, IIf(aAliqPtr[oBrowsePtr:nAT,04] == "2", STR0107, STR0108))
,,,,"LEFT",CalcFieldSize("C",14,0,"",STR0105),.F.,.F.,,,,)) //"Cad Mapa Resumo?####"Cadastrado"####"Não Cadastrado"####"Dispensado"####"Cad Mapa Resumo?"
2118
2119 oBrowsePtr:BHeaderClick := { || STFWzShowLg( { "BR_VERDE", STR0109},,; //"Programado"
2120 {"BR_VERMELHO", STR0110}; //"Não programado"
2121 }, STR0111 ) } //"Legenda Alíquotas"
2122
2123 //Seção 2 Programação de alíquotas
2124
2125
2126 @aPosObj[03,01], 5 to aPosObj[08, 1],nCol OF oPanel PIXEL
2127 tSay():New( aPosObj[04, 1], aPosObj[04, 2]+15 , { || STR0096 } ,oPanel,,,,,.T.,, , ) //"Programar Alíquotas"
2128 tSay():New( aPosObj[05, 1], aPosObj[05, 2]+15 , { || STR0097 } ,oPanel,,,,,.T.,, , ) //"Alíquota"
2129
2130 TGet():New( aPosObj[06, 1],aPosObj[06, 2]+15,{ ||U| Iif(PCOUNT() > 0, nAliquota := U, nAliquota)},oPanel,aObjects[06,01],aObjects[06,02],"@E 99.99",{|| nAliquota >= 0
,,, .T.,.T.,.T., , .F.,.F.,.f.,.F.,,"nAliquota",,,,T.,)
2131 @ aPosObj[05, 1], aPosObj[07, 2]+ aPosObj[05, 04] + 20 +15 RADIO oTipo VAR nTipo SIZE aObjects[04, 01], aObjects[04, 02] PROMPT STR0100, STR0101 OF oPanel
//"ICMS"####"ISS"
2132 TButton():New(aPosObj[05, 1], aPosObj[08, 2]+ aPosObj[05, 04] + aPosObj[07, 04]+50+15, STR0112,oPanel,{|| IIF( STFWzPrAg( Str(nAliquota,5,2), Str(nTipo,1,0), @aAliqECF,
@aAliqPtr, ; //"Programar"
2133 @cLog, nEspacos), ;
2134 (oBrowseECF:SetArray(aAliqECF),
oBrowsePtr:SetArray(aAliqPtr)), ),55,15,,,F.,.T.,.F.,.F.,{ || nAliquota > 0 },,,F. )
2135 TButton():New(aPosObj[05, 1], aPosObj[08, 2]+ aPosObj[05, 04] + aPosObj[07, 04]+50+15+65, STR0113,oPanel,{|| IIF( STFWzPrAg( "0" ,"T", @aAliqECF, @aAliqPtr,;
//"Programar Tudo"
2136 @cLog, nEspacos), ;
2137 (oBrowseECF:SetArray(aAliqECF),
oBrowsePtr:SetArray(aAliqPtr)), ),55,15,,,F.,.T.,.F.,.F.,{ || Len(aAliqPtr) > 0 },,,F. )
2138
2139 Return
2140
2141 //-----
2142 /*{Protheus.doc} STFWzPrAg
2143 Função para programação das alquotas no ECF
2144 @param cAliquota Codigo da Aliquota
2145 @param cImposto Codigo do Imposto 1- ICMS , 2- ISS
2146 @param aAliqPtr Alíquotas do Protheus
2147 @param aAliqECF Alíquotas do ECF
2148 @param cLog Log de Execucao
2149 @param nEspacos Espacos do log

```

```

2150 @retun lPross      Aliquota programada com sucesso
2151 @obs
2152 @sample
2153 /*/
2154 //-----
2155 Function STFWzPrAg( cAliquota, cImposto, aAliqECF, aAliqPtr,;
2156                   cLog, nEspacos)
2157
2158     Local aRet := {}
2159     Local lPross := .T.
2160     Local aDados := {"17", space(30)}
2161     Local nPos := 0
2162     Local lTudo := .F.
2163     Local cLogT := ""
2164     Local nEspacoT := 0
2165     Local cTxtTmp := ""
2166
2167     DEFAULT cAliquota := ""
2168     DEFAULT cImposto := ""
2169     DEFAULT aAliqECF := {}
2170     DEFAULT aAliqPtr := {}
2171     DEFAULT cLog := ""
2172     DEFAULT nEspacos := 5
2173
2174
2175
2176     nEspacoT := nEspacos + 5
2177
2178     lTudo := cAliquota == "0"
2179
2180
2181     aRet := STFFireEvent( ProcName(0)                                ,;      // Nome do processo
2182                          "STPrinterStatus"                          ,;      // Nome do evento
2183                          {"8"                                       ,;
2184                           " " } )
2185
2186
2187     If Len(aRet) > 0 .AND. aRet[1] == 10
2188         //Redução Z Pendente
2189         cTxtTmp := STR0114 //"Redução Z Pendente. Para a programação de alíquotas, faz-se necessario que seja emitida a redução Z. "
2190         STFMessage("Z Reduction", "STOP", cTxtTmp)
2191         STFShowMessage("Z Reduction")
2192         lPross := .F.
2193         cLogT += CRLF + space(nEspacoT) + STR0027 + cTxtTmp //"[ERR0] - "
2194     Else
2195         aRet := STFFireEvent( ProcName(0)                                ,;      // Nome do processo
2196                              "STPrinterStatus"                          ,;      // Nome do evento //
2197                              aDados )
2198         If Len(aDados) < 2 .OR. Len(aRet) = 0 .OR. aRet[1] <> 0
2199             cTxtTmp := STR0116 //"Falha ao capturar Venda Bruta Diária"
2200             STFMessage("Printer Status", "STOP", STR0116) //"Falha ao capturar Venda Bruta Diária"
2201             STFShowMessage("Printer Status")

```

```

2202         cLogT += CRLF + space(nEspacoT) + "[ERRO] - " + cTxtTmp
2203         lPross := .f.
2204     Else
2205         If Val(aDados[2]) > 0
2206             cTxtTmp := STR0117 //"Já foram realizadas vendas para este dia"
2207             STFMessage("Printer Status", "STOP", cTxtTmp)
2208             STFSHOWMESSAGE("Printer Status")
2209             cLogT += CRLF + space(nEspacoT) + STR0027 + cTxtTmp //"ERRO] - "
2210             lPross := .f.
2211         EndIf
2212     EndIf
2213 EndIf
2214
2215
2216 If lPross
2217     If !lTudo
2218         aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
2219                             "STAddAliq" ,; // Nome do evento
2220                             { cAliquota, cImposto} )
2221
2222         If (lPross := Len(aRet) > 0 .AND. aRet[1] == 0)
2223             //Aliquota programada com sucesso
2224             aAdd(aAliqECF, {StrZero(Len(aAliqECF)+1, 2), cImposto, Val(cAliquota)})
2225             //Verifica se a aliquota existe no Array e muda o status
2226             cStatus := "1" //Não programado
2227             If (nPos := aScan(aAliqPtr, {|e| e[2] == cImposto .AND. e[3] == Val(cAliquota) }) > 0
2228                 aAliqPtr[nPos, 01] := "2" //Programado
2229             EndIf
2230         Else
2231             cTxtTmp := STR0118 + IIF(cImposto == "1" , STR0100, STR0101) + Transform(Val(cAliquota), "@E 99.99%") //"Falha ao programar alíquota de : "###"ICMS"###"ISS"
2232             STFMessage("AddAliq", "STOP", cTxtTmp) //STR
2233
2234             STFSHOWMESSAGE("AddAliq")
2235             cLogT += CRLF + space(nEspacoT) + "[ERRO] - " + cTxtTmp
2236         EndIf
2237     Else
2238         Do While (nPos := nPos + 1) <= Len(aAliqPtr)
2239             If aAliqPtr[nPos, 01] == "1" //Não programado
2240                 aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
2241                                     "STAddAliq" ,; // Nome do evento
2242                                     { Str(aAliqPtr[nPos, 03], 5, 2), aAliqPtr[nPos, 02] } )
2243                 lPross := Len(aRet) > 0 .AND. aRet[1] == 0
2244             EndIf
2245             If lPross
2246                 aAdd(aAliqECF, {StrZero(Len(aAliqECF)+1, 2), aAliqPtr[nPos, 02], aAliqPtr[nPos, 03]})
2247                 aAliqPtr[nPos, 01] := "2"
2248             Else
2249                 cTxtTmp := "Falha ao programar alíquota de : " + IIF(aAliqPtr[nPos, 02] == "1" , STR0100, STR0101) + Transform(aAliqPtr[nPos, 03], "@E 99.99%")
2250             // "ICMS"###"ISS"
2251                 STFMessage("AddAliq", "STOP", cTxtTmp) //STR
2252                 STFSHOWMESSAGE("AddAliq")

```

```

2253             cLogT += CRLF + space(nEspacoT) + STR0027 + cTxtTmp           //[ERR0] - "
2254             Exit
2255         EndIf
2256     EndDo
2257 EndIf
2258
2259
2260 EndIf
2261
2262     cLog += CRLF + space(nEspacos) + IIF(lPross, "", STR0072 ) + STR0119 + IIF(!lTudo," ",STR0120)+ STR0121 + cLogT  //[ERR0]#### - Programando alíquota####s ####de ECF"
2263
2264 Return lPross
2265
2266
2267 //-----
2268 /*{Protheus.doc} STFWzLdTIF
2269 Função para identificação dos totalizadores nao-fiscais no ECF
2270 @param cLG_CODIGO    Codigo da Estação
2271 @param cImpFisc      Modelo de Impressora Fiscal
2272 @param cPorta        Porta de Conexao
2273 @param lConecta      ECF Conectado
2274 @param aTotNFisc     Totalizadores não fiscais do ECF
2275 @param lTotvsApi     TotvsApi?
2276 @param cLog          Log de Execucao
2277 @param nEspacos      Espacos do log
2278 @retun  lRet         Execução com sucesso
2279 @obs
2280 @sample
2281 */
2282 //-----
2283 Function STFWzLdTIF(cLG_CODIGO, cImpFisc,  cPorta, lConecta,;
2284                   aTotNFisc, lTotvsApi,  cLog,  nEspacos )
2285
2286 Local aRet      := {} //Retorno do Strategy
2287 Local aDados    := {} //Retorno do ECF
2288 Local lRet      := .T. //Retorno da função
2289 Local cLogT     := ""  //Log temporario
2290 Local nEspacoT := 0    //Espaco Temporario
2291
2292 DEFAULT cLG_CODIGO := " " //codigo da Estação
2293 DEFAULT cImpFisc   := ""  //Impressora Fiscal
2294 DEFAULT cPorta     := ""  //Porta
2295 DEFAULT aTotNFisc  := {}   //totalizadores nao-fiscais cadastrados
2296 DEFAULT lTotvsApi  := .F.  //TotvsApi?
2297 DEFAULT cLog       := ""   //Log da rotina
2298 DEFAULT nEspacos   := 5    //Espacos
2299
2300     nEspacoT := nEspacos+5
2301
2302
2303     If !lConecta
2304         If Empty(cImpFisc)

```

```

2305
2306         cImpFisc := STFGetStat("IMPFISC")
2307     EndIf
2308     If Empty(cPorta)
2309         cPorta := STFGetStat("PORTIF")
2310     EndIf
2311
2312     lConecta := lRet := STFWzCIF(cImpFisc, cPorta, @cLogT, nEspacoT)
2313
2314 EndIf
2315
2316 If lRet
2317
2318     //VERIFICA SE É TOTVSAPI
2319     aDados := {space(30)}
2320     aRet := STFFireEvent( ProcName(0) ,;
2321                         "STRetTypePrt", ;
2322                         aDados )
2323     If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0 .AND. ValType(aDados[1]) == "C"
2324         lTotvsApi := aDados[1] == "TOTVSAPI"
2325     EndIf
2326
2327     aDados := {space(100)}
2328     aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
2329                         "STGetNotFiscalTotal",;
2330                         aDados )
2331
2332     If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0
2333         If aDados[1] == "-99" //função indisponível
2334             STFMessage("XREADER", "YESNO", STR0122) //STR //"Modelo de Impressora Fiscal não possui função para retornar os totalizadores parciais. Imprimir Leitura X?"
2335
2336             If STFSShowMessage("XREADER")
2337                 aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
2338                                     "STReadingX" ,; // Nome do evento
2339                                     {} )
2340
2341                 If Len(aRet) = 0 .OR. aRet[1] <> 0
2342                     STFMessage("XREADER2", "STOP", STR0123) //STR //"Falha na Emissão da Leitura X"
2343
2344                     STFSShowMessage("XREADER2")
2345                     lRet := .F.
2346                 EndIf
2347
2348                 cLogT += CRLF + space(nEspacoT) + IIF(lRet, "", STR0124 ) + STR0125 //"[ERRO] "###" - Emitindo Leitura X. Não foi possível realizar leitura dos
2349                 totalizadores. "
2350             EndIf
2351             aTotNFisc := {{ "XX", STR0126 }} //"MODELO NAO PERMITE IDENTIFICACAO"
2352         Else
2353             aRet := StrTokArr( aDados[1] , "|" )
2354             aEval(aRet, { |l| aAdd(aTotNFisc, StrTokArr( l , "," ))})
2355             aEval(aTotNFisc, { |l| l[1] := PadL(AllTrim(l[1]),2,"0")})
2356         EndIf

```

```

2356         Else
2357             lRet := .F.
2358         EndIf
2359
2360     EndIf
2361
2362     cLog += CRLF + IIF(lRet, "", STR0124 ) + STR0127 + cLogT           //[ERRO] "###" - Capturando Totalizadores Não-Fiscais"
2363
2364     Return lRet
2365
2366     //-----
2367     /*{Protheus.doc} STFwzLdTP
2368     Função para identificação dos totalizadores nao-fiscais no Sistema
2369     @param cLG_CODIGO    Código da Estação
2370     @param aTotPtr       Totalizadores não fiscais do Protheus
2371     @param aTotNFisc     Totalizadores não fiscais do ECF
2372     @param lTotvsApi     TotvsApi?
2373     @param cLog          Log de Execucao
2374     @param nEspacos     Espacos do log
2375     @retun .T.
2376     @obs
2377     @sample
2378     */
2379     //-----
2380     Function STFwzLdTP(cLG_CODIGO, aTotPtr, aTotNFisc, lTotvsApi,;
2381                     cLog,          nEspacos)
2382
2383     Local aRet          := {}           //Retorno do Strategy
2384     Local cTot          := ""          //Totalizador
2385     Local cGetCliDir    := GetClientDir() //Diretório smartclient
2386     Local cStatus       := "3"         //Não Identificado
2387     Local nPos          := 0           //Posicao do array
2388     Local lNoId         := .F.         //Não identificado
2389     Local cDescTot      := ""          //Descrição do Totalizador
2390     Local aLista        := {}          //Lista de Totalizadores
2391     Local aListaECF     := {}          //Lista do ECF
2392     Local nC            := 0           //Contador
2393
2394     DEFAULT cLG_CODIGO := " "         //Codigo da Estação
2395     DEFAULT aTotPtr := {}             //Chave, Seção, Valor, Arquivo - (se em branco e porque a impressora fiscal não é totvsapi, nesse caso não grava)
2396     DEFAULT aTotNFisc := {}          //Totalizador não-fiscal
2397     DEFAULT lTotvsApi := .F.         //TotvsApi
2398     DEFAULT cLog := ""               //log de execução
2399     DEFAULT nEspacos := 5            //Espacos
2400
2401     //Carrega os totalizadores da Sigaloja.ini
2402     //Recebimento de Títulos
2403
2404     lNoId := Len(aTotNFisc) == 1 .AND. aTotNFisc[1,1] == "XX"
2405
2406     //TO DO: TOTALIZADOR DE ESTORNO DE VENDA
2407     aLista := {};

```

```

2408         {"Recebimento Titulos","Totalizadores", cGetCliDir+"SIGALOJA.INI",1 }, ;
2409         {"Comprovante TEF","Totalizador", cGetCliDir+"SIGALOJA.INI", 1}, ;
2410         {"Recarga Celular","Totalizador", cGetCliDir+"SIGALOJA.INI", 1} ,;
2411         {"Correspondente Bancario","Totalizador", cGetCliDir+"SIGALOJA.INI", 1} ;
2412     }
2413
2414     If lTotvsApi
2415
2416         aAdd(aLista, {"Microsiga", "TotalizadorPedido", cGetCliDir+"TOTVSAPI.INI", 2})
2417         aAdd(aLista,{"Microsiga", "TotalizadorTefPedido", cGetCliDir+"TOTVSAPI.INI", 2})
2418         aAdd(aLista,{"Totalizadores", "Sangria", cGetCliDir+"TOTVSAPI.INI", 2})
2419         aAdd(aLista, {"Totalizadores", "Suprimento", cGetCliDir+"TOTVSAPI.INI", 2})
2420
2421     Else
2422         aAdd(aTotPtr, {"3", "", "", "Microsiga", "TotalizadorPedido", "", 0})
2423         aAdd(aTotPtr,{"3", "", "", "Microsiga", "TotalizadorTefPedido", "", 0})
2424         aAdd(aTotPtr,{"3","", "", "Microsiga", "TotalizadorTefPedido", "", 0})
2425         aAdd(aTotPtr,{"3", "", "", "Totalizadores", "Sangria", "", 0})
2426         aAdd(aTotPtr, {"3","", "", "Totalizadores", "Suprimento", "", 0})
2427
2428     EndIf
2429
2430     For nC := 1 to Len(aLista)
2431         cStatus := "3" //Não Identificado
2432         cDescTot := ""
2433         cTot := GetPvProfString(aLista[nC, 01] , aLista[nC, 02], "", aLista[nC, 03])
2434         If !Empty(cTot)
2435             If !lNoId
2436                 cStatus := "1" //Nao programado
2437                 cDescTot := ""
2438                 If (nPos := aScan(aTotNFisc, { |t| Upper(AllTrim(t[aLista[nC, 04]])) == Upper(AllTrim(cTot))} ) ) > 0
2439                     cStatus := "2" //programado
2440                     cDescTot := aTotNFisc[nPos, 02]
2441
2442                     If aLista[nC, 04] == 2
2443                         cTot := aTotNFisc[nPos, 01]
2444                     EndIf
2445                 EndIf
2446             EndIf
2447             If aLista[nC, 04] == 2 .AND. Empty(cDescTot)
2448                 cDescTot := cTot
2449                 cTot := ""
2450             EndIf
2451         EndIf
2452
2453     Else
2454         cStatus := "4" //Não cadastrado
2455     EndIf
2456     //Status //Codigo, //DESCRICAO //Seção //Chave //Arquivo //FORMA GRAVAÇÃO
2457     aAdd(aTotPtr,{ cStatus, cTot, cDescTot , aLista[nC, 01], aLista[nC, 02], aLista[nC, 03] , aLista[nC, 04]} )
2458 Next
2459

```

```

2460      cLog += CRLF + space(nEspacos) + STR0129 //" - Capturando Totalizadores Não-fiscais do Sistema"
2461
2462
2463 Return .T.
2464
2465 //-----
2466 /*{Protheus.doc} STFwzLdTP
2467 Função para cadastro/programação dos totalizadores nao-fiscais
2468 @param cLG_CODIGO    Codigo da Estação
2469 @param aTotPtr       Totalizadores não fiscais do Protheus
2470 @param aTotNFisc     Totalizadores não fiscais do ECF
2471 @param lTotvsApi     TotvsApi?
2472 @param oDLG         Janela
2473 @param nPanel       Painel
2474 @param cLog         Log de Execucao
2475 @param nEspacos     Espacos do log
2476 @obs
2477 @sample
2478 */
2479 //-----
2480 Function STFwzPrTP(cLG_CODIGO, aTotPtr, aTotNFisc, lTotvsApi, ;
2481                  oDLG, nPanel, cLog, nEspacos)
2482 //Monta o TCBrowse
2483 Local oBmp2      := LoadBitmap(GetResources(),'br_verde') //Objeto BMP Programado
2484 Local oBmp1      := LoadBitmap(GetResources(),'br_vermelho') //Objeto BMP nao programado
2485 Local oBmp3      := LoadBitmap(GetResources(),'br_amarelo') //Objeto BMP não identificado
2486 Local oBmp4      := LoadBitmap(GetResources(),'br_laranja') //Objeto BMP nao cadastrado
2487 Local aObjects   := {} //Objetos do panel
2488 Local aPosObj    := {} //Posicionamento do Objeto
2489 Local nTotaliz   := 00 //Codigo do Totalizador
2490 Local cDescr     := space(30) //Descrição do totalizador
2491 Local cChave     := space(20) //Chave do totalizador
2492 Local cSecao     := space(30) //Secao do totalizador
2493 Local cArquivo   := space(100) //Arquivo.ini do totalizador
2494 Local oBrowseECF:= NIL //Browse de totalizadores do ECF
2495 Local oBrowsePtr:= NIL //Browse de totalizadores do Protheus
2496 Local nRow       := 0 //Linhas do Panel
2497 Local nCol       := 0 //Colunas do panel
2498 Local nLargB     := 0 //Largura do Browse
2499 Local aColuna    := {} //Colunas do Browse
2500 Local nSepComp   := 3 //Colunas de espacamento
2501 Local oGet1      := NIL //Objeto Get1
2502 Local oGet2      := NIL //Objeto Get2
2503 Local oGet3      := NIL //Objeto Get3
2504 Local oGet4      := NIL //Objeto Get4
2505 Local oGet5      := NIL //Objeto Get5
2506 Local oBtn1      := NIL //Botao1
2507 Local lGrav      := .T. //Grava arquivo de configuracao
2508 Local oChk1      := NIL //Objeto check1
2509
2510 DEFAULT cLG_CODIGO := SLG->(CriaVar("LG_CODIGO")) //Estacao
2511 DEFAULT aTotPtr     := {} //totalizadores do Protheus

```

```

2512 DEFAULT aTotNFisc := {} //Totalizadores não-fiscais
2513 DEFAULT lTotvsApi := .F. //TotvsApi?
2514 DEFAULT oDlg := NIL//Janela
2515 DEFAULT nPanel := 0 //panel
2516 DEFAULT cLog := "" //log
2517 DEFAULT nEspacos := 5 //Espacos
2518
2519
2520 aColuna := { {0,0}, {0,0}, {0,0}}
2521
2522 oPanel := oDlg:GetPanel(nPanel)
2523
2524 nRow := Int( oPanel:nHeight/2) //Linha do Painel
2525 nCol := Int( oPanel:nWidth/ 2)
2526
2527 oPanel := TScrollBox():New( oPanel, 0,0,nRow,nCol)
2528 nCol := Int( nCol *.97 )
2529
2530 nLargB := (nCol/4) - 10
2531
2532 nTam := CalcFieldSize("C",Len(STR0130),0,"@!",STR0130) //1 //"Totalizadores cadastrados no ECF"###"Totalizadores cadastrados no ECF"
2533 aAdd(aObjects, { nTam, 6, .F., .F.}) //Impar Say //1
2534 aAdd(aObjects, { nLargB, 150+10, .F., .F.}) //Browse //2
2535 aAdd(aObjects, { 10, 10, .F., .F.}) //Blank Line 3
2536 nTam := CalcFieldSize("C",Len(STR0131),0,"@!",STR0131) //"Programar Totalizador"###"Programar Totalizador"
2537 aAdd(aObjects, { nTam, 12, .F., .F.}) //4
2538 aAdd(aObjects, { nTam, 6, .F., .F.}) //Impar Say //5
2539 aAdd(aObjects, { 55, 15, .F., .F.}) //6 botão programar
2540 nTam := CalcFieldSize("C",Len(STR0132),0,"",STR0132) //"Gravar Totalizador no Sistema"###"Gravar Totalizador no Sistema"
2541 aAdd(aObjects, { nTam+5, 10, .F., .F.}) //gravar //7
2542 aAdd(aObjects, { 10, 10, .F., .F.}) //Label //8
2543 aAdd(aObjects, { 10, 60, .F., .F.}) //9 Gets
2544 aAdd(aObjects, { 10, 10, .F., .F.}) //Blank Line 10
2545 aAdd(aObjects, { 10, 10, .F., .F.}) //Blank Line 11
2546
2547 aPosObj := MsObjSize( { 0 , 15 , nRow ,nCol, nSepComp, 3,3,3}, aObjects, .T., .F.)
2548
2549 //Monta o TCBrowse
2550 tSay():New( aPosObj[01, 1], aPosObj[01, 2] , { || STR0130 } ,oPanel,,,,,T,,, , ) //"Totalizadores cadastrados no ECF"
2551
2552 oBrowseECF := TCBrowse():New( aPosObj[02,01] , aPosObj[02,02], aObjects[02,01], aObjects[02,02], ,,,oPanel,,,,,,,,,F,,,T,,,F,,, )
2553 oBrowseECF:SetArray(aTotNFisc)
2554 oBrowseECF:AddColumn(TCColumn():New(STR0098,{|| aTotNFisc[oBrowseECF:nAt,01] },"@!",,,,"LEFT",CalcFieldSize("C",2,0,"@!",STR0098),.F.,.F.,,,,)) //"Código"###"Código"
2555 oBrowseECF:AddColumn(TCColumn():New(STR0133,{|| aTotNFisc[oBrowseECF:nAt,02] },"@!",,,,"LEFT",CalcFieldSize("C",Len(cDescr),0,"@!",STR0133),.F.,.F.,,,,))
//"Descrição"###"Descrição"
2556 oBrowseECF:bChange := {|| IIF(ValType(oGet1) == "0", (nTotaliz := Val(aTotNFisc[oBrowseECF:nAt,01]), cDescr := aTotNFisc[oBrowseECF:nAt,02], oGet1:Refresh(),
oGet2:Refresh() ), NIL) }
2557
2558
2559 tSay():New( aPosObj[01, 1], nLargB + 10 , { || STR0134 } ,oPanel,,,,,T,,, , ) //"Totalizadores do sistema"
2560
2561

```

```

2562 oBrowsePtr := TCBrowse():New( aPosObj[02,01] , aObjects[02,01]+13, nCol-(aObjects[02,01]+13), aObjects[02,02],,,,oPanel,,,,,,.F.,..T.,..F.,.. )
2563 oBrowsePtr:SetArray(aTotPtr)
2564 oBrowsePtr:AddColumn(TCColumn():New(STR0104,{|| IIF(aTotPtr[oBrowsePtr:nAt,01] == "1", oBmp1, IIF(aTotPtr[oBrowsePtr:nAt,01] == "2",oBmp2,IIF(aTotPtr[oBrowsePtr:nAt,01]
== "3",oBmp3,oBmp4))) },,,,,"LEFT",CalcFieldSize("C",2,0,"@!",STR0104),.T.,.F.,,,,)) //"Status"###"Status"
2565 oBrowsePtr:AddColumn(TCColumn():New(STR0098,{|| aTotPtr[oBrowsePtr:nAt,02] },"@!",,,,,"LEFT",CalcFieldSize("C",2,0,"@!",STR0098),.F.,.F.,,,,)) //"Código"###"Código"
2566 oBrowsePtr:AddColumn(TCColumn():New(STR0133,{|| aTotPtr[oBrowsePtr:nAt,03] },"@!",,,,,"LEFT",CalcFieldSize("C",Len(cDescr),0,"@!",STR0133),.F.,.F.,,,,))
//"Descrição"###"Descrição"
2567 oBrowsePtr:AddColumn(TCColumn():New(STR0135,{|| aTotPtr[oBrowsePtr:nAt,04] },"@!",,,,,"LEFT",CalcFieldSize("C",Len(cChave),0,"@!",STR0135),.F.,.F.,,,,))
//"Chave"###"Chave"
2568 oBrowsePtr:AddColumn(TCColumn():New(STR0136,{|| aTotPtr[oBrowsePtr:nAt,05] },"@!",,,,,"LEFT",CalcFieldSize("C",Len(cSecao),0,"@!",STR0136),.F.,.F.,,,,))
//"Seção"###"Seção"
2569 oBrowsePtr:AddColumn(TCColumn():New(STR0137,{||aTotPtr[oBrowsePtr:nAt,06] },"@!",,,,,"LEFT",CalcFieldSize("C",Len(cArquivo),0,"@!",STR0137),.F.,.F.,,,,))
//"Arquivo"###"Arquivo"
2570 oBrowsePtr:bChange := {|| IIF(ValType(oGet1) == "0", ( nTotaliz := IIF(Empty(nTotaliz), Val(aTotPtr[oBrowsePtr:nAt,02]), nTotaliz) , cDescr := IIF(Empty(cDescr),
IIF(Empty(aTotPtr[oBrowsePtr:nAt,03]), aTotPtr[oBrowsePtr:nAt,04], aTotPtr[oBrowsePtr:nAt,03] ), cDescr) , cChave := aTotPtr[oBrowsePtr:nAt,04], cSecao :=
aTotPtr[oBrowsePtr:nAt,05], cArquivo := aTotPtr[oBrowsePtr:nAt,06], lGrav := aTotPtr[oBrowsePtr:nAt,07] > 0, oGet1:Refresh(), oGet2:Refresh(), oGet3:Refresh(),
oGet4:Refresh(), oGet5:Refresh(), oBtn1:Refresh(), oChk1:Refresh() ) , NIL ) }
2571 oBrowsePtr:BHeaderClick := { || STFWzShowLg( { {"BR_VERDE", STR0109},; //"Programado"
2572 {"BR_VERMELHO", STR0110},; //"Não programado"
2573 {"BR_AMARELO", STR0076},; //"Não identificado"
2574 {"BR_LARANJA", STR0107} }, STR0138 ) } //"Não cadastrado"###"Legenda Tot. Sistema"
2575
2576 @aPosObj[03,01], 5 to aPosObj[11, 1] ,nCol OF oPanel PIXEL
2577 tSay():New( aPosObj[04, 1], aPosObj[04, 2]+15 , { || STR0131 } ,oPanel,,,,,.T.,.. , ) //"Programar Totalizador"
2578
2579 //Linha 5 - label da Programação
2580 nTam := CalcFieldSize("C",2,0,"@!",STR0139) //Tamanho do Get Codigo //"Codigo"
2581
2582
2583 aColuna[1,1] := aPosObj[05, 2]+15
2584 aColuna[1,2] := nTam
2585 tSay():New( aPosObj[05, 1], aColuna[1,1] , { || STR0139 } ,oPanel,,,,,.T.,.. , ) //"Codigo"
2586
2587 aColuna[2,1] := aColuna[1, 1] + nTam + nSepComp
2588 nTam := CalcFieldSize("C",Len(cDescr),0,"@!",STR0133) //Tamanho do Get Codigo //"Descrição"
2589 aColuna[2,2] := nTam
2590 tSay():New( aPosObj[05, 1], aColuna[2,1] , { || STR0133 } ,oPanel,,,,,.T.,.. , ) //"Descrição"
2591
2592 aColuna[3,1] := aColuna[2, 1] + nTam + nSepComp
2593 aColuna[3,2] := aObjects[06,01]
2594
2595 //Linha 6 - Gets da Programação
2596 oGet1 := TGet():New( aPosObj[06, 1],aColuna[1,1],{ ||U| Iif(PCOUNT() > 0, nTotaliz := U, nTotaliz)},oPanel, aColuna[1,2],10,"@E 99",{|| nTotaliz >= 0 },,,, .T.,.. .T.,..
.T.,..
.F.,..F.,..f.,..F.,..,"nTotaliz",,,,T.,)
2597 oGet2 := TGet():New( aPosObj[06, 1],aColuna[2,1],{ ||U| Iif(PCOUNT() > 0, cDescr := U, cDescr)},oPanel, aColuna[2,2],10,{|| !Empty(cDescr) },,,, .T.,.. .T.,.. .T.,..
.F.,..F.,..f.,..F.,..,"cDescr",,,,T.,)
2598 oBtn1 := TButton():New(aPosObj[06, 1], aColuna[3,1], STR0112,oPanel,{|| IIF( STFWzPrTg( @nTotaliz, @cDescr, cChave, cSecao, ; //"Programar"
2599 @Arquivo, @aTotNFisc, @aTotPtr, lGrav, ;
2600 @cLog, nEspacos),;
2601 (oBrowseECF:SetArray(aTotNFisc), oBrowsePtr:SetArray(aTotPtr)), ), ;
2602 oGet1:Refresh(), oGet2:Refresh() },aObjects[06,01],aObjects[06,02],,,,F.,..T.,..F.,..F.,{
|| nTotaliz > 0 .AND. !Empty(cDescr) },,.F. )

```

```

2603 oBtn2 := TButton():New(aPosObj[06, 1], aColuna[3,1]+aObjects[06,01]+nSepComp, STR0140,oPanel,{|| IIF( STFWzGrTg( @nTotaliz, @cDescr, cSecao, ; //"Gravar Sistema"
2604                                     cArquivo, @aTotPtr, lGrav, @cLog, ;
2605                                     nEspacos),;
2606                                     oBrowsePtr:SetArray(aTotPtr), );;
2607                                     oGet1:Refresh(), oGet2:Refresh()
                                     },aObjects[06,01],aObjects[06,02],,,,F.,.T.,.F.,.F.,{ || nTotaliz > 0 .AND. !Empty(cDescr) .AND. lGrav .and. !Empty(cArquivo) },,.F. )

2608
2609 //Linha - Check de Gravação
2610 oChk1 := TCheckBox():New(aPosObj[7, 01], aColuna[1,1],STR0132,{ |U| Iif(PCOUNT() > 0, lGrav := U, lGrav)},oPanel, aPosObj[7, 03],aPosObj[7, 04],,,,,,T.,,) //"Gravar
2611 Totalizador no Sistema"

2612
2613 aColuna := { {0,0}, {0,0}, {0,0}}
2614 //Linha 8 - Label do browse do sistema
2615 nTam := CalcFieldSize("C",Len(cChave),0,"@",STR0135) //Tamanho do Get Chave //"Chave"
2616 aColuna[1,1] := aPosObj[08, 2]+15
2617 aColuna[1,2] := nTam
2618 tSay():New( aPosObj[08, 1], aColuna[1,1] , { || STR0135 } ,oPanel,,,,,T.,, , ) //"Chave"
2619
2620 aColuna[2,1] := aColuna[1, 1] + nTam + nSepComp
2621 nTam := CalcFieldSize("C",Len(cSecao),0,"@",STR0136) //Tamanho do Get Seção //"Seção"
2622 aColuna[2,2] := nTam
2623 tSay():New( aPosObj[08, 1], aColuna[2,1] , { || STR0136 } ,oPanel,,,,,T.,, , ) //"Seção"
2624
2625 aColuna[3,1] := aColuna[2, 1] + nTam + nSepComp
2626 nTam := CalcFieldSize("C",Len(cArquivo),0,"@",STR0137) //"Arquivo"
2627 aColuna[3,2] := nTam
2628 tSay():New( aPosObj[08, 1], aColuna[3,1] , { || STR0137 } ,oPanel,,,,,T.,, , ) //"Arquivo"
2629
2630 //Linha 9 - Gets do browse do Sistema - não editável
2631 oGet3 := TGet():New( aPosObj[09, 1],aColuna[1,1],{ |U| Iif(PCOUNT() > 0, cChave := U, cChave)},oPanel, aColuna[1,2],10,"",{ || .t.},,,, .T.,, .T.,, .T.,, , .F.,
.f.,, .T., .F.,,"cChave",,,,T.,)
2632 oGet4 := TGet():New( aPosObj[09, 1],aColuna[2,1],{ |U| Iif(PCOUNT() > 0, cSecao := U, cSecao)},oPanel, aColuna[2,2],10,"",{ || .t.},,,, .T.,, .T.,, .T.,, , .F.,
.F.,, .T., .F.,,"cSecao",,,,T.,)
2633 oGet5 := TGet():New( aPosObj[09, 1],aColuna[3,1],{ |U| Iif(PCOUNT() > 0, cArquivo := U, cArquivo)},oPanel, aColuna[3,2],10,,,,, .T.,, .T.,, .T.,, , .F., .F.,
.T., .F.,,"cArquivo",,,,T.,)

2634
2635 Return
2636
2637 //-----
2638 /*{Protheus.doc} STFWzPrTg
2639 Função para programação para totalizador
2640 @param nTotaliz      Numero do Totalizador
2641 @param cDescr        Descrição do totalizador
2642 @param cChave        Chave do arquivo de totalizador
2643 @param cSecao        Secao do arquivo de totalizador
2644 @param cArquivo      Arquivo de totalizador
2645 @param aTotNFisc     Array dos totalizadores nao-fiscais do ECF
2646 @param aTotPtr        Array dos totalizadores do Sistema
2647 @param lGrav         Gravar o arquivo configuração
2648 @param cLog          Log de Execução da rotina
2649 @param nEspacos      Espacos do log

```

```

2650 @return lPross      Totalizador programado
2651 @obs
2652 @sample
2653 /*/
2654 //-----
2655
2656 Function STFWzPrTg( nTotaliz, cDescr,   cChave,      cSecao, ;
2657                   cArquivo, aTotNFisc, aTotPtr,   lGrav, ;
2658                   cLog, nEspacos)
2659
2660 Local aRet          := {} //Retorno da funcao
2661 Local lPross        := .T. //Retorno da funcao
2662 Local aDados        := {"17", space(30)} //Dados da rotina
2663 Local nPos          := 0   //Posicao do array
2664 Local lTudo         := .F. //Grava tudo
2665 Local cTotaliz      := ""  //Codigo do totalizador
2666 Local nPosT         := 0   //Posicao do totalizador
2667 Local nPosP         := 0   //Posicao do totalizador
2668 Local cValor        := ""  //Valor do totalizador
2669 Local cLogT         := ""  //Log temporario
2670 Local nEspacoT      := 0   //Espaco temporario
2671 Local cTxtTmp       := ""  //Texto temporario
2672
2673 DEFAULT nTotaliz    := 0    //Codigo do totalizador
2674 DEFAULT cDescr      := ""   //Descricao do totalizador
2675 DEFAULT cChave      := ""   //Chave do arquivo de configuracao
2676 DEFAULT cSecao      := ""   //Secao do totalizador
2677 DEFAULT cArquivo    := ""   //Arquivo do totalizador
2678 DEFAULT aTotNFisc    := {}   //Array de totalizadores não-fiscais
2679 DEFAULT aTotPtr      := {}   //Array de totalizadores do protheus
2680 DEFAULT lGrav       := .F.   //Grava o arquivo de totalizadores
2681 DEFAULT cLog        := ""   //Log de execucao
2682 DEFAULT nEspacos    := 5    //Espacos de execucao
2683
2684 lTudo := cTotaliz == "0"
2685
2686 nEspacoT := nEspacos + 5
2687
2688 aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
2689                     "STPrinterStatus" ,; // Nome do evento
2690                     {"8" ,;
2691                      " " } )
2692
2693
2694 If Len(aRet) > 0 .AND. aRet[1] == 10
2695 //Redução Z Pendente
2696 cTxtTmp := STR0141 //"Redução Z Pendente. Para a programação de Totalizador Não-Fiscal, faz-se necessario que seja emitida a redução Z. "
2697 STFMessage("Z Reduction", "STOP", cTxtTmp)
2698 STFShowMessage("Z Reduction")
2699 lPross := .F.
2700 cLogT += CRLF + space(nEspacoT) + "[ERR0] " + "- " + cTxtTmp
2701 Else

```

```

2702     aRet := STFFireEvent(  ProcName(0)                                ,;      // Nome do processo
2703                          "STPrinterStatus"                          ,;      // Nome do evento
2704                          aDados )
2705   If Len(aDados) < 2 .OR. Len(aRet) =0 .OR. aRet[1] <> 0
2706     cTxtTmp := STR0116 //"Falha ao capturar Venda Bruta diária"
2707     STFMessage("Printer Status", "STOP", cTxttmp )
2708     STFShowMessage("Printer Status")
2709     lPross := .F.
2710     cLogT += CRLF + space(nEspacoT) + STR0124 + "- " + cTxtTmp  //"[ERR0] "
2711   Else
2712     If Val(aDados[2]) > 0
2713       cTxtTmp := STR0117 //"Já foram realizadas vendas para este dia"
2714       STFMessage("Printer Status", "STOP", cTxtTmp)
2715       STFShowMessage("Printer Status")
2716       lPross := .f.
2717       cLogT += CRLF + space(nEspacoT) + STR0124 + "- " + cTxtTmp  //"[ERR0] "
2718     EndIf
2719   EndIf
2720 EndIf
2721
2722 If lPross
2723   If !lTudo
2724     cTotaliz := StrZero(nTotaliz,2)
2725     If lGrav .AND. ( nPosP := aScan( aTotPtr, { |p| AllTrim(p[2]) == cTotaliz .AND.;
2726                                     AllTrim(p[4]) == AllTrim(cChave) .AND. ;
2727                                     AllTrim(p[5]) == AllTrim(cSecao) .AND. ;
2728                                     AllTrim(p[6]) == AllTrim(cArquivo ) } ) ) > 0
2729       If aTotPtr[nPosP, 01] == "2"
2730         cTxtTmp := STR0142 //"Totalizador já programado."
2731         STFMessage("TOTPROG","STOP",cTxtTmp) //STR
2732
2733         STFShowMessage("TOTPROG")
2734         lPross := .F.
2735
2736         cLogT += CRLF + space(nEspacoT) + STR0124 + "- " + cTxtTmp  //"[ERR0] "
2737       EndIf
2738     EndIf
2739     If ( nPosT := aScan( aTotNFisc, { |t| AllTrim(t[1]) == cTotaliz .AND. ;
2740                             Upper(AllTrim(t[2])) == Upper(AllTrim(cDescr)) } ) ) > 0
2741       cTxtTmp := STR0142 //"Totalizador já programado."
2742       STFMessage("TOTPROG","STOP",cTxtTmp) //STR
2743
2744       STFShowMessage("TOTPROG")
2745       lPross := .F.
2746       cLogT += CRLF + space(nEspacoT) + STR0124 + "- " + cTxtTmp  //"[ERR0] "
2747     EndIf
2748
2749   If lPross
2750     //Prossegue então programa a alíquota
2751
2752     aRet := STFFireEvent(  ProcName(0)                                ,;      // Nome do processo
2753                          "STNotFiscalTotal"                          ,;      // Nome do evento

```

```

2754             {cTotaliz, cDescr})
2755
2756
2757     If Len(aDados) < 2 .OR. Len(aRet) = 0 .OR. aRet[1] <> 0
2758         cTxtTmp := STR0143 + cTotaliz + " " + cDescr //"Falha ao Programar Totalizador "
2759         STFMessage("TOTPROGERR","STOP", cTxtTmp ) //STR
2760         STFShowMessage("TOTPROGERR")
2761         lPross := .F.
2762         cLogT += CRLF + space(nEspacoT) + STR0124 + " - " + cTxtTmp //"[ERR0] "###"- "
2763     Else
2764
2765         If nPosP > 0
2766             If aTotPtr[nPosP, 01] <> "3" .AND. aTotPtr[nPosP, 07] > 0 //Nao Identificado sem coluna de gravação
2767                 cValor := IIF(aTotPtr[nPosP, 07] == 1,cTotaliz, cDescr)
2768                 WritePProString(aTotPtr[nPosP, 04] , aTotPtr[nPosP, 05], cValor, aTotPtr[nPosP, 06])
2769             EndIf
2770             aTotPtr[nPosP, 01] := "2" //Programado
2771             aTotPtr[nPosP, 02] := cTotaliz
2772             aTotPtr[nPosP, 03] := cDescr
2773         EndIf
2774
2775         aAdd(aTotNFisc, {cTotaliz, cDescr})
2776     EndIf
2777
2778 EndIf
2779
2780 EndIf
2781
2782 EndIf
2783
2784 EndIf
2785 nTotaliz := 0
2786 cDescr := space(30)
2787
2788 cLog += CRLF + space(nEspacos) + IIF(lPross, "", STR0124) + STR0145 + cLogT //"[ERR0] "###"- Programando Totalizador Não-Fiscal"
2789
2790 Return lPross
2791
2792
2793 //-----
2794 /*{Protheus.doc} STFWzGrTg
2795 Função para gravação do totalizador
2796 @param nTotaliz      Numero do Totalizador
2797 @param cDescr        Descrição do totalizador
2798 @param cChave        Chave do arquivo de totalizador
2799 @param cSecao        Secao do arquivo de totalizador
2800 @param cArquivo      Arquivo de totalizador
2801 @param aTotPtr       Array dos totalizadores do Sistema
2802 @param lGrav         Gravar o arquivo configuração
2803 @param cLog          Log de execucao da rotina
2804 @param nEspacos      Espacos do log
2805 @return lPross       Totalizador gravado no arquivo de totalizador

```

```

2806 @obs
2807 @sample
2808 /*/
2809 //-----
2810 Function STFwzGrTg( nTotaliz, cDescr,  cChave, cSecao,;
2811                   cArquivo, aTotPtr,  lGrav,  cLog,;
2812                   nEspacos)
2813
2814 Local lPross      := .T.  //Execucao da rotina
2815 Local nPos        := 0    //Posicao do array
2816 Local lTudo       := .F.  //Programa todos os totalizadores
2817 Local cTotaliz    := ""   //Codigo do totalizador
2818 Local nPosP       := 0    //Posicao do totalizador
2819 Local cValor      := ""   //Valor do totalizador
2820 Local cLogT       := ""   //Log temporario
2821 Local nEspacoT    := 0    //Espaco temporario
2822 Local cTxtTmp     := ""   //Arquivo texto
2823
2824 DEFAULT nTotaliz := 0      //Codigo do totalizador
2825 DEFAULT cDescr   := ""    //Descricao do totalizador
2826 DEFAULT cChave   := ""    //Chave do totalizador
2827 DEFAULT cSecao   := ""    //Secao do totalizador
2828 DEFAULT cArquivo:= ""     //Arquivo do totalizador
2829 DEFAULT aTotPtr  := {}    //Totalizador do sistema
2830 DEFAULT lGrav    := .F.   //Grava o arquivo
2831 DEFAULT cLog     := ""    //Log de execucao
2832 DEFAULT nEspacos:= 5     //Espacos do log
2833
2834
2835 lTudo := cTotaliz == ""
2836
2837 nEspacoT := nEspacos + 5
2838
2839 If !lTudo
2840   cTotaliz := StrZero(nTotaliz,2)
2841   If ( nPosP := aScan( aTotPtr, { |p| AllTrim(p[2]) == cTotaliz .AND.;
2842                           AllTrim(p[4]) == AllTrim(cChave) .AND. ;
2843                           AllTrim(p[5]) == AllTrim(cSecao) .AND. ;
2844                           AllTrim(p[6]) == AllTrim(cArquivo ) } ) ) > 0
2845     If aTotPtr[nPosP, 01] == "2"
2846       cTxtTmp := STR0146 //"Totalizador já configurado"
2847       STFMessage("TOTPROG","STOP", cTxtTmp) //STR
2848       STFSHOWMESSAGE("TOTPROG")
2849       cLogT := CRLF + space(nEspacoT) + STR0027 + cTxtTmp //" [ERR0] - "
2850       lPross := .F.
2851     EndIf
2852   Else
2853     If ( nPosP := aScan( aTotPtr, { |p| AllTrim(p[4]) == AllTrim(cChave) .AND. ;
2854                               AllTrim(p[5]) == AllTrim(cSecao) .AND. ;
2855                               AllTrim(p[6]) == AllTrim(cArquivo ) } ) ) = 0

```

```

2858         cTxtTmp := STR0147 //"Totalizador não cadastrado no sistema"
2859         STFMessage("TOTPROG","STOP", cTxtTmp) //STR
2860
2861         //lPross :=
2862         STFShowMessage("TOTPROG")
2863         lPross := .F.
2864         cLogT := CRLF + space(nEspacoT) + STR0027 + cTxtTmp //" [ERR0] - "
2865     EndIf
2866
2867 EndIf
2868
2869 If lPross
2870
2871     If nPosP > 0
2872         If aTotPtr[nPosP, 01] <> "3" .AND. aTotPtr[nPosP, 07] > 0 //Nao Identificado sem coluna de gravação
2873             cValor := IIF(aTotPtr[nPosP, 07] == 1,cTotaliz, cDescr)
2874             WritePProString(aTotPtr[nPosP, 04] , aTotPtr[nPosP, 05], cValor, aTotPtr[nPosP, 06])
2875         EndIf
2876         aTotPtr[nPosP, 01] := "2" //Programado
2877         aTotPtr[nPosP, 02] := cTotaliz
2878         aTotPtr[nPosP, 03] := cDescr
2879     EndIf
2880
2881 EndIf
2882
2883 EndIf
2884 nTotaliz := 0
2885 cDescr := space(30)
2886
2887 cLog += CRLF + space(nEspacos) + IIF(lPross, "", STR0072) + STR0148 //" [ERR0]###" - Realizando a gravação dos Totalizadores Não-Fiscais nos arquivos do Sistema"
2888
2889 Return lPross
2890
2891
2892 //-----
2893 /*{Protheus.doc} STFWzLdFIF
2894 Carrega as formas de pagamento do ECF
2895 @param cLG_CODIGO    Codigo da Estacao
2896 @param cImpFisc      Impressora Fiscal
2897 @param cPorta        Porta da Impressora Fiscal
2898 @param lConecta      Impressora Conectada
2899 @param aFormaECF     Forma de pagamento da impressora
2900 @param lTotvsApi     Impressora utiliza TotvsApi?
2901 @param cLog          Log de execucao da rotina
2902 @param nEspacos      Espacos do log
2903 @return lRet         Forma de pagamento extraída com sucesso
2904 @obs
2905 @sample
2906 */
2907 //-----
2908 Function STFWzLdFIF(cLG_CODIGO, cImpFisc, cPorta, lConecta,
2909                   aFormaECF, lTotvsApi, cLog, nEspacos)

```

```

2910
2911 Local lRet      := .T. //Retorno da rotina
2912 Local aRet      := {}  //Retorno do Strategy
2913 Local aDados    := {}  //Dados de retorno da funcao
2914 Local cLogT     := ""   //log temporario
2915 Local nEspacoT  := 0    //Espaco temporario
2916
2917 DEFAULT cLG_CODIGO := "" //Codigo da Estacao
2918 DEFAULT cImpFisc   := "" //Modelo de Impressora fiscal
2919 DEFAULT cPorta     := "" //Porta de Comunicação
2920 DEFAULT lConecta   := .F. //Impressora Conectada
2921 DEFAULT aFormaECF  := {} //Forma de pagamento cadastrada no ECF
2922 DEFAULT lTotvsApi  := .F. //Impressora cadastrada na totvsApi
2923 DEFAULT cLog       := "" //Log de execucao da rotina
2924 DEFAULT nEspacos   := 5  //Espacos de execucao da rotina
2925
2926     nEspacoT := nEspacos + 5
2927
2928     aFormaECF := {}
2929
2930     If !lConecta
2931         If Empty(cImpFisc)
2932             cImpFisc := STFFireEvent("IMPFISC")
2933         EndIf
2934         If Empty(cPorta)
2935             cPorta := STFFireEvent("PORTIF")
2936         EndIf
2937     EndIf
2938
2939     lConecta := lRet := STFWzCIF(cImpFisc, cPorta, @cLogT, nEspacoT)
2940
2941 EndIf
2942
2943 If lRet
2944
2945     //VERIFICA SE É TOTVSAPI
2946     aDados := {space(30)}
2947     aRet := STFFireEvent( ProcName(0) , ;
2948                          "STRetTypePrt", ;
2949                          aDados )
2950     If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0 .AND. ValType(aDados[1]) == "C"
2951         lTotvsApi := aDados[1] == "TOTVSAPI"
2952     EndIf
2953
2954 EndIf
2955
2956 If lRet
2957     //Busca as Formas de pagamento
2958     aDados := {space(300), NIL}
2959     aRet := STFFireEvent( ProcName(0) , ;          // Nome do processo
2960                          "STGetConPay"           , ;          // Nome do evento
2961                          aDados )

```

```

2962
2963         If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0
2964             aFormaECF := StrTokArr( aDados[1] , "|" )
2965
2966         Else
2967
2968             lRet := .F.
2969         EndIf
2970
2971     EndIf
2972
2973
2974
2975     cLog += CRLF + space(nEspacos) + IIF(lRet, "", STR0072) + STR0149 + cLogT //"[ERRO]"###" - Verificando formas de pagamento cadastradas no ECF"
2976
2977 Return lRet
2978
2979 //-----
2980 /*{Protheus.doc} STFWzLdFP
2981 Carrega as formas de pagamento do sistema
2982 @param cLG_CODIGO    Codigo da Estacao
2983 @param cImpFisc      Impressora Fiscal
2984 @param cPorta        Porta da Impressora Fiscal
2985 @param aFormaPtr     Formas de pagamento do Protheus
2986 @param aFormaECF     Forma de pagamento da impressora
2987 @param lTotvsApi     Impressora utiliza TotvsApi?
2988 @param cLog          Log de execucao da rotina
2989 @param nEspacos      Espacos do log
2990 @return lRet         Forma de pagamento extraída com sucesso
2991 @obs
2992 @sample
2993 */
2994 //-----
2995 Function STFWzLdFP( cLG_CODIGO, cImpFisc, aFormaPtr, aFormaECF,;
2996                   lTotvsApi, cLog, nEspacos)
2997
2998 Local cGetCliDir := GetClientDir() //Local do smartclient
2999 Local cStatus   := "2"           //Não Programado
3000 Local nPos      := 0             //Posicao do array
3001 Local cDescTot  := ""           //Descricao do totalizador
3002 Local nC        := 0            //Variavel contadora
3003 Local aAreaSAE  := {}           //WorkArea SAE
3004 Local aAreaMDE  := {}           //WorkArea MDE
3005 Local aArea     := GetArea()     //WorkArea ativa
3006 Local cForma    := ""           //Forma de pagamento
3007 Local aAreaSX5  := SX5->(GetArea()) //Descricao da tabela SX5
3008 Local cTabela   := PadR( "24" , LEN(SX5->X5_TABELA) ) //Chave da tabela 24 - Formas de pagamento
3009 Local cFilialX5 := SX5->(xFilial() ) //Filial SX5
3010 Local lPOS      := FindFunction("STFIsPOS") .AND. STFIsPOS() //Pos?
3011
3012 DEFAULT cLG_CODIGO := " "       //Codigo da Estacao
3013 DEFAULT cImpFisc   := ""        //Impressora Fiscal

```

```

3014 DEFAULT aFormaECF      := {}          //Chave, Seção, Valor, Arquivo - (se em branco e porque a impressora fiscal não é totvsapi, nesse caso não grava)
3015 DEFAULT aFormaPtr      := {}          //Forma de pagamento do sistema
3016 DEFAULT lTotvsApi       := .F.         //impressora totvsApi?
3017 DEFAULT cLog            := ""          //log da impressora fiscal
3018 DEFAULT nEspaces        := 5           //espacos do log
3019
3020     aFormaPtr := {}
3021
3022     //Carrega os totalizadores da Sigaloja.ini
3023     //Recebimento de Títulos
3024     cForma := GetPvProfString("Condicao de Pagamento", "Retencoes", "RETENCOES", cGetCliDir+"SIGALOJA.INI")
3025     aAdd(aFormaPtr, {"", AllTrim(cForma), "Retencoes", "Condição de Pagamento de Retenções " + cGetCliDir+"SIGALOJA.INI"})
3026     cForma := GetPvProfString("Recarga Celular", "FormaPgto", " ", cGetCliDir+"SIGALOJA.INI" )
3027     If !Empty(cForma) //to do Validar sitef
3028         aAdd(aFormaPtr, { "", AllTrim(cForma), "Recarga Celular", STR0150 + cGetCliDir+"SIGALOJA.INI"}) // "Forma de pagamento para recarga de celular Sitef "
3029     EndIf
3030
3031     cForma := &(SuperGetMV("MV_NATRECE", NIL, "RECEBIMENTO"))
3032
3033     If !Empty(cForma)
3034         aAdd(aFormaPtr, { "", AllTrim(cForma), "Recebimento", "Recebimento de títulos - parâmetro MV_NATRECE " + cGetCliDir+"SIGALOJA.INI"})
3035     EndIf
3036
3037     //Busca as formas de pagamento
3038     If SX5->(DbSeek(cFilialX5 + cTabela))
3039         Do While SX5->(!Eof()) .AND. X5_FILIAL + X5_TABELA == cFilialX5 + cTabela
3040             aAdd(aFormaPtr, { "", AllTrim(SX5->(X5Descri())), SX5->X5_CHAVE, STR0151}) // "Forma de Pagamento tabela 24"
3041             SX5->(DbSkip(1))
3042         EndDo
3043     EndIf
3044
3045     If lTotvsApi
3046         cForma := GetPvProfString("Microsiga", "FormaPgto", "A VISTA", cGetCliDir + "TOTVSAPI.INI")
3047         aAdd(aFormaPtr, { "", AllTrim(cForma), "FormaPgto", STR0152 + cGetCliDir+"TOTVSAPI.INI"}) // "Forma de Pagamento para Pedido"
3048     EndIf
3049
3050     If "DARUMA FS345" $ cImpFisc
3051         cForma := GetPvProfString("DARUMA", "SUPRIMENTO", "", cGetCliDir+"SIGALOJA.INI")
3052         If !Empty(cForma)
3053             aAdd(aFormaPtr, { "", AllTrim(cForma), "SUPRIMENTO", STR0153 + cImpFisc}) // "Forma de Suprimento da Impressora "
3054         EndIf
3055     EndIf
3056
3057     IF SuperGetMV("MV_LJPAGTO", NIL, 1) == 2
3058         aAreaSAE := SAE->(GetArea()) //WorkArea Anterior SAE
3059         aAreaMDE := MDE->(GetArea()) //WorkArea Anterior MDE
3060
3061         DbSelectArea("MDE")
3062         MDE->(DbSetOrder(1)) //MDE_FILIAL+MDE_CODIGO
3063         DbSelectArea("SAE")

```

```

3066      DbSetOrder(1) //AE_FILIAL+AE_COD
3067      DbSeek(xFilial("SAE"))
3068
3069      While !SAE->(Eof()) .AND. SAE->AE_FILIAL == xFilial("SAE")
3070          //Ascan para não duplicar a descrição
3071          If ( !POS .OR. SuperGetMv("MV_LJADMFI",,.F.) ) .AND. MDE->(DbSeek(xFilial("MDE")+SAE->AE_ADMCART ))
3072              //7
3073              cForma := AllTrim( MDE->MDE_DESC)
3074              Else
3075              cForma := AllTrim(SAE->AE_DESC)
3076              EndIf
3077              If (nPos := aScan( aFormaPtr, { |p| Upper(p[2]) == Upper(cForma) }) = 0
3078                  aAdd(aFormaPtr , { "", AllTrim(cForma), STR0154 + SAE->AE_ADMCART, STR0155})
3079                  EndIf
3080                  SAE->(DbSkip())
3081
3082
3083      End
3084
3085      RestArea(aAreaSAE)
3086      RestArea(aAreaMDE)
3087
3088
3089      EndIf
3090      //Realiza a consistência
3091      For nC := 1 to Len(aFormaPtr)
3092          If aScan(aFormaECF, { |e| AllTrim(e) == aFormaPtr[nC, 02] } ) > 0
3093              aFormaPtr[nC, 01] := "2" //programado
3094          Else
3095              aFormaPtr[nC, 01] := "1" //Não Programado
3096          EndIf
3097      Next nC
3098
3099      RestArea(aAreaSX5)
3100      RestArea(aArea)
3101
3102      cLog += CRLF + space(nEspacos) + STR0156 //"Capturando as Formas de Pagamento do Sistema."
3103
3104      Return .T.
3105
3106      //-----
3107      /*{Protheus.doc} STFwzPrFP
3108      Interface para programacao das formas de pagamento
3109      @param cLG_CODIGO      Codigo da Estacao
3110      @param aFormaPtr      Formas de pagamento do Protheus
3111      @param aFormaECF      Forma de pagamento da impressora
3112      @param lTotvsApi      Impressora utiliza TotvsApi?
3113      @param oDLG           Janela
3114      @param nPanel         Painel atual
3115      @param cLog           Log de execucao da rotina

```

```

3117 @param nEspacos      Espacos do log
3118 @obs
3119 @sample
3120 /*/
3121 //-----
3122 Function STFwzPrFP(cLG_CODIGO, aFormaPtr,   aFormaECF, lTotvsApi, ;
3123                  oDLG,      nPanel,      cLog,      nEspacos)
3124
3125 Local oBmp2      := LoadBitmap(GetResources(),'br_verde')    //Objeto BMP programado
3126 Local oBmp1      := LoadBitmap(GetResources(),'br_vermelho') //Objeto BMP nao programado
3127 Local aObjects   := {}                                     //Objetos da janela
3128 Local aPosObj    := {}                                     //Posicao dos objetos
3129 Local cDescr     := space(20)                             //Descricao dos objetos
3130 Local oBrowseECF:= NIL                                     //Browse do ECF
3131 Local oBrowsePtr:= NIL                                     //Browse do Protheus
3132 Local nRow       := 0                                     //Linhas do Panel
3133 Local nCol       := 0                                     //Colunas do panel
3134 Local nLargB     := 0                                     //Largura do Browse
3135 Local nSepComp   := 3                                     //Separacao dos componentes
3136 Local oGet1      := NIL                                   //Get1
3137 Local oBtn1      := NIL                                   //Botao1
3138 Local nColT      := 0                                     //coluna temporaria
3139
3140 DEFAULT cLG_CODIGO := SLG->(CriaVar("LG_CODIGO"))          //codigo da estacao
3141 DEFAULT oDlg       := NIL                                   //janela
3142 DEFAULT aFormaPtr  := {}                                    //forma de pagamento do sistema
3143 DEFAULT aFormaECF  := {}                                    //forma de pagamento do ECF
3144 DEFAULT lTotvsApi  := .F.                                  //totvsApi
3145 DEFAULT nPanel     := 1                                    //Panel atual
3146 DEFAULT cLog       := ""                                   //log da rotina
3147 DEFAULT nEspacos   := 5                                    //Espacos
3148
3149 oPanel := oDlg:GetPanel(nPanel)
3150 nRow   := Int( oPanel:nHeight/2) //Linha do Painel
3151 nCol   := Int( oPanel:nWidth/ 2)
3152 oPanel := TScrollBox():New( oPanel, 0,0,nRow,nCol)
3153 nCol := Int( nCol *.97 )
3154 nLargB := (nCol/4) - 10
3155
3156 nTam := CalcFieldSize("C",Len(STR0157),0,"@!",STR0157) //1 //"Formas cadastradas no ECF"###"Formas cadastradas no ECF"
3157 aAdd(aObjects, { nTam, 6, .F., .F.}) //Impar Say //1
3158 aAdd(aObjects, { nLargB, 150+10, .F., .F.}) //Browse //2
3159 aAdd(aObjects, { 10, 10, .F., .F.}) //Blank Line 3
3160 nTam := CalcFieldSize("C",Len(STR0158),0,"@!",STR0158) //"Programar Forma"###"Programar Forma"
3161 aAdd(aObjects, { nTam, 12, .F., .F.}) //4
3162 aAdd(aObjects, { nTam, 6, .F., .F.}) //Impar Say //5
3163 aAdd(aObjects, { 55, 15, .F., .F.}) //6 botão programar
3164 aAdd(aObjects, { 10, 10, .F., .F.}) //Blank Line 7
3165 aAdd(aObjects, { 10, 10, .F., .F.}) //Blank Line 8
3166
3167
3168 aPosObj := MsObjSize( { 0 , 15 , nRow ,nCol, nSepComp, 3,3,3}, aObjects, .T., .F.)

```

```

3169
3170
3171 //oPanel := oDlg:GetPanel(nPanel)
3172
3173 //Monta o TCBrowse
3174 tSay():New( aPosObj[01, 1], aPosObj[01, 2] , { || STR0157 } ,oPanel,,,,,T,,, , ) //"Formas cadastradas no ECF"
3175
3176 oBrowseECF := TCBrowse():New( aPosObj[02,01] , aPosObj[02,02], aObjects[02,01], aObjects[02,02], ,,,oPanel,,,,,,F,,,T,,,F,,, )
3177 oBrowseECF:SetArray(aFormaECF)
3178 oBrowseECF:AddColumn(TCColumn():New(STR0133,{|| aFormaECF[oBrowseECF:nAt] },"",,,,"LEFT",CalcFieldSize("C",Len(cDescr),0,"@!",STR0133),.F.,.F.,,,,))
//"Descrição"###"Descrição"
3179
3180 tSay():New( aPosObj[01, 1], nLargB + 10 , { || STR0159 } ,oPanel,,,,,T,,, , ) //"Formas de Pagamento do sistema"
3181 oBrowsePtr := TCBrowse():New( aPosObj[02,01] , aObjects[02,01]+13, nCol-(aObjects[02,01]+13), aObjects[02,02],,,oPanel,,,,,,F,,,T,,,F,,, )
3182 oBrowsePtr:SetArray(aFormaPtr)
3183 oBrowsePtr:AddColumn(TCColumn():New(STR0104,{|| IIF(aFormaPtr[oBrowsePtr:nAt,01] == "1", oBmp1, oBmp2) },"",,,,"LEFT",CalcFieldSize("C",2,0,"@!",STR0104),.T.,.F.,,,,))
//"Status"###"Status"
3184 oBrowsePtr:AddColumn(TCColumn():New(STR0160,{|| aFormaPtr[oBrowsePtr:nAt,02] },"@!",,,,"LEFT",CalcFieldSize("C",Len(cDescr),0,"@!",STR0160),.F.,.F.,,,,))
//"Forma"###"Forma"
3185 oBrowsePtr:AddColumn(TCColumn():New(STR0135,{|| aFormaPtr[oBrowsePtr:nAt,03] },"@!",,,,"LEFT",CalcFieldSize("C",30,0,"@!",STR0135),.F.,.F.,,,,)) //"Chave"###"Chave"
3186 oBrowsePtr:AddColumn(TCColumn():New(STR0133,{|| aFormaPtr[oBrowsePtr:nAt,04] },"@!",,,,"LEFT",CalcFieldSize("C",60,0,"@!",STR0161),.F.,.F.,,,,))
//"Descrição"###"Descricao"
3187 oBrowsePtr:bChange := {|| IIF(ValType(oGet1) == "0", ( cDescr := aFormaPtr[oBrowsePtr:nAt,02], oGet1:Refresh(), oBtn1:Refresh() ) , NIL ) }
3188 oBrowsePtr:BHeaderClick := { || STFWzShowLg( { {"BR_VERDE", STR0109},; //"Programado"
3189 {"BR_VERMELHO", STR0110}; //"Não programado"
3190 }, STR0162 ) } //"Legenda Formas Pagto"
3191
3192 @aPosObj[03,01], 5 to aPosObj[08, 1] ,nCol 0F oPanel PIXEL
3193 tSay():New( aPosObj[04, 1], aPosObj[04, 2]+15 , { || STR0163 } ,oPanel,,,,,T,,, , ) //"Programar Forma de Pagamento"
3194
3195 //Linha 5 - label da Programação
3196 nTam := CalcFieldSize("C",Len(cDescr),0,"@!",STR0160) //Tamanho do Get Forma //"Forma"
3197 tSay():New( aPosObj[05, 1], aPosObj[05, 2]+15 , { || STR0160 } ,oPanel,,,,,T,,, , ) //"Forma"
3198
3199
3200 //Linha 6 - Gets da Programação
3201 nCol :=aPosObj[06, 2] + 15
3202 oGet1 := TGet():New( aPosObj[06, 1],nCol,{ |U| Iif(PCOUNT() > 0, (cDescr := U, oBtn1:Refresh()), cDescr)},oPanel, nTam,10,"@!",{|| !Empty(cDescr) },,,, .T,,, .T,,, .T.,
, .F., .F., .f., .F. ,,"cDescr",,,,T.,)
3203 nCol += nTam + (nSepComp *2)
3204 oBtn1 := TButton():New(aPosObj[06, 1],nCol, STR0112,oPanel,{|| IIF( STFWzPrFg( @cDescr, @aFormaECF, @aFormaPtr, @cLog, ; //"Programar"
3205 nEspacos),;
3206 (oBrowseECF:SetArray(aFormaECF), oBrowsePtr:SetArray(aFormaPtr)), ), ;
3207 oGet1:Refresh() },aObjects[06,01],aObjects[06,02],,,.F.,.T.,.F.,.F.,{ || !Empty(cDescr)
},,.F. )
3208 nCol += aObjects[06,01] + (nSepComp *2)
3209 TButton():New(aPosObj[06, 1],nCol, STR0113,oPanel,{|| IIF( STFWzPrFg( "", @aFormaECF, @aFormaPtr, @cLog,; //"Programar Tudo"
3210 nEspacos),;
3211 (oBrowseECF:SetArray(aFormaECF), oBrowsePtr:SetArray(aFormaPtr)), ), ;
3212 oGet1:Refresh() , oBtn1:Refresh()),aObjects[06,01],aObjects[06,02],,,.F.,.T.,.F.,.F.,{ || .t.
},,.F. )
3213

```

```

3214 Return
3215
3216 //-----
3217 /*/{Protheus.doc} STFWzPrFg
3218 Funcao para programacao das formas de pagamento
3219 @param cDescr      Forma de pagamento
3220 @param aFormaECF    Forma de pagamento da impressora
3221 @param aFormaPtr    Formas de pagamento do Protheus
3222 @param cLog         Log de execucao da rotina
3223 @param nEspacos     Espacos do log
3224 @return lPross      Programacao realizada com sucesso
3225 @obs
3226 @sample
3227 */
3228 //-----
3229 Function STFWzPrFg( cDescr, aFormaECF, aFormaPtr, cLog,;
3230                   nEspacos)
3231
3232   Local aRet := {}
3233   Local lPross := .T.
3234   Local aDados := {"17", space(30)}
3235   Local lTudo := .F.
3236   Local cTotaliz := ""
3237   Local nPosT := 0
3238   Local nPosP := 0
3239   Local cValor := ""
3240   Local nC := 0
3241   Local nPos := 0
3242   Local cTxtTmp := ""
3243   Local cLogT := ""
3244   Local nEspacoT := 0
3245
3246   DEFAULT cDescr := ""
3247   DEFAULT aFormaECF := {}
3248   DEFAULT aFormaPtr := {}
3249   DEFAULT cLog := ""
3250   DEFAULT nEspacos := 5
3251
3252   lTudo := Empty(cDescr)
3253   nEspacoT := nEspacos + 5
3254
3255
3256   aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
3257                        "STPrinterStatus" ,; // Nome do evento
3258                        {"8" ,;
3259                        " " } )
3260
3261
3262   If Len(aRet) > 0 .AND. aRet[1] == 10
3263     //Redução Z Pendente
3264     cTxtTmp := STR0164 //"Redução Z Pendente. Para a programação de Forma de Pagamento, faz-se necessario que seja emitida a redução Z. "
3265     STFMessage("Z Reduction", "STOP", cTxtTmp)

```

```

3266     STFSHShowMessage("Z Reduction")
3267     lPross := .F.
3268     cLogT := CRLF + space(nEspacoT) + cTxtTmp
3269 Else
3270     aRet := STFFireEvent( ProcName(0)                ,;      // Nome do processo
3271                          "STPrinterStatus"           ,;      // Nome do evento
3272                          aDados )
3273     If Len(aDados) < 2 .OR. Len(aRet) = 0 .OR. aRet[1] <> 0
3274         cTxtTmp := STR0165 //"Falha ao capturar Venda Bruta diária."
3275         STFSHMessage("Printer Status", "STOP", cTxtTmp)
3276         STFSHShowMessage("Printer Status")
3277         lPross := .F.
3278         cLogT := CRLF + space(nEspacoT) + cTxtTmp
3279     Else
3280         If Val(aDados[2]) > 0
3281             cTxtTmp := STR0166 //"Já foram realizadas vendas para este dia."
3282             STFSHMessage("Printer Status", "STOP", cTxtTmp)
3283             STFSHShowMessage("Printer Status")
3284             cLogT := CRLF + space(nEspacoT) + cTxtTmp
3285             lPross := .F.
3286         EndIf
3287     EndIf
3288 EndIf
3289
3290 If lPross
3291     If !lTudo
3292
3293         If ( nPost := aScan( aFormaECF, { |t| AllTrim(t) == AllTrim(cDescr) } ) ) > 0
3294             cTxtTmp := STR0167 //"Totalizador já programado"
3295             STFSHMessage("FORMPROG","STOP", cTxtTmp) //STR
3296
3297             STFSHShowMessage("FORMPROG")
3298             lPross := .F.
3299             cLogT := CRLF + space(nEspacoT) + cTxtTmp
3300
3301         EndIf
3302
3303         If lPross
3304             //Prossegue então programa o totalizador
3305
3306             aRet := STFFireEvent( ProcName(0)                ,;      // Nome do processo
3307                                  "STRecCondPay"              ,;      // Nome do evento
3308                                  {cDescr})
3309
3310             If Len(aDados) < 2 .OR. Len(aRet) = 0 .OR. aRet[1] <> 0
3311                 cTxtTmp := STR0168 + cDescr //"Falha ao programar forma de pagamento "
3312                 STFSHMessage("FORMPROGERR","STOP", cTxtTmp)
3313                 STFSHShowMessage("FORMPROGERR")
3314                 lPross := .F.
3315                 cLogT := CRLF + space(nEspacoT) + cTxtTmp
3316             EndIf
3317         EndIf
3318     EndIf

```

```

3318         Else
3319
3320             nC := 0
3321             Do While (nC := nC + 1) <= Len(aFormaPtr)
3322                 If AllTrim(aFormaPtr[nC, 02]) == AllTrim(cDescr) .AND. aFormaPtr[nC,1] == "1"
3323                     //Como pode existir vários totalizadores iguais, atualiza tudo
3324                     aFormaPtr[nC, 01] := "2" //Programado
3325                 EndIf
3326             EndDo
3327
3328             aAdd(aFormaECF, cDescr)
3329         EndIf
3330
3331     EndIf
3332
3333 Else
3334     Do While (nPos := nPos + 1) <= Len(aFormaPtr)
3335
3336         If aFormaPtr[nPos, 01] == "1" .AND. ;
3337             aSCan(aFormaECF, { |e| AllTrim(e) == AllTrim(aFormaPtr[nPos, 02])} ) == 0 // VERIFICAR SE O TOTALIZADOR NÃO EXISTE A FORMA ecf //Não programado
3338             aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
3339                 "STRecCondPay" ,; // Nome do evento
3340                 { AllTrim(aFormaPtr[nPos, 02])} )
3341             lPross := Len(aRet) > 0 .AND. aRet[1] == 0
3342
3343         If lPross
3344             aAdd(aFormaECF, AllTrim(aFormaPtr[nPos, 02]))
3345             cDescr := AllTrim(aFormaPtr[nPos, 02] )
3346             nC := nPos
3347             Do While (nC := nC + 1) <= Len(aFormaPtr)
3348                 If AllTrim(aFormaPtr[nC, 02]) == AllTrim(cDescr) .AND. aFormaPtr[nC,1] == "1"
3349                     //Como pode existir vários totalizadores iguais, atualiza tudo
3350                     aFormaPtr[nC, 01] := "2" //Programado
3351                 EndIf
3352             EndDo
3353
3354         Else
3355
3356             cTxtTmp := STR0169 + aFormaPtr[nPos, 02] //"Falha ao programar Forma de pagamento: "
3357             STFMessage("FORMPROGERR","STOP",cTxtTmp ) //STR
3358
3359             STFS showMessage("FORMPROGERR")
3360             cLogT := CRLF + space(nEspacoT) + cTxtTmp
3361             Exit
3362         EndIf
3363     EndDo
3364
3365 EndIf
3366
3367 EndDo
3368
3369

```

```

3370      EndIf
3371
3372
3373      EndIf
3374      cDescr := space(20)
3375
3376      cLog += CRLF + space(nEspacos) + IIF(lPross,"",STR0072) + STR0170 + IIF(!lTudo," ", STR0120) + STR0171 + cLogT    //[ERRO]###" - Programando forma###s "###" de
        pagamento"
3377
3378 Return lPross
3379
3380 //-----
3381 /*{Protheus.doc} STFwzPrHP
3382 Interface para programacao do horário de verao
3383 @param cLG_CODIGO    Codigo da estacao
3384 @param cImpfisc      Impressora Fiscal
3385 @param lConecta      Impressora Conectada
3386 @param oDLG          Janela
3387 @param cLog          Log de execucao da rotina
3388 @param nEspacos      Espacos do log
3389 @return lRet         Programacao realizada com sucesso
3390 @obs
3391 @sample
3392 */
3393 //-----
3394 Function STFwzPrHP(cLG_CODIGO, cImpFisc, lConecta, oDLG, ;
3395                  nPanel,      cLog,      nEspacos)
3396
3397 Local lRet      := .T.      //Retorno da rotina
3398 Local aDados   := {}       //Dados retornados pelo ECF
3399 Local dDataPrt := CtoD("")  //Data do Protheus
3400 Local cHoraPrt := ""       //Hora do Protheus
3401 Local dDataECF := CtoD("")  //Data do ECF
3402 Local cHoraECF := ""       //Hora do ECF
3403 Local nDiff    := 0        //Diferença (em minutos)
3404 Local oGet1    := NIL      //Item a ser programado
3405 Local oGet2    := NIL      //Item a ser programado
3406 Local oGet3    := NIL      //Item a ser programado
3407 Local nAdic    := 1        //Adiciona/Avança
3408 Local aObjects := {}       //Array do objetos
3409 Local aPosObj  := {}       //Posicao dos objetos
3410 Local nSepComp := 3        //Separacao de componentes
3411 Local oPanel   := NIL      //Panel
3412 Local nRow     := 0        //Linha
3413 Local nCol     := 0        //Coluna
3414 Local cDtHrECF := ""       //Data/Hora do ECF
3415 Local cDtHrPrt := ""       //Data/Hora do Protheus
3416 Local nCol2    := 0        //Coluna
3417 Local oAdic    := NIL      //Adiciona
3418 Local oBtn1    := NIL      //Botao1
3419 Local cLogT    := ""       //log temporario
3420 Local nEspacoT := 0        //espaco temporario

```

```

3421
3422 DEFAULT cLG_CODIGO := SLG->(CriaVar("LG_CODIGO")) //Codigo da Estacao
3423 DEFAULT oDlg := NIL //Janela
3424 DEFAULT cImpFisc := "" //Impressora fiscal
3425 DEFAULT lConecta := .F. //Impressora Conectada
3426 DEFAULT oDlg := NIL //Janela
3427 DEFAULT nPanel := 1 //Panel atual
3428 DEFAULT cLog := "" //Log
3429 DEFAULT nEspacos := 5 //Espaco temporario
3430
3431 nEspacoT := nEspacos + 5
3432 If !lConecta
3433     If Empty(cImpFisc)
3434
3435         cImpFisc := STFGetStat("IMPFISC")
3436     EndIf
3437     If Empty(cPorta)
3438         cPorta := STFGetStat("PORTIF")
3439     EndIf
3440
3441     lConecta := lRet := STFWzCIF(cImpFisc, cPorta, @cLogT, nEspacoT)
3442
3443 EndIf
3444
3445 cLog += cLogT
3446
3447 lRet := lRet .AND. STFWzLdHC( @dDataECF, @cHoraECF, @dDataPrt, @cHoraPrt, ;
3448                             @cDtHrECF, @cDtHrPrt, @nAdic, oGet1, ;
3449                             oGet2, oGet3, oBtn1, ,;
3450                             @cLog, nEspacos)
3451
3452
3453 If lRet
3454     //Monta os panels
3455     oPanel := oDlg:GetPanel(nPanel)
3456
3457     nRow := Int( oPanel:nHeight/2) //Linha do Painei
3458     nCol := Int( oPanel:nWidth/ 2)
3459
3460     oPanel := TScrollBox():New( oPanel, 0,0,nRow,nCol)
3461     nCol := Int( nCol *.97 )
3462
3463     nTam := CalcFieldSize("C",19,0,"@!",STR0172) //1 //"Data/Hora do ECF"
3464     aAdd(aObjects, { nTam, 6, .F., .F.}) //Impar Say //1
3465     aAdd(aObjects, { nTam, 10, .F., .F.}) //Get2
3466
3467     nTam := CalcFieldSize("C",19,0,"@!",STR0173) //1 //"Data/Hora da Estação"
3468     aAdd(aObjects, { nTam, 6, .F., .F.}) //Impar Say //3
3469     aAdd(aObjects, { nTam, 10, .F., .F.}) //Get4
3470     nTam := CalcFieldSize("N",2,0,"@!",STR0174) //1 //"Diferença (em minutos)"
3471     aAdd(aObjects, { nTam, 6, .F., .F.}) //Impar Say //5
3472     aAdd(aObjects, { nTam, 10, .F., .F.}) //Impar Say //6

```

```

3473      aAdd(aObjects, { 10, 6, .F., .F.}) //Blank Line 7
3474      aAdd(aObjects, { 10, 1, .F., .F.}) //Blank Line 8
3475
3476      nTam := CalcFieldSize("C",Len(STR0175),0,"@!",STR0175) //Programar Horário de Verão####Programar Horário de Verão"
3477      aAdd(aObjects, { nTam, 12, .F., .F.}) //9
3478      aAdd(aObjects, { 55, 15, .F., .F.}) //10 botão programar /Adiciona/Avança /Refresh
3479      aAdd(aObjects, { 10, 10, .F., .F.}) //Blank Line11 - ATRASAR
3480      aAdd(aObjects, { 10, 10, .F., .F.}) //Blank Line12 - LINE
3481      aAdd(aObjects, { 10, 6, .F., .F.}) //Blank Line 13
3482      aAdd(aObjects, { 80, 15, .F., .F.}) //14 Imprimir página de teste
3483
3484
3485      aPosObj := MsObjSize( { 0 , 15 , nRow ,nCol, nSepComp, 3,3,3}, aObjects, .T., .F.)
3486
3487
3488      //Monta o Label dos Horários
3489      tSay():New( aPosObj[01, 01], aPosObj[01,02] , { || STR0172 } ,oPanel,,,,,.T.,, , ) //"Data/Hora do ECF"
3490      oGet1 := TGet():New( aPosObj[02, 01],aPosObj[02, 02],{ |U| IIf(PCOUNT() > 0, cDtHrECF := U, cDtHrECF) },oPanel, aObjects[02,01],aObjects[02,02],",",{|| .t. },,,,
.T.,, .T.,, .T.,, , .F., .F.,, .T., .F.,,"cDtHrECF",,,,,.T.,)
3491
3492      tSay():New( aPosObj[03, 01], aPosObj[03,02] , { || STR0173 } ,oPanel,,,,,.T.,, , ) //"Data/Hora da Estação"
3493      oGet2 := TGet():New( aPosObj[04, 01],aPosObj[04, 02],{ |U| IIf(PCOUNT() > 0, cDtHrPrt := U, cDtHrPrt) },oPanel, aObjects[04,01],aObjects[04,02],",",{|| .t. },,,,
.T.,, .T.,, .T.,, , .F., .F.,, .T., .F.,,"cDtHrPrt",,,,,.T.,)
3494
3495      tSay():New( aPosObj[05, 01], aPosObj[05,02] , { || STR0174 } ,oPanel,,,,,.T.,, , ) //"Diferença (em minutos)"
3496      oGet3 := TGet():New( aPosObj[06, 01],aPosObj[06, 02],{ |U| IIf(PCOUNT() > 0, nDiff := U, nDiff) } ,oPanel, aObjects[06,01],aObjects[06,02],"@ 99",{|| .t. },,,, .T.,,
.T.,, .T.,, , .F., .F.,, .T., .F.,,"nDiff",,,,,.T.,)
3497
3498
3499      @aPosObj[08,01], 5 to aPosObj[13, 1] ,nCol OF oPanel PIXEL
3500      tSay():New( aPosObj[09, 1], aPosObj[09, 02]+15 , { || STR0176 } ,oPanel,,,,,.T.,, , ) //"Programar Horario de Verão"
3501      nCol2 := aPosObj[10, 2] +15
3502      nTam := 60
3503
3504
3505      oAdic := TRadMenu():New(aPosObj[10, 1], nCol2, {STR0177, STR0178, STR0179}, { |u| Iif (PCOUNT()==0,nAdic,nAdic:=u)}, oPanel, ,; //"Adianta"####Atrasa"####Não Programar"
3506      /* { || oBtn1:Refresh() }*/ , , , , , ;
3507      nTam, 10,{ || oBtn1:Refresh(), .T. }, , , .T.)
3508      // @ aPosObj[10, 1], nCol2 RADIO oAdic VAR nAdic SIZE nTam, 10 PROMPT "Adianta", "Atrasa", "Não Programar" OF oPanel ON CHANGE oBtn1:Refresh()
3509      nCol2 += nTam + ( nSepComp * 2)
3510      oBtn1 := TButton():New(aPosObj[10, 1],nCol2, STR0112,oPanel,{|| STFWzPrHg(@dDataECF, @cHoraECF, @dDataPrt, @cHoraPrt, ; //"Programar"
3511      @cDtHrECF , @cDtHrPrt , @nAdic, oGet1, ;
3512      oGet2, oGet3, oBtn1, oAdic, ;
3513      @cLog, nEspacos) },aObjects[10,01],aObjects[10,02],,,,F.,.T.,.F.,.F.,{ || nAdic <= 2
},,.F. )
3514      nCol2 += aObjects[10,01] + (nSepComp *2)
3515      TButton():New(aPosObj[10, 1],nCol2, STR0180,oPanel,{|| STFWzLdHC( @dDataECF, @cHoraECF, @dDataPrt, @cHoraPrt, ; //"Recapturar Dados"
3516      @cDtHrECF , @cDtHrPrt , @nAdic, oGet1, ;
3517      oGet2, oGet3, oBtn1, oAdic, ;
3518      @cLog, nEspacos) },aObjects[10,01],aObjects[10,02],,,,F.,.T.,.F.,.F.,{ || .t. },,.F. )
3519
3520

```

```

3521         TButton():New(aPosObj[14, 1],aPosObj[14, 2], STR0181,oPanel,{|| STFWzPrTst(@cLog, nEspacos) },aObjects[14,01],aObjects[14,02],,,.F.,.T.,.F.,,.F.,{ || .t. },,.F. )
3522         // "Imprimir Página de Teste"
3523
3524     EndIf
3525
3526
3527 Return lRet
3528
3529 //-----
3530 /*{Protheus.doc} STFWzLdHIF
3531 Captura a data/hora do ECF
3532 @param dData      Data do ECF
3533 @param cHora      Hora do ECF
3534 @param cDataHora  Data/Hora do ECF
3535 @param cLog       Log de execucao da rotina
3536 @param nEspacos   Espacos do log
3537 @return lRet      Programacao realizada com sucesso
3538 @obs
3539 @sample
3540 */
3541 //-----
3542 Function STFWzLdHIF(dData, cHora, cDataHora, cLog, ;
3543                    nEspacos)
3544 Local aDados      := {} //Dados do ECF
3545 Local aRet        := .T. //Retorno do Strategy
3546 Local lRet        := .T. //Retorno da funcao
3547
3548 DEFAULT dData      := CtoD("") //Data do ECF
3549 DEFAULT cHora      := "" //Hora do ECF
3550 DEFAULT cDataHora  := "" //Data/Hora do ECF
3551 DEFAULT cLog       := "" //log da rotina
3552 DEFAULT nEspacos   := 5 //espacos
3553
3554 cDataHora := ""
3555
3556 //Executa a Captura da Data/Hora do ECF
3557
3558 aDados := {"2", space(20)}
3559 aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
3560                      "STPrinterStatus"
3561                      aDados )
3562                      ,; // Nome do evento
3563
3564 If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 1 .AND. !Empty(aDados[2])
3565     dData := CtoD( AllTrim(aDados[2]))
3566
3567     aDados := {"1", space(20)}
3568     aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
3569                         "STPrinterStatus"
3570                         aDados )
3571                         ,; // Nome do evento
3572
3573     If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 1 .AND. !Empty(aDados[2])
3574         cHora := AllTrim(aDados[2])

```

```

3572         cLog += CRLF + space(nEspaces) + STR0182 //" - Capturando Data/Hora do ECF"
3573     Else
3574         lRet := .F.
3575         cLog += CRLF + space(nEspaces) + STR0183 //" [ERR0] - Capturando Hora do ECF"
3576     EndIf
3577 Else
3578     lRet := .F.
3579     cLog += CRLF + space(nEspaces) + STR0184 //" [ERR0] - Capturando Data do ECF"
3580 EndIf
3581
3582 If !lRet
3583     STFMessage("DATAHORA","STOP",STR0185) //STR //"Falha ao capturar Data/Hora da Impressora"
3584
3585     STFShowMessage("DATAHORA")
3586     lRet := .F.
3587 Else
3588     cDataHora := DtoC(dData) + " " + cHora
3589 EndIf
3590
3591 Return lRet
3592
3593 Function STFWzLdHP( dData, cHora, cDataHora )
3594     dData := GetRmtDate()
3595     cHora := GetRmtTime()
3596
3597     cDataHora := DtoC(dData) + " " + cHora
3598 Return .T.
3599
3600
3601 //-----
3602 /*{Protheus.doc} STFWzLdHD
3603 Calcula a diferenca entra data do protheus e a data do remote
3604 @param dDataECF      Data do ECF
3605 @param cHoraECF      Hora do ECF
3606 @param dDataPtr      Data do Protheus
3607 @param cHoraPtr      Hora do Protheus
3608 @param nAdic         Adiciona/Atrasa o horario
3609 @return nDiff        Diferenca de horario
3610 @obs
3611 @sample
3612 */
3613 //-----
3614 Function STFWzLdHD(dDataECF, cHoraECF, dDataPtr, cHoraPtr, nAdic)
3615 Local nX      := 0      //Variavel temporaria
3616 Local nHoraPtr := 0      //Hora do Protheus
3617 Local nHoraECF := 0      //Hora do ECF
3618 Local nDiff    := 0      //Diferenca
3619
3620 nX := ( dDataECF - dDataPtr ) * 24 * 60
3621 nHoraPtr := (Val (SubStr (cHoraPtr, 1, 2))*60) + Val (SubStr (cHoraPtr, 4, 2))
3622 nHoraECF := (Val (SubStr (cHoraECF, 1, 2))*60) + Val (SubStr (cHoraECF, 4, 2))
3623

```

```

3624      nDiff := (nX + nHoraECF - nHoraPtr)
3625
3626      nAdic := IIF( nDiff <= -60, 1, IIF(nDiff >= 60, 2, 3) )
3627
3628 Return nDiff
3629
3630 //-----
3631 /*{Protheus.doc} STFwzLdHC
3632 Captura da Data/Hora do ECF e do sistema
3633 @param dDataECF      Data do ECF
3634 @param cHoraECF      Hora do ECF
3635 @param dDataPtr      Data do Protheus
3636 @param cHoraPtr      Hora do Protheus
3637 @param cDtHrECF      Data/Hora do ECF
3638 @param cDtHrPtr      Data/Hora do Protheus
3639 @param nAdic         Adiciona/Atrasa o horario
3640 @param oGet1         Objeto Get1
3641 @param oGet2         Objeto Get2
3642 @param oGet3         Objeto Get3
3643 @param oBtn1         Botao 1
3644 @param oAdic         Adiciona o Horario
3645 @param cLog          Log de execucao da rotina
3646 @param nEspacos      Espacos do Log
3647 @return lRet         Retorno da funcao
3648 @obs
3649 @sample
3650 */
3651 //-----
3652 Function STFwzLdHC( dDataECF, cHoraECF, dDataPtr, cHoraPtr, ;
3653                   cDtHrECF, cDtHrPtr, nAdic, oGet1, ;
3654                   oGet2, oGet3, oBtn1, oAdic, ;
3655                   cLog, nEspacos)
3656
3657 Local lRet      := .T. //Retorno da rotina
3658 Local cLogT     := ""  //log temporio da rotina
3659 Local nEspacoT  := 0   //espaco temporario
3660
3661 DEFAULT dDataECF := CtoD("") //Data do ECF
3662 DEFAULT cHoraECF := ""      //Hora do ECF
3663 DEFAULT dDataPtr := CtoD("") //Data do Protheus
3664 DEFAULT cHoraPtr := ""      //Hora do Protheus
3665 DEFAULT cDtHrECF := ""      //Data/Hora do ECF
3666 DEFAULT nAdic    := 1       //Adiciona
3667 DEFAULT oGet1    := NIL     //Objeto Get1
3668 DEFAULT oGet2    := NIL     //Objeto Get2
3669 DEFAULT oGet3    := NIL     //Objeto Get3
3670 DEFAULT oBtn1    := NIL     //Botao1
3671 DEFAULT oAdic    := NIL     //Adiciona
3672 DEFAULT cLog     := ""      //Log de execucao da rotina
3673 DEFAULT nEspacos := 5       //Espacos de execucao da rotina
3674
3675      dDataECF := dDataPtr := CTod("")

```

```

3676   cHoraECF := cHoraPrt := cDtHrECF := cDtHrPrt := ""
3677   nAdic := 0
3678
3679   nEspacoT := nEspacos + 5
3680
3681   If (lRet := STFWzLdHIF( @dDataECF, @cHoraECF, @cDtHrECF, @cLogT, ;
3682                         nEspacoT) )
3683
3684       If (lRet := STFWzLdHP( @dDataPrt, @cHoraPrt, @cDtHrPrt ) )
3685           nDiff := STFWzLdHD(dDataECF, cHoraECF, dDataPrt, cHoraPrt, @nAdic)
3686       Else
3687           dDataECF := dDataPrt := CTod("")
3688           cHoraECF := cHoraPrt := cDtHrECF := cDtHrPrt := ""
3689           nAdic := 0
3690
3691       EndIf
3692
3693   Else
3694       dDataECF := CtoD("")
3695       cHoraECF := cDtHrECF := ""
3696   EndIf
3697
3698
3699   If oGet1 <> NIL
3700       oGet1:Refresh()
3701       oGet2:Refresh()
3702       oGet3:Refresh()
3703       oBtn1:Refresh()
3704       oAdic:Refresh()
3705   EndIf
3706
3707   cLog += CRLF + space(nEspacos) + IF(lRet, "", STR0072) +STR0186 + cLogT //"[ERR0]"###" - Capturando Data/Hora do ECF e Sistema"
3708
3709   Return lRet
3710
3711 //-----
3712 /*/{Protheus.doc} STFWzLdHC
3713 Programacao de Horario de Verão
3714 @param dDataECF      Data do ECF
3715 @param cHoraECF      Hora do ECF
3716 @param dDataPtr      Data do Protheus
3717 @param cHoraPtr      Hora do Protheus
3718 @param cDtHrECF      Data/Hora do ECF
3719 @param cDtHrPtr      Data/Hora do Protheus
3720 @param nAdic         Adiciona/Atrasa o horario
3721 @param oGet1         Objeto Get1
3722 @param oGet2         Objeto Get2
3723 @param oGet3         Objeto Get3
3724 @param oBtn1         Botao 1
3725 @param oAdic         Adiciona o Horario
3726 @param cLog          Log de execucao da rotina
3727 @param nEspacos      Espacos do Log

```

```

3728 @return lRet      Retorno da funcao
3729 @obs
3730 @sample
3731 /*/
3732 //-----
3733 Function STFWzPrHg( dDataECF, cHoraECF, dDataPrt, cHoraPrt, ;
3734                   cDtHrECF , cDtHrPrt , nAdic,   oGet1, ;
3735                   oGet2,   oGet3,   oBtn1,  oAdic,;
3736                   cLog,    nEspacos)
3737
3738 Local lRet      := .T. //Retorno da funcao
3739 Local aRet      := {}  //Retorno do strategy
3740 Local aDados    := {}  //Dados de retorno do ECF
3741 Local cLogT     := ""   //Log temporario
3742 Local nEspacoT  := 5    //Espacos temporarios
3743
3744 DEFAULT dDataECF := CtoD("") //Data do ECF
3745 DEFAULT cHoraECF := ""       //Hora do ECF
3746 DEFAULT dDataPrt := Ctod("") //Data do Protheus
3747 DEFAULT cHoraPrt := ""       //Hora do Protheus
3748 DEFAULT cDtHrECF := ""       //Data/Hora do ECF
3749 DEFAULT cDtHrPrt := ""       //Data/Hora do Protheus
3750 DEFAULT nAdic    := 1        //Adiciona/Retrocede
3751 DEFAULT oGet1    := NIL      //Objeto Get1
3752 DEFAULT oGet2    := NIL      //Objeto Get2
3753 DEFAULT oGet3    := NIL      //Objeto Get3
3754 DEFAULT oBtn1    := NIL      //Botao1
3755 DEFAULT oAdic    := NIL      //Objeto Adiciona
3756 DEFAULT cLog     := ""       //Log
3757 DEFAULT nEspacos := 5        //Espacos
3758
3759     nEspacoT := nEspacos + 5
3760
3761     If nAdic <> 3
3762
3763         aDados := { IIF( nAdic == 1,'+', '-' )}
3764
3765         aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
3766                             "STHourSummerAdjust" ,; // Nome do evento
3767                             aDados )
3768
3769         If Len(aRet) = 0 .OR. aRet[1] <> 0
3770             STFMessage("HRVERA0","STOP",STR0187) //STR //"Falha ao programar horário de verão"
3771
3772             STFShowMessage("HRVERA0")
3773             lRet := .F.
3774         Else
3775             lRet := STFWzLdHC( @dDataECF, @cHoraECF, @dDataPrt, @cHoraPrt, ;
3776                               @cDtHrECF , @cDtHrPrt ,@nAdic, oGet1,;
3777                               oGet2, oGet3, oBtn1, ,;
3778                               @cLogT, nEspacoT)
3779         EndIf

```

```

3780     EndIf
3781
3782     cLog := CRLF + space(nEspacos) + IIF(lRet, "", STR0072) + STR0188 + cLogT //[ERR0]####" - Programando Horário de Verão"
3783 Return lRet
3784
3785 //-----
3786 /*{Protheus.doc} STFWzPrtTst
3787 Impressão de pagina de teste
3788 @param cLog      Log de execucao da rotina
3789 @param nEspacos   Espacos do Log
3790 @return lRet      Retorno da funcao
3791 @obs
3792 @sample
3793 */
3794 //-----
3795 Function STFWzPrtTst(cLog, nEspacos)
3796 Local lRet      := .T. //Retorno da rotina
3797 Local aRet      := {} //Retorno do Strategy
3798 Local aDados    := {StrTran(0emToAnsi(NoAcento(cLog)), space(5), space(1)), 1} //parametros da rotina
3799
3800
3801 DEFAULT cLog := "" //Log
3802 DEFAULT nEspacos := 5 //Espacos
3803
3804     aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
3805                          "STManagReportPrint"
3806                          aDados )
3807
3808     If Len(aRet) = 0 .OR. aRet[1] <> 0
3809         STFMessage("MANAGRPT","STOP",STR0189) //STR //"Falha ao imprimir Relatório Gerencial"
3810
3811         STFShowMessage("MANAGRPT")
3812         lRet := .F.
3813     EndIf
3814
3815
3816     cLog += CRLF + space(nEspacos) + IIF(lRet, "", STR0072) + STR0190 //[ERR0]####" - Imprimindo página de teste"
3817 Return lRet
3818
3819 //-----
3820 /*{Protheus.doc} STFLdDTEF
3821 Resultados de comunicacao TEF
3822 @param oDlg      Janela
3823 @param nPanel    Painel atual
3824 @param cLog      Log de execucao da rotina
3825 @obs
3826 @sample
3827 */
3828 //-----
3829 Function STFLdDTEF(oDlg, nPanel, cLog)
3830
3831 Local aObjects := {} //Array de objetos

```

```

3832 Local aPosObj := {}      //Posicao de objetos
3833 Local nRow := 0          //Linhas do Panel
3834 Local nCol := 0          //Colunas do panel
3835 Local oPanel := NIL      //panel
3836 Local nRow1 := 0         //Linha
3837
3838
3839 DEFAULT oDlg := NIL //Janela
3840 DEFAULT nPanel := 1      //Panel
3841 DEFAULT cLog := ""       //Log
3842
3843 oPanel := oDlg:GetPanel(nPanel)
3844
3845 nRow := Int( oPanel:nHeight/2) //Linha do Painel
3846 nCol := Int( oPanel:nWidth/ 2)
3847
3848 oPanel := TScrollBar():New( oPanel, 0,0,nRow,nCol)
3849 nCol := Int( nCol *.97 )
3850 nRow1 := Int( nRow *.93)
3851
3852 aObjects := { {CalcFieldSize("C",110,0),nRow1, .F., .F.} }
3853 aPosObj2 := MsObjSize( { 0 , 15 , nRow1 ,nCol, 2, 3,3,3}, aObjects, .T., .F.)
3854
3855 cLog := UPPER(STR0191) + cLog //"Resultado da transação de Teste de Comunicação TEF "
3856 TMultiGet():New(aPosObj2[1, 1],aPosObj2[1, 2],{|| Iif(PCOUNT() > 0, cLog := U,cLog)} ,oPanel,;
3857 aObjects[1, 01],aObjects[1, 02],/*oFont*/,.T.,;
3858 /*nClrFore*/,/*nClrBack*/,/*oCursor*/,.T.,;
3859 /*cMg*/,.T.,/*lCenter*/,;
3860 /*lRight*/,.T.,/*bChange*/,;
3861 ,.F., .T.)
3862
3863 Return
3864 //-----
3865 /*{Protheus.doc} STFwzShowLg
3866 Exibe a legenda
3867 @param aLegenda Array de Legenda
3868 @param cTitle Titulo
3869 @obs
3870 @sample
3871 */
3872 //-----
3873 Function STFwzShowLg(aLegenda, cTitle)
3874 Local oDlgSelImp := NIL //Janela
3875 Local aCoord := MsAdvSize(.T.) //Coordenadas da janela
3876 Local oBMP := NIL //Objeto BMP
3877 Local nI := 0 //Variavel contatadora
3878 Local cRow := "" //Linha
3879 Local cVar := "" //Variavel
3880 Local cRes := "" //Resource
3881
3882 DEFAULT aLegenda := {} //Legenda
3883 DEFAULT cTitle := STR0144 //"Legenda"

```

```

3884
3885         oDlgSelImp := TDialog():New(000,000,aCoord[6]/6,aCoord[5]/5,0emToAnsi(cTitle),,,,,,oMainWnd,.T.) //
3886
3887         For nI := 1 to len(aLegenda)
3888
3889             cRow          := Str(010*nI)
3890             cVar           := "{ | | ' " + aLegenda[nI, 02] + "' } "
3891             cRes           := "' " + aLegenda[nI, 01] + "' "
3892
3893             TBitmap():Create ( oDlgSelImp, &cRow, 003,10, 10, &cRes, , .t., , ,.F., .F., , , , .T., , , , )
3894
3895             TSay():New( &cRow,20,&cVar,oDlgSelImp,,,F.,.F.,.F.,.T.,CLR_BLACK,CLR_WHITE,50,030)
3896         Next
3897
3898         oDlgSelImp:Activate(,,,T.)
3899
3900 Return
3901
3902 //-----
3903 /*/{Protheus.doc} STFRetrNfe
3904 Exibe a tela do Wizard no TOTVS PDV para retransmissão da nota.
3905 @return Nil , Nulo
3906 @obs
3907 @sample
3908 */
3909 //-----
3910 Function STFRetrNfe()
3911
3912 //Instancia a classe FWWizard
3913 oStepWiz:= FWWizardControl():New() // Define o tamanho do wizard ex: {600,800}
3914 oStepWiz:ActiveUISteps()
3915
3916
3917 // Retransmissão da NF-e
3918 oNewPag := oStepWiz:AddStep("1") // Primeira tela do wizard
3919 oNewPag:SetStepDescription("Retransmissão NF-e PDV On-line") //Descrição
3920 oNewPag:SetConstruction({|Panel|STFRtNfe_1(Panel)}) //Bloco de construção
3921 oNewPag:SetNextAction({|| SFTValRtrans(cL1_DOC,cL1_SERIE)}) //Bloco ao clicar no botão Próximo
3922 oNewPag:SetCancelAction({||SFTTrasMit()}) //Valida acao do cancelar
3923
3924 oNewPag := oStepWiz:AddStep("2", {|Panel|STFFinWiz(Panel)})
3925 oNewPag:SetStepDescription("Retransmissão")//Retransmissão
3926 oNewPag:SetNextAction({|| lFimWizard := SFTTrasMit(), lFimWizard })
3927 oNewPag:SetCancelWhen({||.F.})
3928
3929 //Ativa Wizard
3930 oStepWiz:Activate()
3931
3932 //Desativa Wizard
3933 oStepWiz:Destroy()
3934
3935 Return

```

```

3936
3937 //-----
3938 /*{Protheus.doc} STFRtNfe_1
3939 Exibe os campos para obter a nota que será retransmitida pelo TOTVS PDV.
3940 @param oPanel Objeto Objeto do Wizard
3941 @return Nil , Nulo
3942 @obs
3943 @sample
3944 */
3945 //-----
3946
3947 Static Function STFRtNfe_1(oPanel)
3948
3949 Local oSayPdv      := Nil //Numero da estacao PDV
3950 Local oTGetPDV     := Nil //Numero da estacao PDV
3951 Local oSaySerie    := Nil //Serie da estacao
3952 Local oTGetSerie   := Nil //Serie da Nota
3953 Local cGetSerie    := "" //Serie da nota na SL1
3954
3955 Default oPanel := Nil
3956
3957 //Documento da Nota
3958 cL1_DOC := Iif(ExistFunc("STDLastPdvOn"),STDLastPdvOn("L1_DOC",@cGetSerie),SL1->L1_DOC) //Busca a ultima numeração da nota
3959 oSayPdv := TSay():New(05,10,{|| "Número do Documento" },oPanel,,,,,T,,,100,25) //"Numero do Documento"
3960 oTGetPDV := TGet():New(15,10,{|u| if( PCount() > 0, cL1_DOC := u, cL1_DOC ) },oPanel,130,025, "@E " + Replicate("9",TamSx3("L1_DOC")[1] )
    ,,,0,,,F,,,T,,,F,,,F,,,F,,,F,,,F,,,cL1_DOC,,, )
3961
3962 //Serie da Nota
3963 cL1_SERIE := Iif(!Empty(cGetSerie),cGetSerie, SL1->L1_SERIE)
3964 oSaySerie := TSay():New(05,150,{|| "Série do Documento"},oPanel,,,,,T,,,300,25) //"Série do Documento"
3965 oTGetSerie := TGet():New( 15,150,{|u| if( PCount() > 0, cL1_SERIE := u, cL1_SERIE ) },oPanel,80,025, "@E " + Replicate("9",TamSx3("L1_SERIE")[1]
    ),,0,,,F,,,T,,,F,,,F,,,F,,,F,,,F,,,cL1_SERIE,,, )
3966
3967 Return Nil
3968
3969 //-----
3970 /*{Protheus.doc} SFTValRtrans
3971 Valida se existe a nota e se é NF-e.
3972 @param cDoc Caractere Numero da nota fiscal
3973 @param cSerie Caractere Serie da nota fiscal
3974 @return lRet , Logico, Retorna se a nota existe ou não.
3975 @obs
3976 @sample
3977 */
3978 //-----
3979 Static function SFTValRtrans(cDoc, cSerie)
3980
3981 Local lRet      := .F. //Indica se o processo pode ser concluído.
3982 Local aAreaSL1 := SL1->(Getarea()) //Area da SL1 selecionada.
3983
3984 DbSetOrder(2) //L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
3985 If DbSeek(xFilial("SL1") + cSerie + cDoc) .And. SL1->L1_IMPNF == .T.

```

```

3986     lRet := .T.
3987     LjGrvLog("SFTValCanc", "0 Documento " + cDoc + " Serie " + cSerie + " Está disponível para retransmissão. ", lRet)
3988 Else
3989     LjGrvLog("SFTValCanc", "0 Documento " + cDoc + " Serie " + cSerie + " Não foi localizado ou campo L1_IMPNEF está como .F. por não se tratar de uma NF-e", lRet)
3990 EndIf
3991
3992 If lRet
3993     LjAutoNfe(cDoc,cSerie, SL1->L1_CLIENTE, SL1->L1_LOJA) // realiza retransmissão.
3994 EndIf
3995
3996 If lRet
3997     SFTMoniTNfe(cDoc,cSerie, @lRet)
3998 EndIf
3999
4000 If lRet
4001     MsgInfo("Retransmissão realizada com sucesso !!!")
4002 Else
4003     MsgAlert("Está venda não está disponível para Retransmissão !!!")
4004     LjGrvLog("SFTValCanc", "0 Documento " + cDoc + " Serie " + cSerie + " Não foi possível retransmitir a NF-e", lRet)
4005 EndIf
4006
4007 RestArea(aAreaSL1)
4008
4009 Return lRet
4010
4011 //-----
4012 /*{Protheus.doc} SFTTrasMit
4013 Conclui o processo e limpa as mensagens da tela.
4014 @return lRet , Logico, Retorna se a nota existe ou não.
4015 @obs
4016 @sample
4017 */
4018 //-----
4019 Static Function SFTTrasMit()
4020     STFCleanInterfaceMessage()
4021 Return .T.
4022
4023 //-----
4024 /*{Protheus.doc} SFTMoniTNfe
4025 Monitoria a nota para verificar se ela foi transmitida.
4026 @param cDoc      caracter, Numero do documento da nota.
4027 @param cSerie    caracter, Numero da serie da nota.
4028 @param lRet      Logico , Retorna se a nota está autorizada.
4029 @return Nil , Nulo,
4030 @obs
4031 @sample
4032 */
4033 //-----
4034 Static Function SFTMoniTNfe(cDoc, cSerie, lRet)
4035
4036 Local cEntidade      := LjTSSIDEnt("55") //Entidade do TSS.
4037 Local aNotas         := {}              //Nota que enviaremos para função ProcMonitorDoc()

```

```

4038 Local aInfMonNFe      := {}           //Recebe o retorno da função ProcMonitorDoc()
4039 Local cCodRetNFE      := ""           //Codigo de retorno da SEFAZ.
4040
4041
4042 Aadd(aNotas, cSerie)
4043 Aadd(aNotas, cDoc)
4044 Aadd(aNotas, cDoc)
4045
4046 lRet := .F. // Somente se a nota for autorizada retorna o lRet para .T.
4047
4048 aInfMonNFe := ProcMonitorDoc(cEntidade , AllTrim( SuperGetMv("MV_SPEDURL", "") ) ,aNotas , 1 /*nTpMonitor*/, "55" /*cModelo*/, .F. /*lCte*/)
4049
4050 If len(aInfMonNFe)
4051     cCodRetNFE := aInfMonNFe[1][05]
4052     If cCodRetNFE == "100"
4053         lRet := .T.
4054     EndIf
4055 EndIf
4056
4057 Return
4058 //-----
4059 /*{Protheus.doc} STFFinWiz
4060 Say de finalização do Wizard.
4061 @param oPanel  Objeto      Painel do Wizard.
4062 @return Nil , Nulo,
4063 @obs
4064 @sample
4065 */
4066 //-----
4067 Static Function STFFinWiz(oPanel)
4068
4069 Local oSayInfo := Nil //Conclusão do processo.
4070
4071 oSayInfo := TSay():New(05,10,{|| "Procedimento de retransmissão concluído !" },oPanel,,,,.T.,,100,25) //"Procedimento de retransmissão concluído !"
4072
4073 Return Nil

```